



UIT University
Department of Computer Science
CS/SE/AI/DS
FINAL YEAR PROJECT REPORT
Batch 2023 Spring

INTELLIGENT CI/CD ORCHESTRATOR

By

Syed Muhammad Haider Ali Naqvi (23SP-032-SE)

Syeda Zainab Fatima (23SP-055-SE)

Mudassar Ghulam Sarwar (23SP-011-SE)

Syed Muhammad Saqib Hasan (23SP-013-SE)

Supervised by

Engr. Laiba Mughal

Declaration

I hereby declare that this project report is based on my original work except for citations & quotations which have been duly acknowledged. I also declare that it has not been previously & concurrently submitted for any other degree or award at UIT University or other institutions.

Student Name (Student ID): Syed Muhammad Haider Ali Naqvi (23SP-032-SE)

Signature: _____

Student Name (Student ID): Syeda Zainab Fatima (23SP-055-SE)

Signature: _____

Student Name (Student ID): Mudassar Ghulam Sarwar (23SP-011-SE)

Signature: _____

Student Name (Student ID): Syed Muhmmad Saqib Hasan (23SP-013-SE)

Signature: _____

Date: _____

Acknowledgments

All praises and thanks are due to Almighty Allah, the Most Gracious and the Most Merciful, whose blessings and guidance have enabled me to complete this project successfully. I am deeply grateful to Dr. Vali Uddin, Honorable Vice Chancellor, for providing the facilities, support, and encouragement that made this work possible. His leadership and dedication to academic excellence have been a true source of inspiration. I would also like to express my sincere thanks to Dr. Wasim, Head of the Department, for his support and guidance. Furthermore, I express my heartfelt appreciation to my project supervisor, Engr. Laiba Mughal, for her continuous guidance, valuable suggestions, and motivation throughout the final year project titled “Intelligent CI/CD Orchestrator.”

INTELLIGENT CI/CD ORCHESTRATOR

ABSTRACT

Continues integration and continues deployment (CI/CD) have important practices in modern software deployment which enable build, testing and deployment processes. Jenkins and other traditional platform face challenges in huge, large-scale workloads which include static queue scheduling ineffective resource management and manual configuration of YAML based pipelines. Due to these limitations it increased waiting time, limited system efficiency and makes it less user friendly. To solve this problem the project proposes a solution for smart CI/CD Software as a service (SaaS) platform which integrates a large language model (LLM) for chatbot to generate YAML files and an intelligent dynamic queue optimization system. The platform allows users to interact with pipelines with natural language which reduces manual effort. To dynamically reorder the job system a priority scoring based algorithm is used based on factors such as urgency, dependency, execution time which improves throughput and reduces congestion. A real time dashboard also shows monitoring and control which makes the platform more efficient and developer friendly.

Table of contents

Contents

DECLARATION.....	1
ACKNOWLEDGMENTS	3
ABSTRACT	4
TABLE OF CONTENTS	5
LIST OF TABLES	8
LIST OF FIGURES	9
LIST OF SYMBOLS AND UNITS	11
1. INTRODUCTION.....	12
2. BACKGROUND & LITERATURE REVIEW.....	14
2.1 NATURAL LANGUAGE–DRIVEN CI/CD AUTOMATION	14
2.1.1 <i>Natural Language Understanding & Intent Recognition</i>	14
2.2 CICD RUNNER QUEUE OPTIMIZATION	15
2.3 DYNAMIC QUEUE REORDERING	16
2.3.3 <i>Calculation of Execution Time</i>	16
2.3.4 <i>Urgency Mapping</i>	16
2.3.5 <i>Heap Based Sorting</i>	17
2.4 CI/CD FRAMEWORK	17
2.5 CONCLUSION.....	18
3. AIM & STATEMENT OF PROBLEM.....	19
3.1 AIM OF THE PROJECT	19
3.2 STATEMENT OF THE PROBLEM	19
3.3 SCOPE OF THE PROJECT.....	21
3.4 APPROACH TO THE PROBLEM.....	21
3.5 MONITORING & VISUALIZATION LAYER.....	22
4. HARDWARE, SOFTWARE ANALYSIS & REQUIREMENTS.....	23
4.1 INTRODUCTION	23
4.2 FACTFINDING TECHNIQUES USED.....	23
4.3 HARDWARE ANALYSIS & REQUIREMENTS	24
4.4 SOFTWARE ANALYSIS & TECHNOLOGY SELECTION.....	25
4.5 ALGORITHM & DATA STRUCTURE ANALYSIS	27
4.6 OVERALL SYSTEM STRUCTURE.....	28
4.7 FUNCTIONAL REQUIREMENTS IN USER STORY FORMAT	28
4.8 NONFUNCTIONAL REQUIREMENTS.....	29
4.9 CHAPTER SUMMARY	29
5. SOFTWARE DESIGN AND MODELING.....	33
5.1 INTRODUCTION	33
5.2 CODING ARCHITECTURE: THE LAYERED ARCHITECTURE	33

5.3	REVISED ACTOR USE CASE DIAGRAM	35
5.4	UC-01: AUTHENTICATE AND MANAGE ACCESS	37
5.5	UC-02: SUBMIT NL CI/CD COMMAND	41
5.6	UC-03: GENERATE AND VALIDATE YAML PIPELINE.....	44
5.7	UC-04: SUBMIT PRIORITIZED JOB.....	47
5.8	UC-05: OPTIMIZE DYNAMIC QUEUE (POC CORE).....	51
5.9	UC-06: DISPATCH JOB TO RUNNER	56
5.10	UC-07: EXECUTE PIPELINE STAGES.....	58
5.11	UC-08: MONITOR DASHBOARD AND ANALYZE PERFORMANCE	61
5.12	LANDING, MAIN AND HOME INTERFACE.....	64
5.13	COMPLETE SYSTEM CLASS DIAGRAM.....	65
5.14	COMPLETE DATABASE SCHEMA.....	65
5.15	CHAPTER SUMMARY	67
6.	ALGORITHM ANALYSIS AND COMPLEXITY	69
6.1	INTRODUCTION	69
6.2	ALGORITHM 1: WEIGHTED PRIORITY SCORING.....	69
6.2.1	<i>Significance</i>	69
6.2.2	<i>Pseudocode</i>	70
6.2.3	<i>Comparison with Alternative Algorithms</i>	70
6.3	ALGORITHM 2: MAX-HEAP BASED SCHEDULING.....	71
6.3.1	<i>Significance</i>	71
6.3.2	<i>Pseudocode</i>	72
6.3.3	<i>Comparison with Alternative Algorithms</i>	72
6.4	ALGORITHM 3: DEPENDENCY GROUPING WITH UNION-FIND.....	73
6.4.1	<i>Significance</i>	73
6.4.2	<i>Pseudocode</i>	74
6.4.3	<i>Comparison with Alternative Algorithms</i>	74
6.5	ALGORITHM 4: EXECUTION TIME ESTIMATION BY SIMILARITY MATCHING..	75
6.5.1	<i>Significance</i>	75
6.5.2	<i>Pseudocode</i>	76
6.5.3	<i>Comparison with Alternative Algorithms</i>	77
6.6	END-TO-END COMPLEXITY OF THE SCHEDULING PIPELINE.....	78
6.7	CHAPTER SUMMARY	78
12.	ACHIEVEMENTS.....	80
12.1	OVERVIEW	80
12.2	ACHIEVEMENTS OF MILESTONE 1	80
12.3	ACHIEVEMENTS OF MILESTONE 2	80
12.1.1	<i>Design and Modeling</i>	80
12.1.2	<i>Algorithms</i>	81
12.1.3	<i>Proof of Concept Implementation</i>	81
12.1.4	<i>Skills and Process</i>	82
13.	APPENDICES.....	83
13.1	APPENDIX A: GLOSSARY OF TERMS.....	83
13.2	APPENDIX B: BASELINE VERSUS OPTIMIZED EXPERIMENT PROTOCOL	83
13.3	APPENDIX C: WORKED EXAMPLE OF THE PRIORITY SCORE.....	84
13.4	APPENDIX D: PLUGIN CONFIGURATION SAMPLE	85

13.5	APPENDIX E: TOOLS AND VERSIONS	86
13.6	APPENDIX F: KPI DEFINITIONS AND FORMULAS	86
	REFERENCES	88

List of tables

Table 5.1: Class types used in system according to the Layered Architecture.....	34
Table 5.2: Summary of the revised use cases	37
Table 6.1: Comparison of priority assignment algorithms (per job decision)	70
Table 6.2: Comparison of priority queue structures for n waiting jobs	72
Table 6.3: Comparison of grouping approaches for n jobs and m dependency edges.	74
Table 6.4: Comparison of execution time estimation methods (h historical builds, f features).....	76
Table 6.5: Combined complexity summary of the optimized scheduler	78
Table 12.1: Baseline versus optimized results of the final experiment run	81
Table 13.1: Glossary of technical terms used in this report.....	82
Table 13.2: Worked scoring example for a queue of four jobs	84
Table 13.3: Tools and versions used for the Milestone 2 POC	85
Table 13.4: Definitions of the key performance indicators compared on the dashboard	85

List of figures

Figure 1.1: Architecture of the Intelligent CI/CD.....	14
Figure 4.1: System Diagram of the Intelligent CI/CD.....	31
Figure 4.1: System Diagram of the Intelligent CI/CD.....	31
Figure 4.2: Activity Diagram of the Intelligent CI/CD.....	32
Figure 4.3: Use Case Diagram of the Intelligent CI/CD.....	33
Figure 5.1: Layered Architecture of the Intelligent CI/CD.....	36
Figure 5.2: Revised Actor Use Case Diagram of the Intelligent CI/CD Orchestrator.....	37
Figure 5.3: Class Diagram for UC-01 Authenticate and Manage Access	39
Figure 5.4: Interface design for UC-01 (login screen and admin role panel)	39
Figure 5.5: Sequence Diagram for UC-01 Authenticate and Manage Access.....	40
Figure 5.6: Object Diagram for UC-01 (snapshot of a successful admin login)	41
Figure 5.7: ERD fragment for UC-01 (users, audit_logs)	42
Figure 5.8: Class Diagram for UC-02 Submit NL CI/CD Command.....	43
Figure 5.9: Interface design for UC-02 (command console with clarification prompt)	43
Figure 5.10: Sequence Diagram for UC-02 Submit NL CI/CD Command.....	44
Figure 5.11: Object Diagram for UC-02 (parsed command snapshot).....	44
Figure 5.12: ERD fragment for UC-02 (users, nl_commands).....	45
Figure 5.13: Class Diagram for UC-03 Generate and Validate YAML Pipeline	46
Figure 5.14: Interface design for UC-03 (pipeline preview with validation result)	46
Figure 5.15: Sequence Diagram for UC-03 Generate and Validate YAML Pipeline ..	47
Figure 5.16: Object Diagram for UC-03 (validated pipeline snapshot).....	47
Figure 5.17: ERD fragment for UC-03 (nl_commands, generated_pipelines).....	48
Figure 5.18: Class Diagram for UC-04 Submit Prioritized Job.....	49
Figure 5.19: Interface design for UC-04 (job submission form with priority and dependencies).....	49
Figure 5.20: Sequence Diagram for UC-04 Submit Prioritized Job	49
Figure 5.21: Object Diagram for UC-04 (a submitted job with priority High and one dependency)	50
Figure 5.22: ERD fragment for UC-04 (jobs, job_dependencies)	51
Figure 5.23: Class Diagram for UC-05 Optimize Dynamic Queue.....	53
Figure 5.24: Interface design for UC-05 (live queue board with priority scores)	54
Figure 5.25: Sequence Diagram for UC-05 Optimize Dynamic Queue	54
Figure 5.26: Object Diagram for UC-05 (three queued jobs, two grouped, with computed scores)	54
Figure 5.27: ERD fragment for UC-05 (jobs, job_runs, dependency_groups).....	56
Figure 5.28: Class Diagram for UC-06 Dispatch Job to Runner	57
Figure 5.29: Interface design for UC-06 (executors panel showing the dispatch decision).....	58

Figure 5.30: Sequence Diagram for UC-06 Dispatch Job to Runner	58
Figure 5.31: Object Diagram for UC-06 (dispatcher allowing the heap top, blocking the rest).....	59
Figure 5.32: Class Diagram for UC-07 Execute Pipeline Stages	59
Figure 5.33: Interface design for UC-07 (pipeline stages view with live status)	60
Figure 5.34: Sequence Diagram for UC-07 Execute Pipeline Stages.....	60
Figure 5.35: Object Diagram for UC-07 (a completed run with recorded duration)...	61
Figure 5.36: ERD fragment for UC-07 (job_runs, resource_samples).....	61
Figure 5.37: Class Diagram for UC-08 Monitor Dashboard and Analyze Performance	62
Figure 5.38: Interface design for UC-08 (baseline versus optimized comparison dashboard).....	63
Figure 5.39: Sequence Diagram for UC-08 Monitor Dashboard and Analyze Performance	63
Figure 5.40: Object Diagram for UC-08 (comparison of two experiments).....	64
Figure 5.41: ERD fragment for UC-08 (experiments, experiment_metrics, job_runs)	64
Figure 5.42: Landing / home interface of the Intelligent CI/CD Orchestrator	65
Figure 5.43: Complete class diagram of the Intelligent CI/CD Orchestrator (all layers)	66
Figure 5.44: Complete database schema of the Intelligent CI/CD Orchestrator (all PostgreSQL tables)	67

List of symbols and Units

1. Introduction

Modern software development has evolved significantly with the adoption of Continuous Integration & Continuous Deployment (CI/CD) pipelines, which are used to automate the process of building, testing, & deploying applications. These pipelines play a critical role in usage of rapid & reliable software delivery, allowing development teams to integrate changes frequently while also detecting issues early. Despite their advantages, CI/CD system faces major challenges as the scale & complexity of software projects increases. One of the most critical issues is inefficient job scheduling within pipeline queues. Most existing systems like Jenkins, rely on simple scheduling strategies such as FirstInFirstOut where jobs are executed in the order they are received. This approach fails to consider important factors such as job priority & execution time & resource availability. As a result, high priority or time sensitive tasks may experience unnecessary delays leading to increased waiting times, reduced throughput & poor resource utilization. This project was undertaken to address these limitations by designing a smart CI/CD pipeline management system that introduces intelligent scheduling & improved user interaction.

The primary function is to enhance the efficiency of CI/CD pipelines by integrating an intelligent scheduling mechanism with Jenkins & to provide a user-friendly interface through a chatbot. The system allows users to interact with the pipeline using natural language commands thus eliminating the need for complex manual configurations or scripting. Commands such as triggering builds, running tests or deploying applications can be easily issued through the chatbot which processes the input & translates it into appropriate Jenkins pipeline actions. The entire system architecture consists of several interconnected components including the chatbot interface, a dynamic scheduling module, Jenkins pipeline integration, a containerized execution environment using docker & Kubernetes & a monitoring & visualization dashboard. The user input is first captured by the chatbot, then forwarded to the scheduling module where jobs are analyzed & assigned priority scores. Based on these scores, the queue is dynamically reordered before the jobs are dispatched to Jenkins agent for execution. The performance metrics are continuously monitored & displayed on a dashboard.

The performance of the system is evaluated based on several key variables that directly influence the efficiency & reliability of CI/CD pipelines. These include job waiting time, system throughput, resource utilization, execution latency & overall system reliability. The objective is to minimize waiting time & latency while maximizing throughput & ensuring efficient use of computational resources. Important variables that affect performance include job priority score (based on urgency & importance), estimated execution time, queue length, & available resources such as CPU, memory, & container instances. The system dynamically adjusts these variables through the scheduling module to maintain suboptimal performance under dynamic workload conditions. For example, urgent jobs are assigned higher priority scores & moved ahead in the queue, while long running jobs may be scheduled in parallel environments to reduce delays. This dynamic approach ensures better responsiveness & improved system efficiency compared to traditional methods.

To manage the complexity of the system & ensure a structured development process, the project is divided into several subprojects. The first subproject focuses on the

development of the chatbot interface, The second involves integration with Jenkins to automate pipeline execution. The third subproject is the implementation of the dynamic scheduling module, which is the core component responsible for queue optimization & intelligent job prioritization. The fourth subproject includes setting up the containerized execution environment using Docker & Kubernetes to ensure scalability & consistent resource management. The final subproject focuses on the monitoring & dashboard module, which tracks system performance metrics & provides real time insights into pipeline operations. Each of these components works together to form a cohesive & efficient CI/CD pipeline management system.

This report has been structured to provide a clear & comprehensive understanding of the project. Chapter 1 presents the introduction & motivation for the study. Chapter 2 provides a detailed background & literature review discussing existing CI/CD systems & related optimization techniques. Chapter 3 outlines the aim & statement of the problem, clearly defining the issues addressed & the objectives of the proposed system. Chapter 4 describes the hardware & software analysis & requirements necessary for implementing the system. Overall, these chapters guide the reader from fundamental concepts to the detailed design & requirements of the proposed solution.

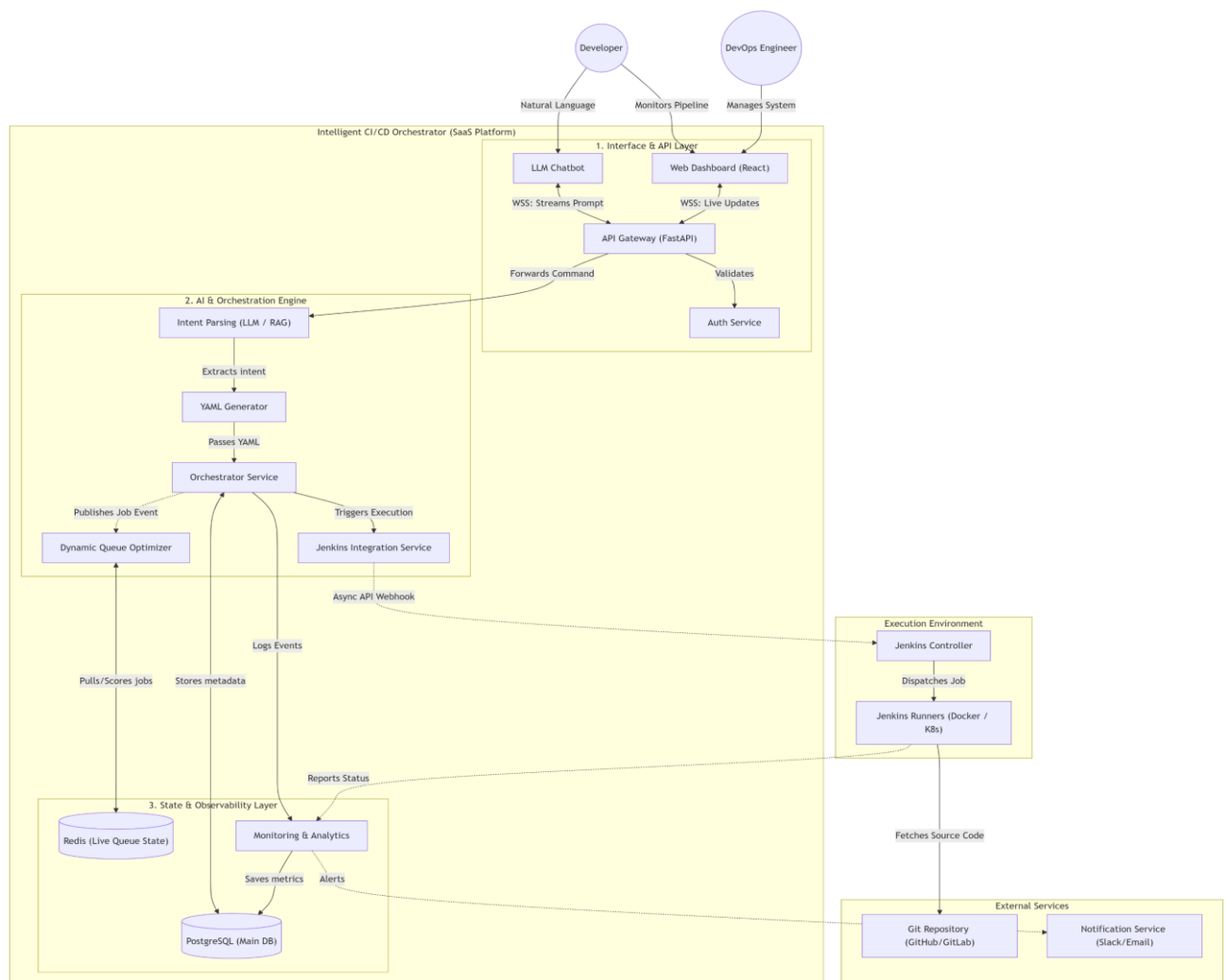


Figure 1.1: Architecture of the Intelligent CI/CD

2. Background & Literature Review

Continuous Integration & Continuous Deployment (CI/CD) pipelines have become fundamental to modern software engineering by automating build, test, & deployment processes. Tools such as Jenkins & Git based platforms rely on distributed runner-based architectures to execute jobs efficiently. However, as system complexity & workload scale increase, CI/CD environments face critical challenges including queue congestion, inefficient scheduling, increased latency & poor resource utilization.

Recent research has explored two major ways to address these issues. The first focuses on LLM driven automation, enabling natural language interfaces for pipeline generation & execution. The second emphasizes CI/CD optimization techniques, such as dynamic queue reordering. This literature review integrates both perspectives to provide a comprehensive understanding of intelligent CI/CD systems.

2.1 *Natural Language–Driven CI/CD Automation*

2.1.1 Natural Language Understanding & Intent Recognition

The key requirement of LLM based CI/CD systems is transforming unstructured user input into executable actions. Research in “semantic parsing” & AI planning demonstrates effective techniques for converting the textual commands into its structured representations.

The study Integrating AI Planning with Natural Language Processing highlights methods for extracting actionable workflows from natural language [1]. Similarly, Intent Based Networking Using large language models introduces intent extraction techniques where high-level user goals are translated into executable system actions.[2]

Further advancements such as demonstrations of how LLM can generate structured workflow representations (e.g. JSON/DAGs with feedback loops for refinement).

2.1.2 Structured Output & Pipeline Generation

Once our intent is extracted, generating a structurally correct pipeline configuration becomes essential part. The study shows that LLM performance depends on custom prompt design & model tuning.[3]

To improve reliability through schema constrained generation, significantly syntactic correctness. This is critical for CI/CD pipelines, where minor errors may lead to a system failure. [4]

2.1.3 YAML Based CI/CD Pipeline Generation

Pipelines system relies on YAML or batch script-based configurations. The study demonstrates that domain specific pretraining significantly improves the accuracy of YAML generation. [5]

Similarly, provides standardized datasets & evaluation metrics for YAML based workflow generation, enabling more consistent benchmarking across approaches. [6]

Furthermore, it evaluates LLM generated CICD pipelines & highlights key limitations, including structural inaccuracies, syntactic violations, & incomplete or missing configuration elements.[7]

2.1.4 Tool Augmented LLMs for Workflow Automation

Modern research extends LLM capabilities by integrating external tools. Tool use in language models enables API trigger & external function execution, improving their ability to perform real world tasks [8]. Another approach combines reasoning with action execution, allowing models to interconnect decision making with step-by-step interactions in dynamic environments [9].

Workflow automation systems further extend these capabilities by enabling the generation of executable “DAG” based workflows for complex tasks. These systems can automatically construct structured workflows based on high level user intent [10], while other approaches focus on fully automating workflow generation using LLMs [11]. These methods closely align with CI/CD pipelines, where task dependencies, execution order, & structured orchestration are critical for reliable automation.

2.1.5 Chatbot Integration in CI/CD Systems

Chatbots are increasingly being integrated into DevOps workflows to assist developers in querying & managing CI/CD systems. One study demonstrates a Lambasted chatbot that leverages Retrieval Augmented Generation (RAG) to enhance the accuracy & relevance of responses regarding developer queries related to CI/CD processes [12].

In addition, Jenkins supports pipelines as code through Groovy based syntax, which enables flexible & programmable CI/CD pipeline definitions. Official documentation highlights the structured requirements of this syntax & emphasizes the importance of generating pipelines that are fully compatible with execution environments to ensure successful deployment & automation.[13].

2.2 CICD Runner Queue Optimization

2.2.1 Queueing Theory Foundations

Queueing theory provides a mathematical basis for modelling CI/CD systems. CI/CD runners are often represented using “M/M/c queueing” models, where multiple servers handle incoming job requests. This modelling demonstrates that as the arrival rate of jobs increases, system congestion rises, leading to higher waiting times & increased latency [14].

2.2.2 Limitations of Traditional Scheduling

Traditional scheduling approaches such as FIFO & Round Robin are not adaptive to dynamic workloads. These approaches struggle to efficiently handle varying job priorities, resource demands, & execution times in modern CI/CD environments. As a result, they are often led to suboptimal resource utilization & increased waiting times, highlighting the need for intelligent & context aware scheduling mechanisms [15].

2.3 Dynamic Queue Reordering

2.3.1 Adaptive Scheduling Strategies

Dynamic queue reordering improves system performance by prioritizing jobs based on multiple criteria such as urgency, complexity & resource requirements. Adaptive scheduling approaches introduce intelligent decision-making mechanisms that optimize execution order & improve overall efficiency.[16].

Similarly, machine learning based approaches have been used to predict queue wait times in high performance computing systems, enabling more accurate scheduling decisions & improved system responsiveness. [17]

2.3.2 Recording Based on Priority Scoring

To reorder the queue tasks can be scheduled using a priority scoring system where each task is assigned a score based on the factors like urgency, dependency, execution and resource usage. The score decides the order of task in queue. Tasks should be kept updated continuously instead of remaining fixed. This approach ensures to execute important tasks earlier while making fairness among all tasks. [19],[18]

2.3.3 Calculation of Execution Time

In scheduling execution time is an important factor as it affects how tasks are quickly completed. It is the difference between submission and start time. It can be declared by the developer manually and by past experiences. It can also be calculated by using historical data such as previous runs of similar tasks. Using modern technologies like AI can be calculated based on some factors. [20],[21]

2.3.4 Urgency Mapping

The urgency map technique prioritizes urgent jobs to be performed by giving values to High, Medium, Low. The urgency of each job is determined depending on how soon it needs to be completed or the significance of the job. Jobs that require early completion receive high urgency ranking. It is a straightforward technique for enhancing system efficiency without involving complicated processes. [20]

2.3.5 Heap Based Sorting

A max-heap structure is applied to ensure effective handling of tasks in terms of their ordering. This means that the most prioritized task will be positioned at the top of the list. The use of heap structures is quite useful in dealing with dynamic scheduling since insertion and deletion can be done fast. [19]

2.3.6 Benefits of Dynamic Reordering

Dynamic scheduling provides several advantages, including:

1. Reduced waiting time
2. Improved prioritization of critical jobs
3. Better resource utilization
4. Adaptive workload handling

2.4 CI/CD Framework

2.4.1 Integrated Optimization Framework

Based on the existing literature, an intelligent CI/CD system should integrate:

1. Natural language input via LLM
2. Intent recognition & workflow generation
3. Structured pipeline generation (YAML /Jenkins file)
4. Dynamic queue reordering

This approach ensures efficient, adaptive, & optimized CI/CD pipeline execution.

2.4.2 Research Gap

Existing studies primarily focus on isolated components such as:

1. LLM based pipeline generation
2. Scheduling optimization techniques

However, there is a lack of an end-to-end integrated framework that combines as:

1. Natural language interfaces
2. Dynamic scheduling mechanisms
3. Jenkins specific implementation

This gap motivates the need for a unified intelligent CI/CD optimization system.

2.5 Conclusion

CI/CD systems are evolving toward intelligent, automated, & adaptive architectures. While LLMs enable natural language driven automation, optimization techniques such as dynamic scheduling improve performance & reliability.

A unified framework combining these approaches provides a hopeful promising solution for next generation CI/CD systems, particularly in staging environments where efficiency & rapid feedback are critical.

3. Aim & Statement of Problem

3.1 *Aim of the Project*

The aim of this project is to design & develop a smart CI/CD SaaS platform that enhances traditional pipeline management by integrating a Large Language Model (LLM) based chatbot interface & a dynamic queue optimization mechanism for CI/CD runners.

The system enables developers to interact with CI/CD pipelines using natural language commands while improving execution efficiency through an intelligent queue reordering approach based on a computed priority scoring function. This scoring mechanism dynamically evaluates incoming jobs & reorders them in real time to reduce waiting time, avoid bottlenecks & improve resource utilization in staging environments.

3.2 *Statement of the Problem*

Modern CI/CD platforms such as Jenkins, GitLab CI, & GitHub Actions provide reliable automation but still depend on static scheduling mechanisms & complex configuration workflows, which create inefficiencies in both system performance & usability.

The key problem is that current systems do not intelligently manage CI/CD runner queues or simplify pipeline configuration for developers, resulting in inefficiencies during high load & rapid deployment scenarios.

3.2.1 *Limitations of Static Queue Scheduling*

Existing CI/CD systems primarily use FIFO or fixed priority queues, which lead to:

1. Long waiting times during burst pipeline submissions
2. No dynamic adjustment of job order once queued
3. Inefficient handling of urgent deployments
4. Poor utilization of available runners under varying workloads

As a result, pipeline execution becomes slower & less responsive during high traffic conditions.

3.2.2 *Complexity of YAML-Based Pipeline Configuration*

Another major limitation is the heavy reliance on YAML based configuration files for defining CI/CD pipelines.

In current systems:

1. Developers must manually write & maintain complex YAML files
2. Even small pipeline changes require editing structured configuration syntax
3. YAML files are prone to indentation & syntax errors
4. Nonexpert users face a steep learning curve
5. Debugging pipeline configuration issues is time consuming

This creates a barrier to entry for developers who are not familiar with Dev Ops tooling & reduces overall productivity in fast development environments.

3.2.3 Lack of Dynamic Priority Based Reordering

Current CI/CD systems do not continuously evaluate queued jobs once they are submitted. Once a job enters the queue, its position is generally fixed.

This project identifies a key gap:

There is no Realtime scoring mechanism that dynamically reorders jobs based on multiple contextual factors such as:

1. Job urgency or deployment type (critical vs noncritical)
2. Estimated execution time
3. Resource requirements
4. Queue congestion level
5. Historical runtime patterns

Due to this limitation, important or time sensitive deployments may be delayed behind lower priority tasks.

3.2.4 Absence of Intelligent Queue Optimization in Open Source CI/CD Tools

While enterprise CI/CD solutions provide limited autoscaling features, most opensource platforms lack:

1. Realtime queue scoring systems
2. Adaptive reordering of jobs in staging environments
3. Predictive or context aware scheduling logic

This results in inefficient handling of pipeline bursts, where multiple jobs compete for limited runner resources.

3.2.5 Developer Experience & Usability Issues

CI/CD pipelines typically require significant technical expertise, including:

1. Writing & managing YAML configuration files
2. Using CLI based commands for pipeline execution
3. Understanding platform specific syntax & debugging logs

Existing solutions mostly provide notifications or monitoring capabilities, but they do not fully replace YAML based configuration or provide end-to-one pipeline control through natural language.

3.2.6 Stakeholder Challenges

Different stakeholders face distinct challenges:

1. **Developers:** Difficulty managing complex YAML configurations & queue delays
2. **DevOps Engineers:** Lack of real time control over job prioritization & scheduling efficiency
3. **Organizations:** Increased deployment delays affecting release cycles & productivity
4. **System Administrators:** Inefficient resource utilization during peak workloads

3.3 Scope of the Project

The scope of this project is to develop a SaaS based intelligent CI/CD platform that enhances traditional pipeline execution by integrating natural language interaction, dynamic queue optimization. The system is designed as an enhancement layer over existing CI/CD tools to improve both usability & performance efficiency in staging environments.

The key components within the scope include:

3.3.1 LLM Based Chatbot Interface

An intelligent natural language interface that allows developers to trigger & manage CI/CD operations such as build, test, & deployment without requiring manual YAML configuration or CLI commands. The chatbot translates user input into structured pipeline execution commands.

3.3.2 Dynamic Queue Reordering System

A priority scoring based scheduling mechanism that continuously evaluates CI/CD jobs in the runner queue & dynamically reorders them in real time. This ensures optimal execution order based on job importance, system load, & resource availability, reducing overall waiting time & improving pipeline efficiency.

3.3.3 Web Based Dashboard

A monitoring dashboard that visualizes:

1. Pipeline execution status
2. Queue ordering & priority scores
3. Runner resource utilization & system performance metrics

Exclusions

The following are outside the scope of this project:

1. Development of new containerization systems (existing tools like Docker are used)
2. Design of physical infrastructure or hardware systems
3. Replacement of existing CI/CD platforms; this system acts as an intelligent optimization layer

3.4 Approach to the Problem

The proposed system addresses the identified challenges in CI/CD pipeline management through a multilayered intelligent framework that combines natural language understanding, dynamic scheduling, & adaptive execution strategies.

The overall approach is divided into the following key components:

3.4.1 Natural Language Interface Layer

The system integrates a Large Language Model (LLM) based chatbot that enables developers to interact with the CI/CD pipeline using natural language commands. These commands are processed & converted into structured pipeline operations such as build, test, & deployment. This eliminates the dependency on manual YAML configuration & CLI based execution, making the system more accessible & user friendly.

3.4.2 Dynamic Queue Optimization Layer

At the core of the system is a priority scoring-based dynamic queue reordering algorithm. Each incoming CI/CD job is assigned a score based on multiple factors such as:

1. Job urgency
2. Estimated execution time
3. Resource requirements
4. Current queue load

The scheduler continuously reevaluates queued jobs & dynamically reorders them to ensure optimal execution flow. This helps reduce waiting time, avoid bottlenecks, & improve overall pipeline efficiency in staging environments.

3.4.3 System Integration Layer

All components are integrated into a unified SaaS based CICD platform, ensuring smooth interaction between the chatbot, scheduler & execution engine. This layer manages:

1. Job lifecycle execution
2. Communication between queue & runners
3. Realtime monitoring & logging
4. Coordination of selected test execution strategy

3.5 Monitoring & Visualization Layer

A web-based dashboard is included to provide real-time visibility into:

1. Pipeline execution status
2. Dynamic queue ordering & priority scores
3. Resource utilization of CICD runners

4. Hardware, Software analysis & requirements

4.1 Introduction

This chapter presents the hardware & software part of analysis for the given proposed Intelligent CI/CD Orchestrator. The project is designed as an intelligent enhancement layer on top of Jenkins based CI/CD infrastructure. Its main functions are natural language interaction through an LLM chatbot, dynamic queue reordering using a priority scoring mechanism. The project clearly describes the goal of the system as an integrated SaaS platform that improves efficiency, reduces waiting time, & optimizes runner utilization in staging environments.

The goal of this chapter is to identify what hardware is required, what software technologies are most suitable, which algorithms & data structures fit the system best & how the functional requirements can be expressed in user story way with also acceptance criteria. The chapter also explains the system structure using the architectural views defined in the report system diagram, actors use case diagram, activity diagram & operational diagram.

4.2 FactFinding Techniques Used

The fact-finding approach for this project is primarily document based & comparative. The report shows that the requirements were derived from a detailed literature review of LLM driven CI/CD automation, runner queue optimization, together with analysis of the limitations of Jenkinsstyle FIFO scheduling & YAM heavy workflows. This means the project's requirements were not guessed randomly; they were extracted by studying existing CI/CD practices, comparing approaches, & identifying the exact gaps that the proposed system is meant to solve.

The main techniques used are therefore:

1. **Literature review:** To identify current CI/CD challenges, LLM based automation methods & queue optimization techniques.
2. **Comparative analysis:** to compare FIFO scheduling with intelligent queue reordering.
3. **Workflow analysis:** to understand the end-to-end flow from user command to YAML generation, queue scoring, Jenkins execution & dashboard monitoring.
4. **Requirement derivation from problem analysis:** to convert the limitations of static scheduling & manual configuration into concrete system requirements.

4.3 Hardware Analysis & Requirements

4.3.1 Hardware Scope

This project does not require designing new physical hardware. We instead rely on existing technologies such as Docker & Kubernetes for execution. In other words, the “hardware” in this project refers to the execution environment: developer machines, cloud virtual machines, Jenkins controller/agents & container hosts.

4.3.2 Required Hardware Components

The following hardware infrastructure is suitable for the project:

1. **Developer workstation** for coding, testing, & running the frontend/backend locally.
2. **Cloud VM or server** to host the SaaS platform & Jenkins services.
3. **Runner/agent machines** for executing builds, tests & deployments.
4. **Sufficient CPU & RAM** to support concurrent pipeline jobs & dashboard services.
5. **Storage** for logs, pipeline history, & monitoring data.

4.3.3 Comparison of Hardware Options

Option 1: Local single machine setup

This is the simplest choice for development, but it is not suitable for realistic CI/CD load. It cannot scale well & the queue optimization feature would not be meaningful under burst workloads.

Option 2: Dedicated on premise server

This gives more control, but it requires capital expenditure, maintenance, & manual scaling. It is also less flexible for a SaaS style project.

Option 3: Cloud VM + containerized runner environment

This is the best fit for the project. It supports scaling, isolation, & multi runner execution. Jenkins itself is designed to run as an automation server & can be deployed in different environments, including Docker & Kubernetes. Docker separates applications from infrastructure, while Kubernetes automates deployment, scaling, & management of containerized workloads.

4.3.4 Recommended Hardware Choice

The recommended hardware architecture is a cloud-based setup with:

1. one application server
2. one database/cache node or managed service
3. one Jenkins controller

4. multiple Jenkins agents/runners

Docker runtime on the runner This choice is significant because the project's main problem is queue congestion underload. A cloud based, multi runner architecture is the only realistic way to evaluate.

4.4 *Software Analysis & Technology Selection*

4.4.1 **Front End Technology**

The front end of the system will be developed using React, following a Single Page Application (SPA) approach. This means the application loads once and then updates content dynamically, so users don't experience full page reloads again. As a result, the system feels faster and smoother to use.

React is based on reusable components, which makes it a great choice for building a dashboard that includes chat panels, queue metrics, pipeline status cards, logs, and live notifications. Each part of the interface can be built as a separate component, making the system easier to organize and manage. This modular approach also helps the dashboard grow over time, as new features can be added without making the system overly complex or difficult to maintain.

Why React is better than alternatives here:

1. **React vs Angular:** React is lighter for a dashboard centric SaaS UI & easier to split into reusable widgets.
2. **React vs Vue:** React has stronger ecosystem support for complex dashboards & developer tooling in many academic projects.
3. **React vs plain HTML/JS:** plain JS becomes hard to maintain once the dashboard needs live updates, filtering, & multiple panels.

4.4.2 **Backend Technology**

The back end should be implemented using Fast API, which is a modern Python web framework for building APIs, & its official documentation emphasizes high performance & support for asynchronous programming. This is important because the system needs to handle chatbot requests, queue updates, dashboard polling, Jenkins triggers, & monitoring events efficiently.

Why Fast API is better than alternatives here:

1. **Fast API vs Django:** Fast API is better suited for API first services & async workloads.
2. **Fast API vs Node.js:** Fast API keeps the Python ecosystem available for LLM orchestration & queue logic.
3. **Fast API vs Flask:** Fast API gives stronger request validation & modern API support out of the box.

4.4.3 Middle Layer / Intelligence Layer

The middle layer should contain the LLM orchestration service responsible for:

1. intent recognition
2. command verification
3. YAML/pipeline action generation
4. validation before execution
5. error feedback to the user

The chatbot is expected to translate natural language commands into Jenkins pipeline action & our literature review highlights structured output generation, tool augmented LLMs, & workflow automation as the foundation for this layer.

This layer can be implemented either with:

1. direct LLM API calls plus validation logic, or
2. an orchestration framework such as LangChainstyle tool.

For this project, the safest approach is LLM + structured validation + command schema, because we stress correctness of generated pipeline structure & the need to avoid syntactic errors in CI/CD definitions.

4.4.4 Database Technology

The main persistent database should be PostgreSQL. PostgreSQL is an object relational database system & a relational database management system (RDBMS) which makes it suitable for storing structured entities such as users, jobs, pipelines, job history, and queue decisions.

Why PostgreSQL is better than alternatives here:

1. **PostgreSQL vs MongoDB:** the project has highly structured data, so a relational model is better than document only storage.
2. **PostgreSQL vs MySQL:** PostgreSQL gives strong relational features & flexibility for analytical queries on queue history.
3. **PostgreSQL vs SQLite:** SQLite is fine for prototypes, but not for a multiuser SaaS style system.

4.4.5 Cache / Queue State Store

The live queue state should be stored in Redis. Redis is an in-memory data store used for streaming, & fast state access. That makes it appropriate for queue state, temporary scoring data, & scheduling decisions that need low latency reading & writing.

Why Redis is better than alternatives here:

1. **Redis vs database only queueing:** a relational database is too slow for real time queue state changes.

2. **Redis vs file-based state:** file storage is not suitable for concurrent scheduling decisions
3. **Redis vs message queue only:** Redis can act as the fast state layer while PostgreSQL remains the record system.

4.4.6 Execution & CI/CD Tools

The CI/CD execution engine will remain Jenkins. Jenkins is an open-source automation server used for building; testing & deploying software & Jenkins Pipeline adds automation tools for continuous integration & delivery workflows.

The execution environment should use Docker & Kubernetes. Docker is an open platform for developing, shipping, & running applications, while Kubernetes is an open-source system for automating deployment, scaling, & management of containerized applications. That combination is ideal for staging jobs, isolated test runs, & scalable runner execution.

4.4.7 Source Control & Notification Services

The platform should integrate with Git repositories such as GitHub or GitLab, since the Jenkins pipeline needs to fetch source code before building & test execution. It should also integrate with a notification channel such as email, slack or webhooks to inform users when a pipeline succeeds, fails or needs attention.

4.5 Algorithm & Data Structure Analysis

4.5.1 Queue Scheduling Algorithm

The core algorithm of the project is a priority scoring dynamic queue reordering algorithm. The jobs are assigned scores using factors such as urgency, estimated execution time, resource requirements, queue congestion, & historical runtime patterns. The queue is then dynamically reordered so that urgent & efficient jobs can move ahead of less critical ones.

This algorithm is better than FIFO because FIFO does not consider urgency or resource availability.

4.5.2 Recommended Data Structures

The best data structure combination for this project is:

1. **Priority queue / heap** for choosing the next job to run.
2. **Redis** for live queue state caching.
3. **PostgreSQL tables** for persistent job records & history.
4. **State machine** for job lifecycle management.

A heap-based priority queue is better than a simple list because the system must repeatedly pick the highest priority job as queue conditions dynamically change. A normal FIFO list forces linear waiting order thus defeating the purpose of dynamic optimization.

4.6 Overall System Structure

The system is structured into four views, each of which serves a specific purpose:

1. **System Diagram:** shows the complete architecture of the platform, including the chatbot, orchestrator, scheduler, execution environment & database layer.
2. **Actor Use Case Diagram:** shows the external users & systems interacting with the platform.
3. **Activity Diagram:** shows the workflow from user command to pipeline execution.

This layered representation is appropriate because the project is not just a chatbot or just a scheduler, it is a full CI/CD optimization platform with a visible user interface, a decision layer, an execution layer & dashboard for monitoring.

4.7 Functional Requirements in User Story Format

User stories are commonly written in the form “As a “who”, I want “what” so that is “why” & acceptance criteria are often expressed using “Given–When–Then” scenarios. This is consistent with **Mike Cohn’s** user story guidance & standard agile practice.

4.7.1 User Story 1: Natural Language Pipeline Control

As a DevOps engineer, I want to submit a natural language command through the chatbot so that I can trigger CI/CD actions without manually writing pipeline scripts.

Acceptance Criteria

1. **Given** I am authenticated, **when** I enter a valid command, **then** the system parses the intent & returns a structured pipeline action.
2. **Given** the command is invalid, **when** I submit it, **then** the system shows a validation error & does not trigger Jenkins.
3. **Given** the command is valid, **when** it is processed, **then** the system generates Jenkins ready output.

4.7.2 User Story 2: Dynamic Queue Optimization

As a DevOps engineer, I want the system to reorder the CI/CD queue dynamically so that urgent jobs can be executed earlier.

Acceptance Criteria

1. **Given** multiple jobs are waiting, **when** a new high priority job arrives, **then** the scheduler recalculates the order.
2. **Given** runner availability changes, **when** the queue is reevaluated, **then** the next most suitable job is selected.
3. **Given** queue conditions change, **when** the priority score is updated, **then** the dashboard reflects the new ordering.

4.7.3 User Story 4: Live Monitoring

As a developer, I want to view pipeline status & queue metrics on a dashboard so that I can monitor execution in real time.

Acceptance Criteria

1. **Given** that a pipeline is running, **when** the dashboard is opened, **then** the current job status is visible.
2. **Given** queue reordering occurs, **when** the state changes, **then** the queue metrics update live.
3. **Given** a job completes, **when** results are received, **then** logs & execution status appear on the dashboard.

4.7.4 User Story 5: Secure Access

As a system administrator, I want to authenticate users & control access so that only authorized users can operate the platform.

Acceptance Criteria

1. **Given** a user is not logged in, **when** they access the platform, **then** the system denies access.
2. **Given** a valid user session exists, **when** the user performs an action, **then** the system allows only permitted operations.
3. **Given** suspicious activity occurs, **when** it is detected, **then** the system logs the event for audit purposes.

4.8 NonFunctional Requirements

The system must satisfy the following quality requirements:

1. Performance.
2. Scalability.
3. Reliability.
4. Usability.
5. Maintainability.
6. Security.

4.9 Chapter Summary

This chapter analyses the underlying hardware & software requirements of our Intelligent CI/CD Orchestrator. The project is best implemented as a cloud-based SaaS platform using React for the frontend, Fast API for the backend, PostgreSQL for persistent metadata, Redis for fast queue state storage, Jenkins for execution & Docker Kubernetes for containerized runners. The recommended algorithmic foundation is a priority scoring dynamic queue with a state machine for job management. The system requirements were also expressed as user stories with “Given–When–Then” acceptance criteria so that they can be tested clearly & implemented systematically.

System Diagram:

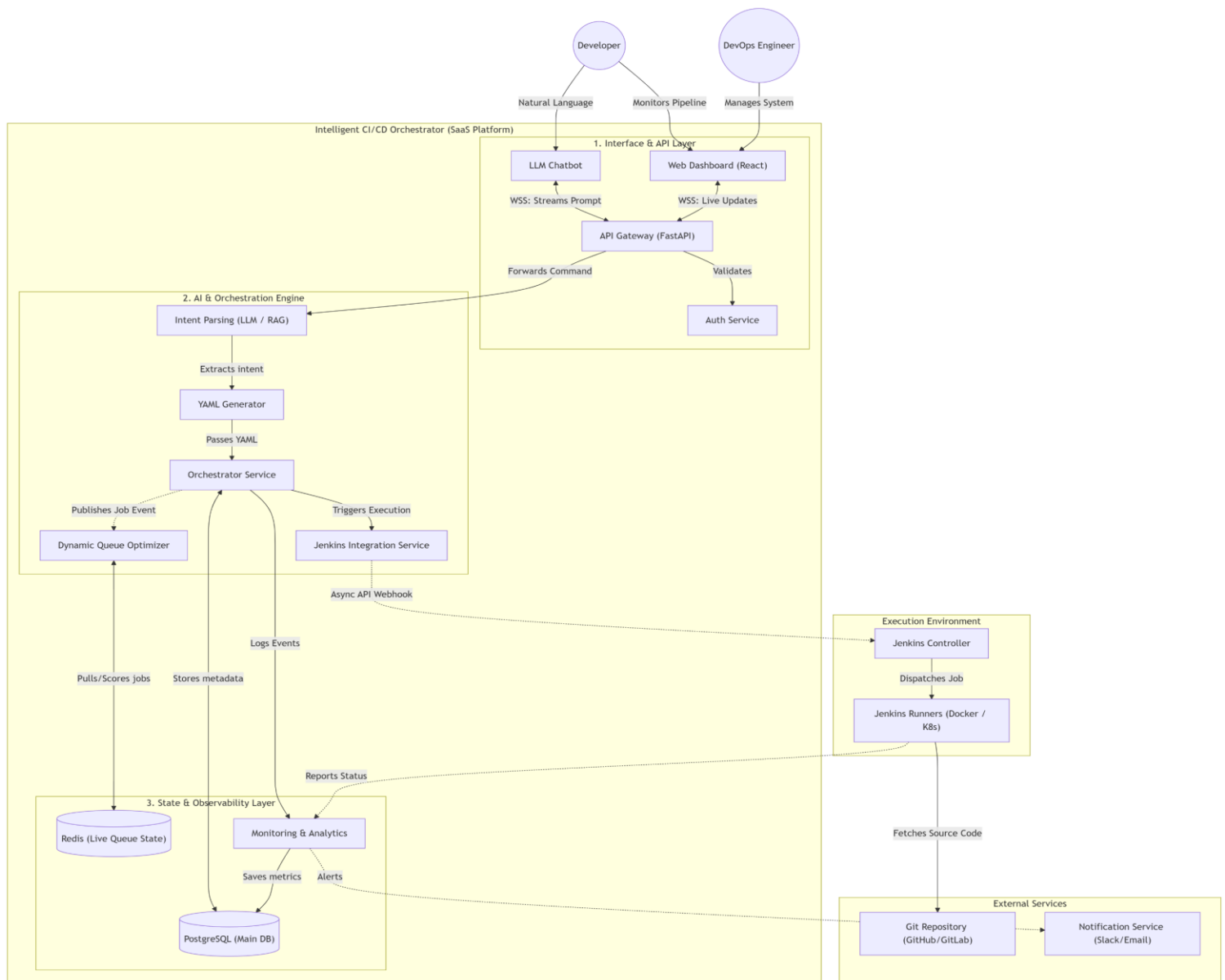


Figure 4.1: System Diagram of the Intelligent CI/CD

INTELLIGENT CI/CD ORCHESTRATOR

Intelligent CI/CD Orchestrator — Activity Diagram with Swimlanes

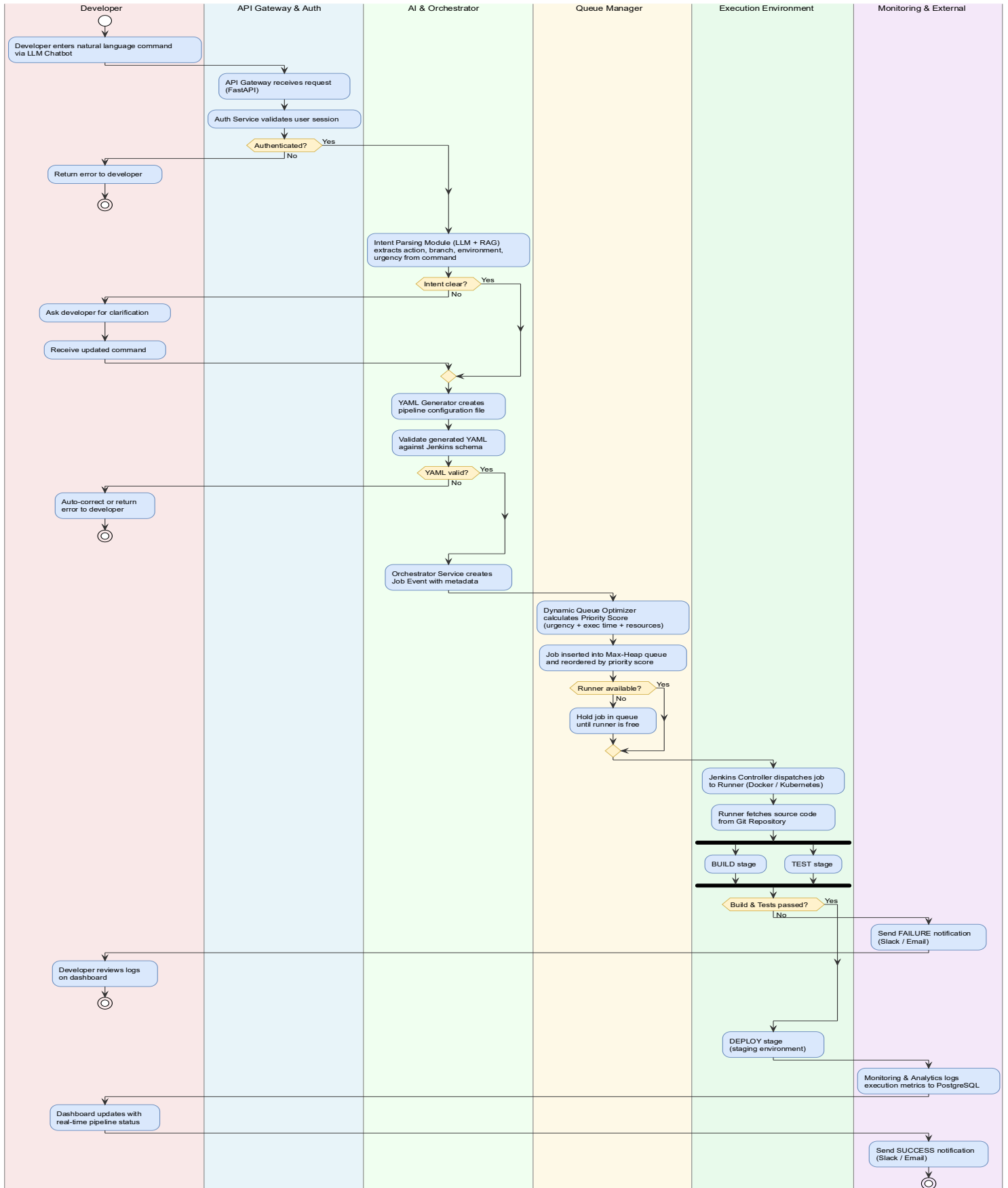


Figure 4.2: Activity Diagram of the Intelligent CI/CD

Use case diagram:

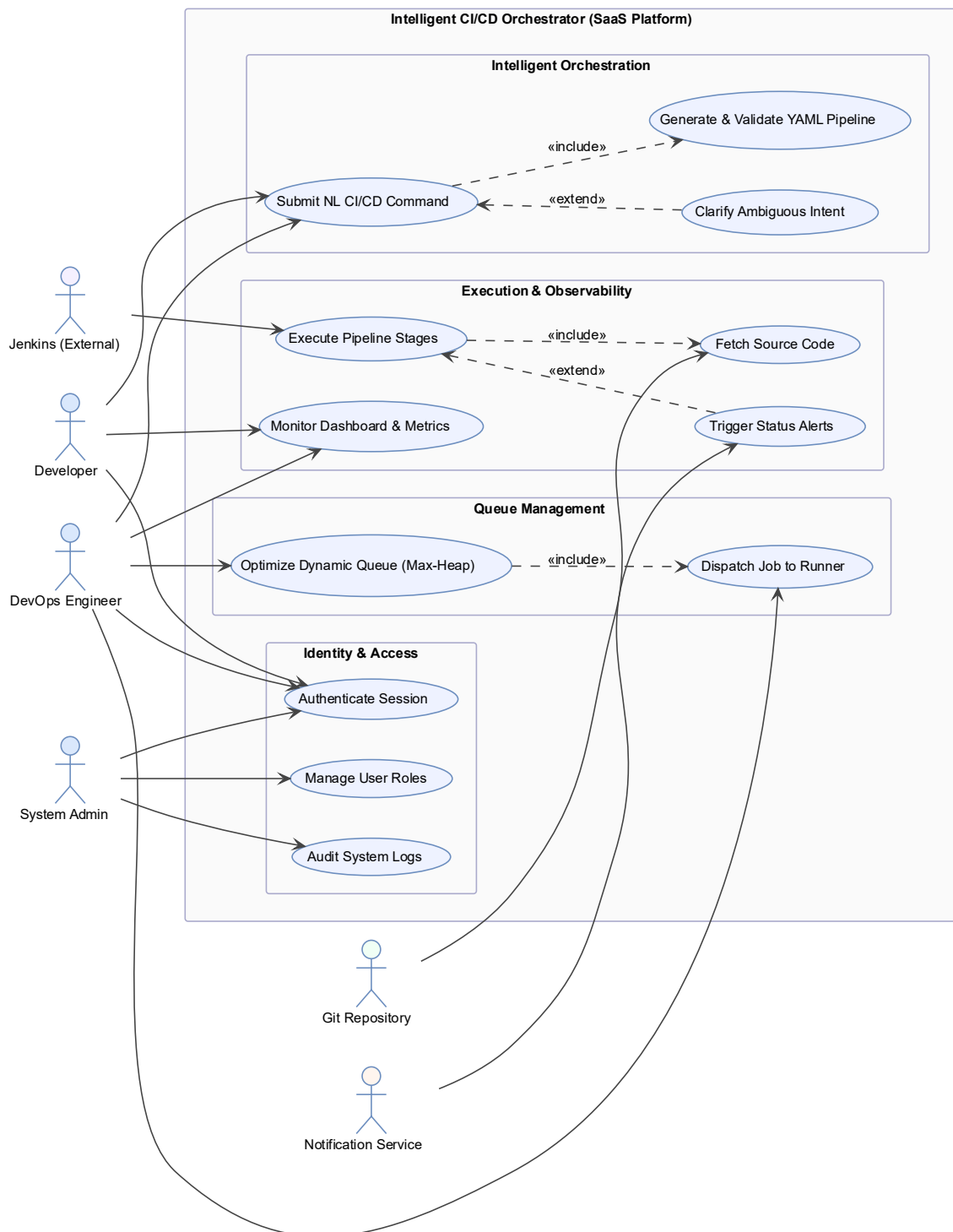


Figure 4.3: Use Case Diagram of the Intelligent CI/CD

5. Software Design and Modeling

5.1 *Introduction*

In this chapter we present the software design of the Intelligent CI/CD Orchestrator, with special attention to the Proof of Concept (POC) assigned to us for Milestone 2, which is the Dynamic Queue Optimization module built as a custom Jenkins plugin. We first describe the coding architecture that we adopted from an external industry source, because every class diagram in this chapter follows that architecture. We then present the revised actor use case diagram, and for each use case we provide its class diagram, its user interface design, its sequence diagram, and, where the use case touches our PostgreSQL database, its object diagram and entity relationship diagram (ERD). At the end of the chapter we present the complete system class diagram and the complete database schema. All diagrams in this chapter use UML 2.5 notation as defined by the Object Management Group .

The design we present here is not only documentation. It is the same structure that we follow in the actual POC code, so the class names, table names, and flows shown in this chapter appear with identical names in our implementation and in the diagrams of this report.

5.2 *Coding Architecture: The Layered Architecture*

We adopted the Layered Architecture pattern as described by Mark Richards in *Software Architecture Patterns*, published by O'Reilly Media. Richards describes it as the most common architecture pattern in industry, also known as the n-tier pattern. In this pattern the components of a system are organized into horizontal layers, where each layer has a specific role and responsibility, and a request moves from the top layer down through the layers below it. Richards defines four standard layers: the presentation layer, the business layer, the persistence layer, and the database layer . We selected this pattern for three reasons. First, our system naturally separates into a user-facing dashboard, decision-making scheduling logic, data access code, and stored data, so the pattern fits without forcing anything. Second, the pattern enforces what Richards calls layers of isolation, meaning a change in one layer does not ripple through the others, which protects our team when we work on different modules in parallel. Third, it is

simple to explain and test, which matters for a final year project that must be demonstrated and defended.

Following this architecture, every class in our system belongs to exactly one of four class types, and our class diagrams mark each class with its layer. The behavior each class type contains is fixed by the architecture, as summarized in Table 5.1.

Table 5.1: Class types used in system according to the Layered Architecture

Layer	Class type (stereotype)	Behavior the classes contain	Examples in our system
Presentation	Controller / Boundary	Accept user or system input, validate the request format, call a service, and return or display the result. No business rules.	MetricsController, QueueController, PriorityDispatcher, QueueStateListener, React views such as QueueBoard
Business	Service	Business rules and decisions of the system, such as computing priority scores, grouping dependent jobs, and choosing the next job.	PriorityScoringService, DependencyGroupService, ExecutionTimeEstimator, HeapSchedulerService, ComparisonService
Persistence	Repository	Read and write data on behalf of services, hiding SQL and external APIs behind simple methods.	JobRunRepository, ExperimentRepository, BuildHistoryService, MetricsPublisher
Database	Entity / Model	Plain data structures that represent rows of our PostgreSQL tables or value objects passed between layers. No logic except validation.	User, Job, JobRun, Experiment, ScoredJob, BuildRecord, JobGroup

Two clarifications are needed for the Jenkins plugin part of the POC. The plugin runs inside Jenkins, not inside our web application, but we kept the same four class types inside the plugin code. Classes that Jenkins calls directly, such as PriorityDispatcher which extends the Jenkins extension point QueueTaskDispatcher, act as boundary classes of the presentation layer, because they receive requests from Jenkins exactly like a controller receives requests from a browser. The scoring, grouping, estimation, and heap classes form the plugin business layer, and BuildHistoryService acts as a persistence class because it is the only class that talks to the Jenkins build history API. This way one single architecture governs the plugin, the FastAPI backend, and the React dashboard.

Within this architecture we also use a small number of named design patterns from Gamma, Helm, Johnson, and Vlissides and from Fowler, so that we can point to a known reference for each design decision: the *Strategy* pattern for the scoring and estimation logic (so a different formula can be swapped in without touching the dispatcher), the *Observer* pattern which Jenkins itself applies to our QueueStateListener and BuildMetricsListener classes (Jenkins notifies them when queue or build events happen), the *Singleton* pattern for the plugin global configuration class PriorityConfiguration, the *Facade* pattern for BuildHistoryService which hides the Jenkins API behind one simple interface, and Fowler's *Service Layer* and *Repository* patterns for the backend.

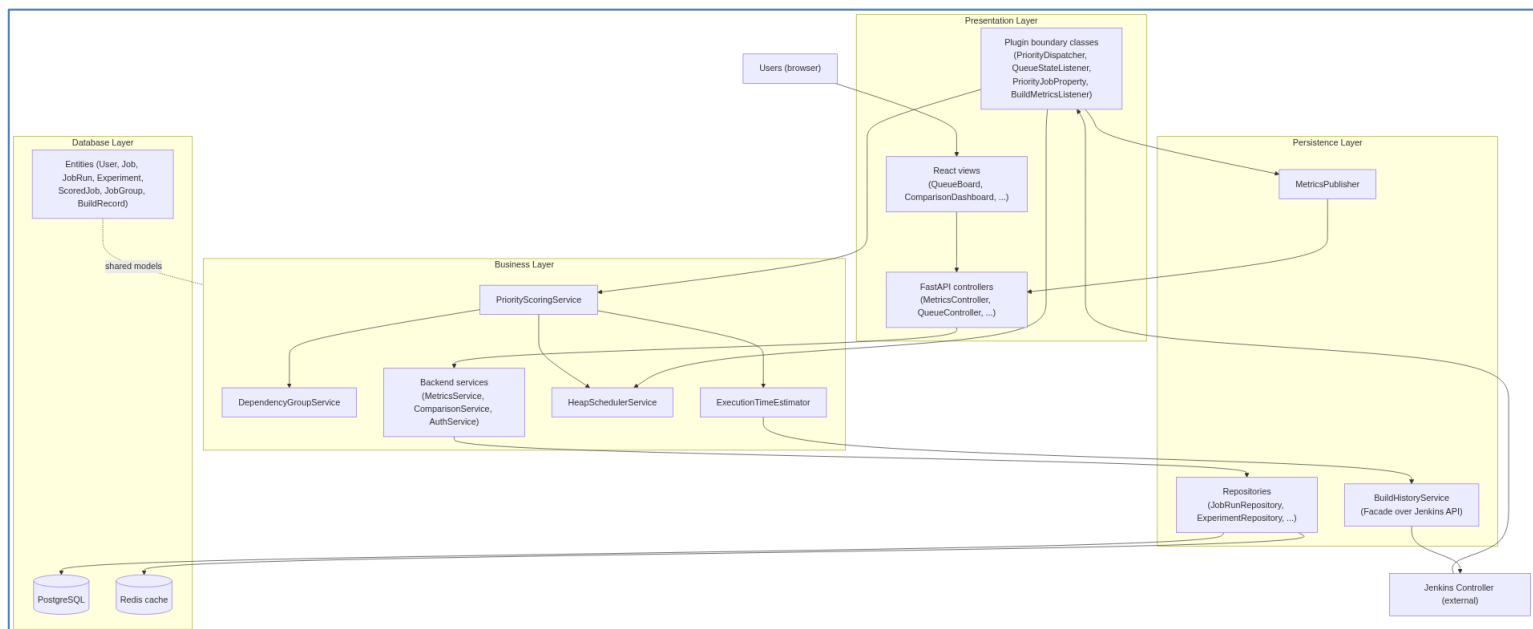


Figure 5.1: Layered Architecture of the Intelligent CI/CD

5.3 Revised Actor Use Case Diagram

For Milestone 2 we revised the actor use case diagram that we submitted in Milestone 1. The original diagram contained twelve use cases. After our POC was assigned, we consolidated closely related use cases and expanded the queue optimization area, which was previously a single use case, into the set of use cases that our plugin actually implements. Authenticate Session and Manage User Roles were merged into one access management use case with auditing included, and Trigger Status Alerts became an extension of the monitoring use case instead of a separate top level use case. We also added Submit Prioritized Job, because in the POC a user explicitly submits a job with

a priority level and optional dependencies, which was not visible in the old diagram. The revised diagram contains eight use cases.

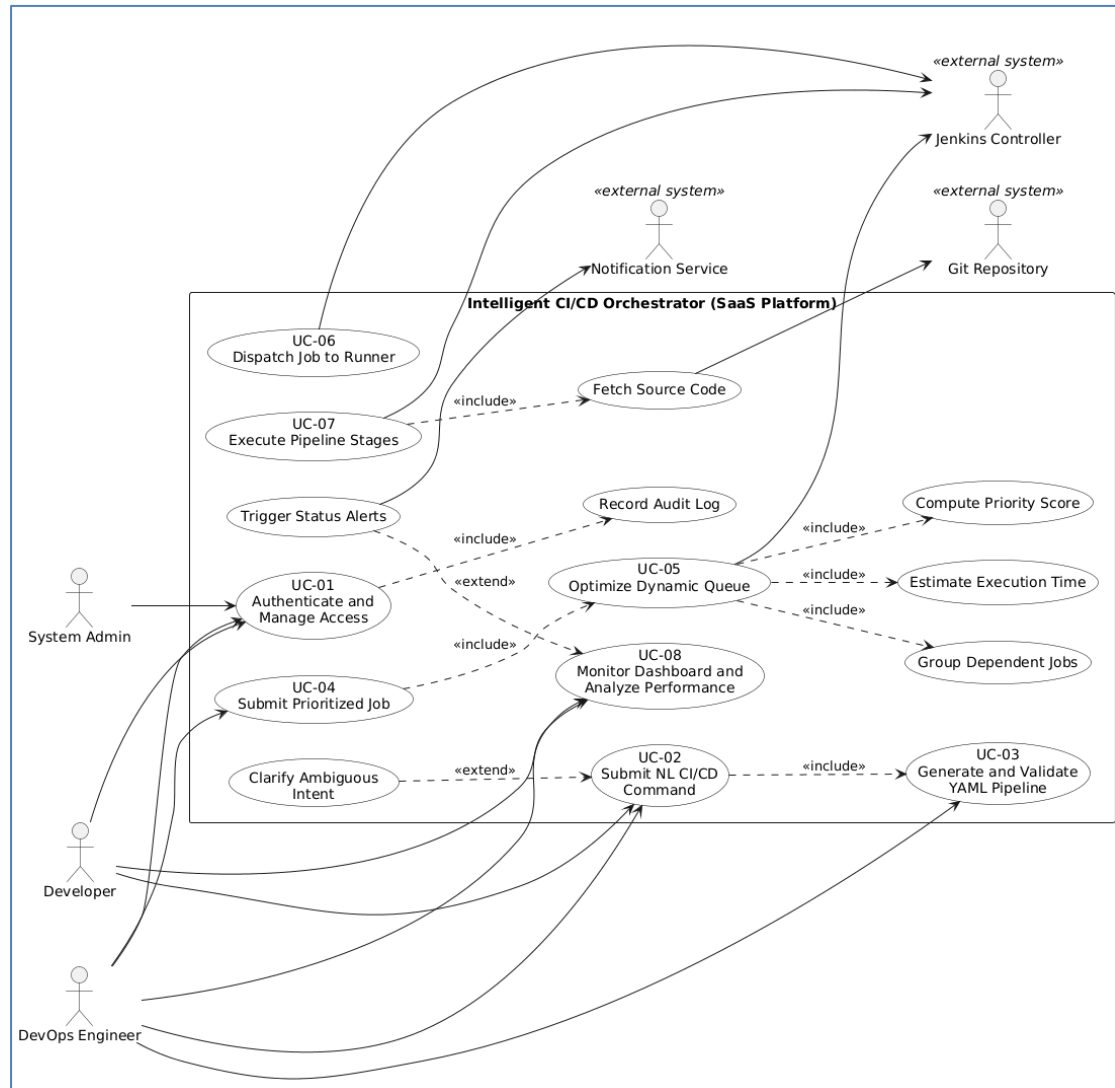


Figure 5.2: Revised Actor Use Case Diagram of the Intelligent CI/CD Orchestrator

Table 5.2: Summary of the revised use cases

ID	Use case	Primary actors	Supporting actors / systems	POC
UC-01	Authenticate and Manage Access	Developer, DevOps Engineer, System Admin	PostgreSQL	Supports POC
UC-02	Submit NL CI/CD Command	Developer, DevOps Engineer	LLM service	Milestone 3
UC-03	Generate and Validate YAML Pipeline	DevOps Engineer	LLM service, Jenkins	Milestone 3
UC-04	Submit Prioritized Job	DevOps Engineer	Jenkins Controller	POC
UC-05	Optimize Dynamic Queue	System (plugin)	Jenkins Controller, PostgreSQL	POC (core)
UC-06	Dispatch Job to Runner	System (plugin)	Jenkins Controller	POC
UC-07	Execute Pipeline Stages	Jenkins Controller	Git Repository	POC (observed)
UC-08	Monitor Dashboard and Analyze Performance	Developer, DevOps Engineer	Notification Service, PostgreSQL	POC

The actors remain the same as in Milestone 1: the Developer, the DevOps Engineer, and the System Admin are human actors, while Jenkins, the Git repository, and the notification service are external systems. In the sections that follow, we present the four required design artifacts for each use case. The class diagrams show only the classes that participate in that use case, each tagged with its layer; the complete class diagram in Section 5.13 shows everything together.

5.4 UC-01: Authenticate and Manage Access

This use case lets a user sign in to the platform and lets the System Admin manage user roles. Every privileged action is recorded in an audit log. Although authentication is not the focus of our POC, the dashboard that presents the POC results sits behind this use case, so we designed and implemented a simple version of it.

The participating classes follow the layered architecture. In the presentation layer, the LoginView React component collects credentials and the AuthController exposes the /auth endpoints of the FastAPI backend. In the business layer, AuthService verifies credentials, issues session tokens, and applies role rules. In the persistence layer, UserRepository and AuditLogRepository read and write the users and audit_logs tables. The User and AuditLog entities represent the stored rows.

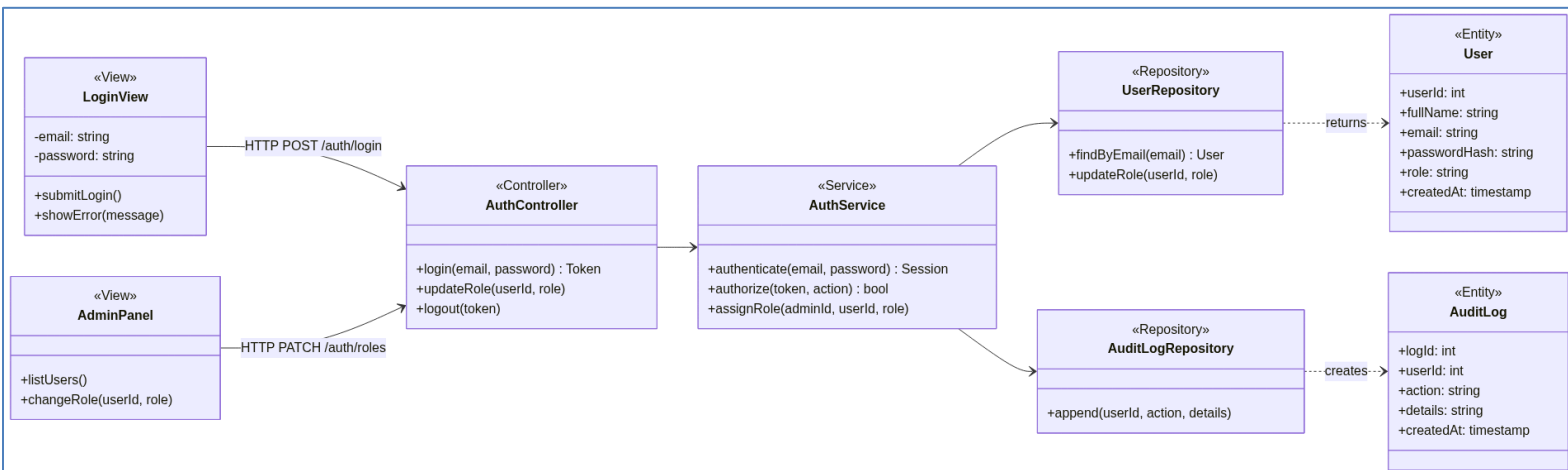


Figure 5.3: Class Diagram for UC-01 Authenticate and Manage Access

The interface for this use case is the login screen, followed by the admin panel for role management. The login screen contains the email and password fields and shows validation errors returned by AuthController. The admin panel lists users with their roles and lets the System Admin change a role from a dropdown, which immediately writes an audit entry.

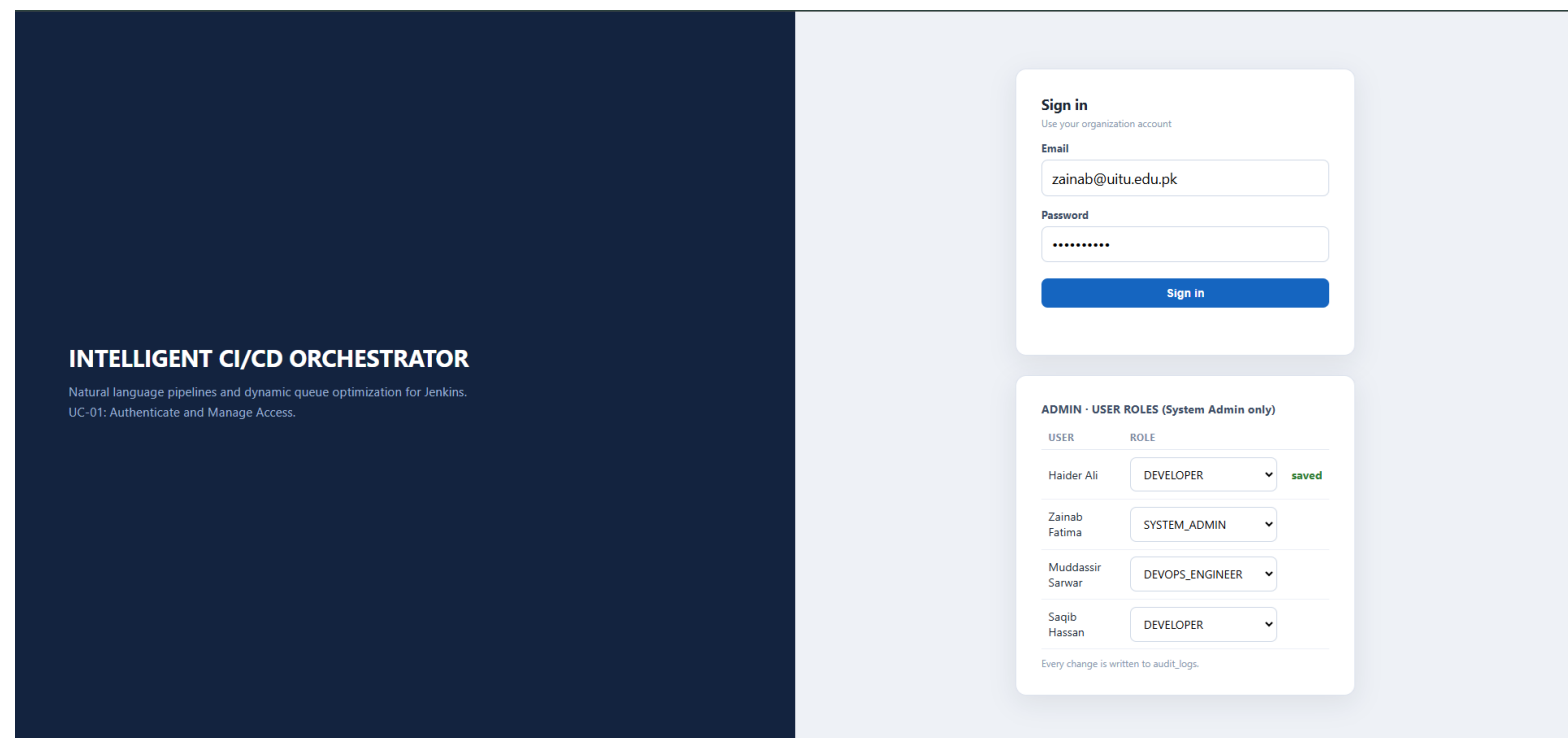


Figure 5.4: Interface design for UC-01 (login screen and admin role panel)

The sequence diagram shows the success path and the failure path. LoginView sends the credentials to AuthController, which delegates to AuthService. AuthService asks UserRepository for the user record, verifies the password hash, creates a session token, and writes an audit entry through AuditLogRepository before returning the token to the view.

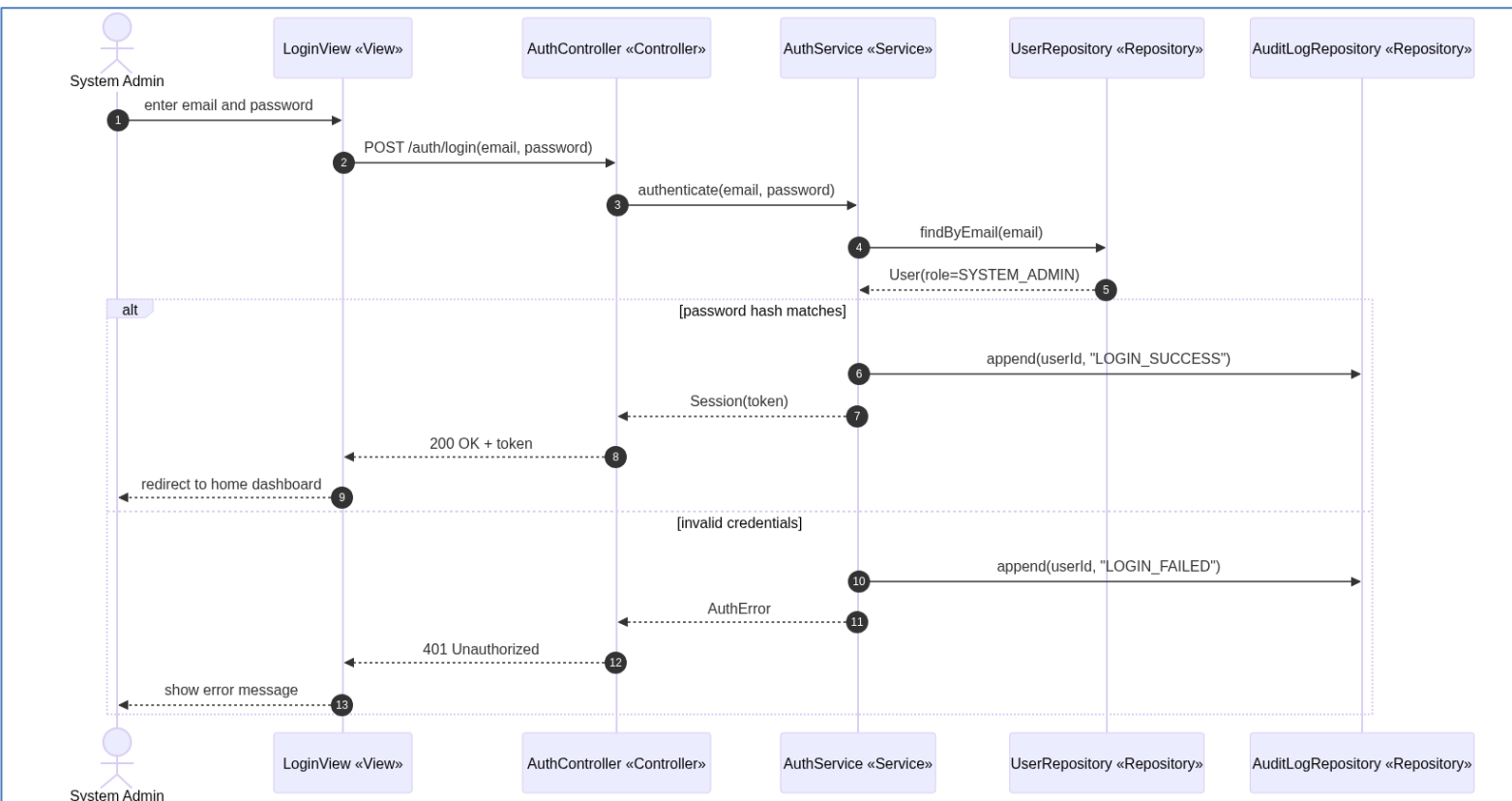


Figure 5.5: Sequence Diagram for UC-01 Authenticate and Manage Access

The object diagram shows a concrete snapshot during a successful admin login, and the ERD fragment shows the two tables this use case touches. The users table stores the account and role, and audit_logs stores one row per recorded action with a foreign key to users.

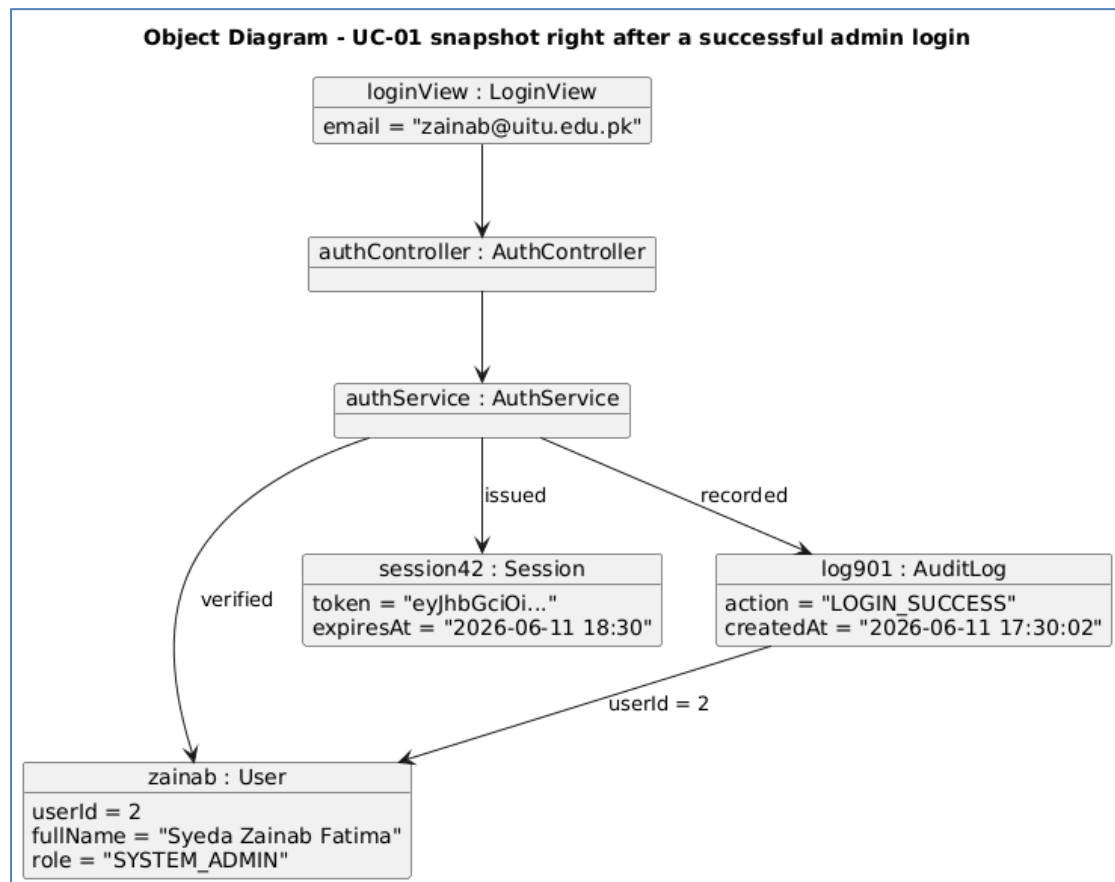


Figure 5.6: Object Diagram for UC-01 (snapshot of a successful admin login)

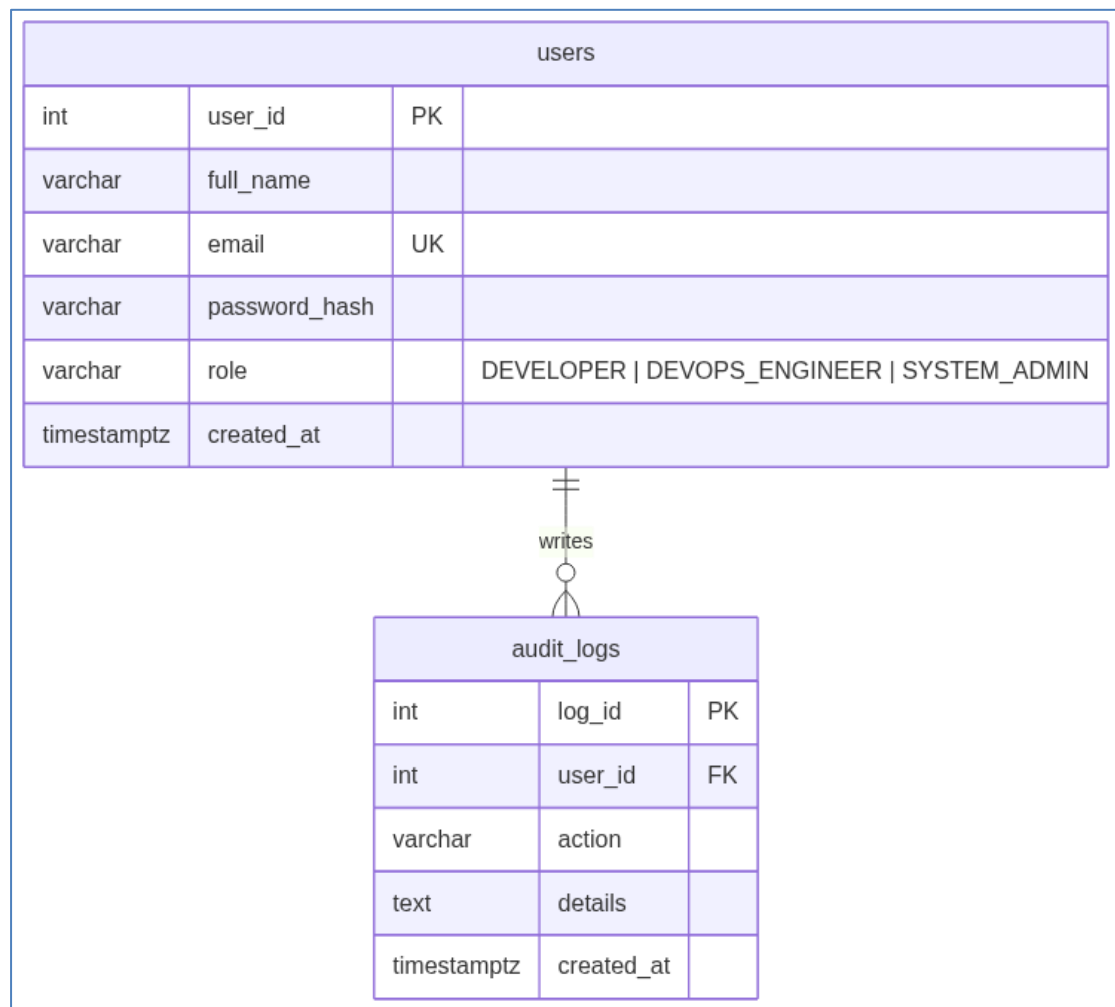


Figure 5.7: ERD fragment for UC-01 (users, audit_logs)

5.5 UC-02: Submit NL CI/CD Command

This use case belongs to the natural language interface of the platform. A developer types a command in plain English, for example a request to build and deploy a service to staging. The system parses the intent and either proceeds to pipeline generation (UC-03) or asks a clarification question when the intent is ambiguous, which is the extend relationship Clarify Ambiguous Intent shown in the use case diagram. The full LLM integration is scheduled for Milestone 3, so for Milestone 2 we designed the classes and interfaces and implemented a stub that recognizes a fixed set of commands.

In the presentation layer, the CommandConsole React component provides the chat style input and the CommandController exposes the /commands endpoint. In the

business layer, IntentParsingService extracts the action, target branch, environment, and urgency from the text, and ClarificationService produces a follow-up question when a required field is missing. In the persistence layer, CommandRepository stores every submitted command and its parsing result in the nl_commands table, which gives us an audit trail and training examples for later.

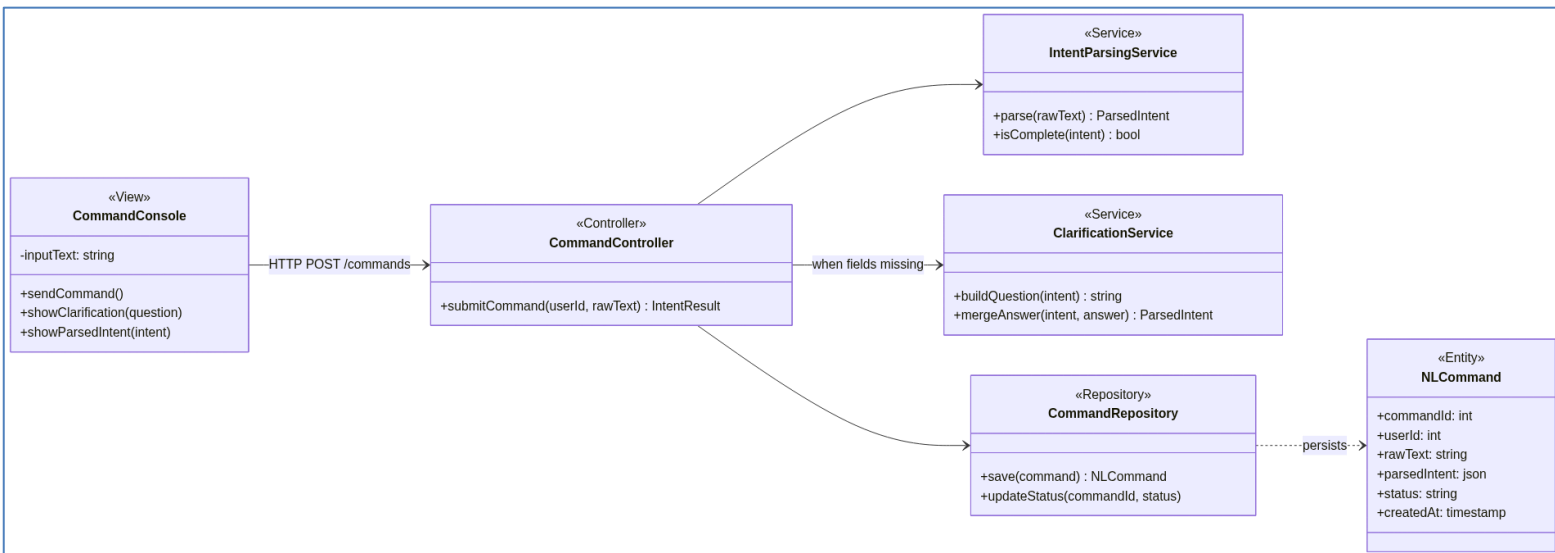


Figure 5.8: Class Diagram for UC-02 Submit NL CI/CD Command

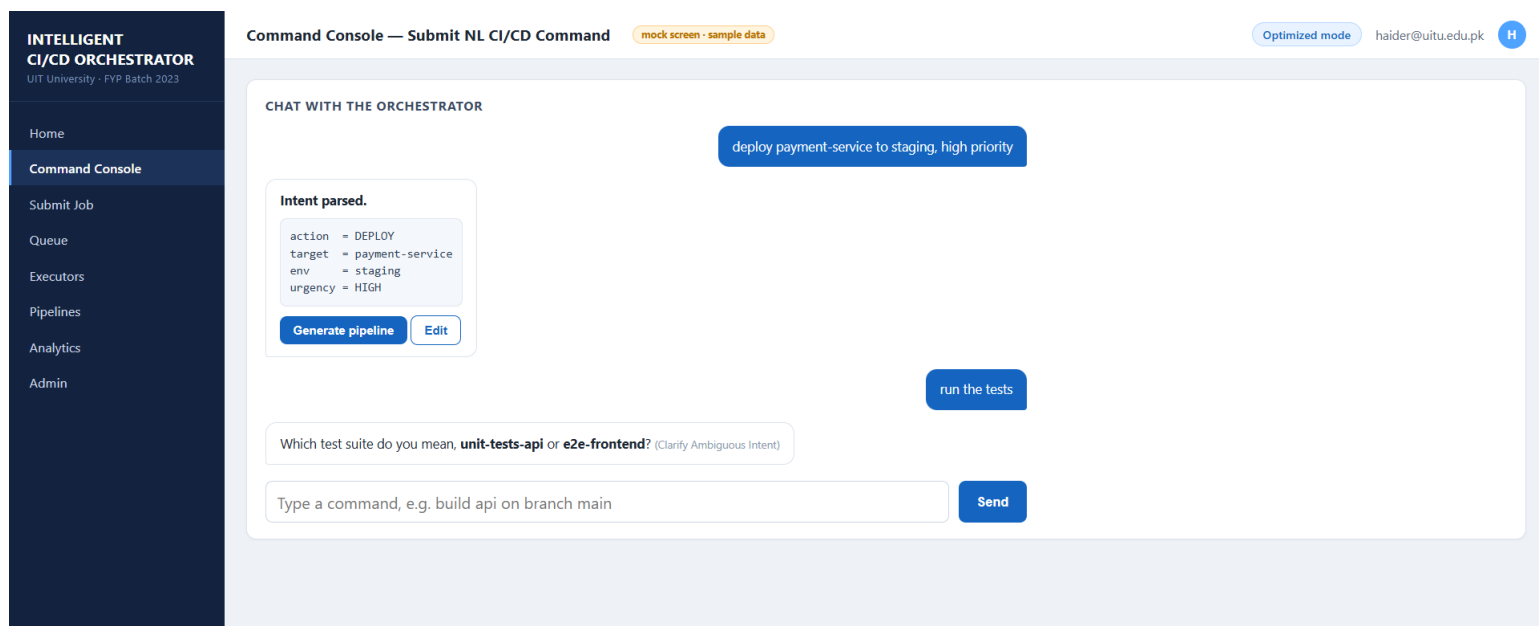


Figure 5.9: Interface design for UC-02 (command console with clarification prompt)

In the sequence diagram, CommandConsole posts the raw text to CommandController, which calls IntentParsingService. If parsing succeeds, the parsed intent is stored through CommandRepository and control passes to UC-03. If a field is missing, ClarificationService returns a question to the console and the sequence pauses until the user answers.

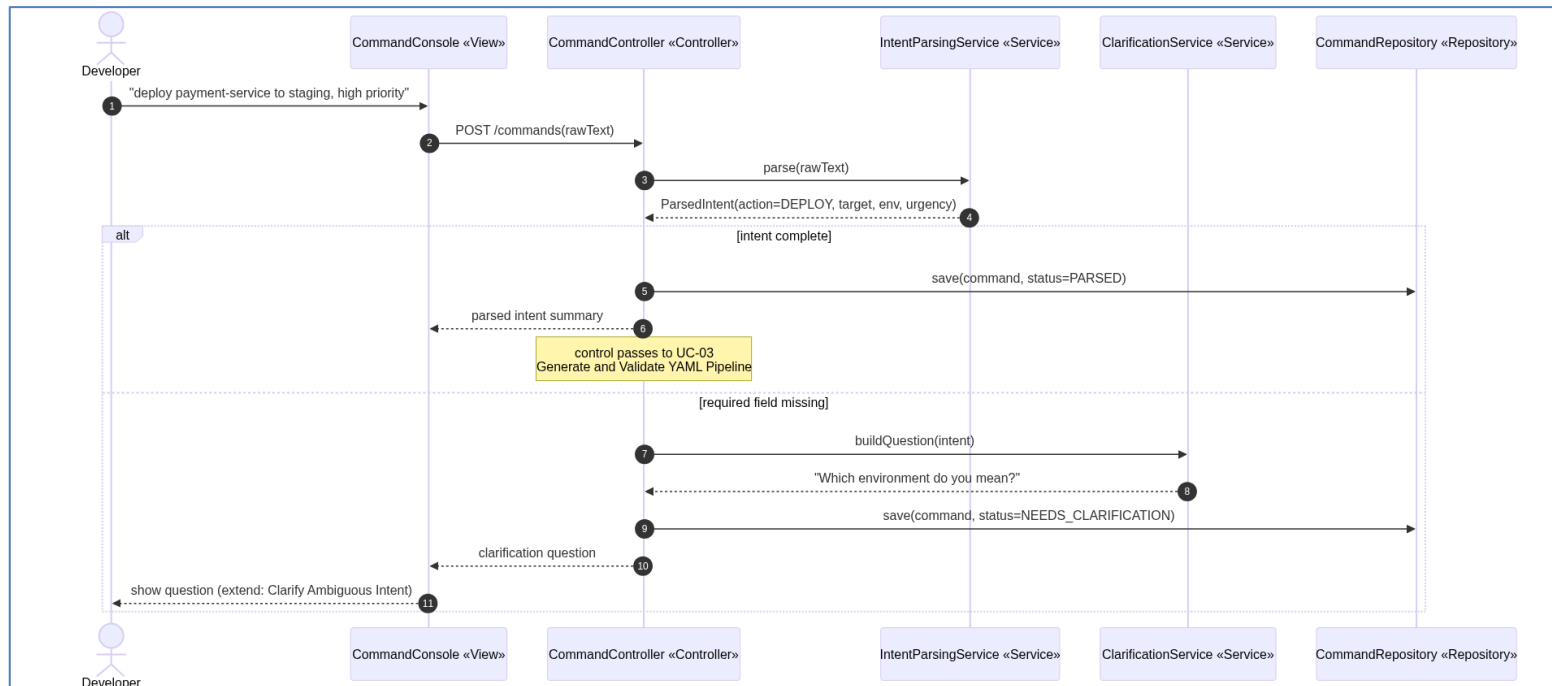


Figure 5.10: Sequence Diagram for UC-02 Submit NL CI/CD Command

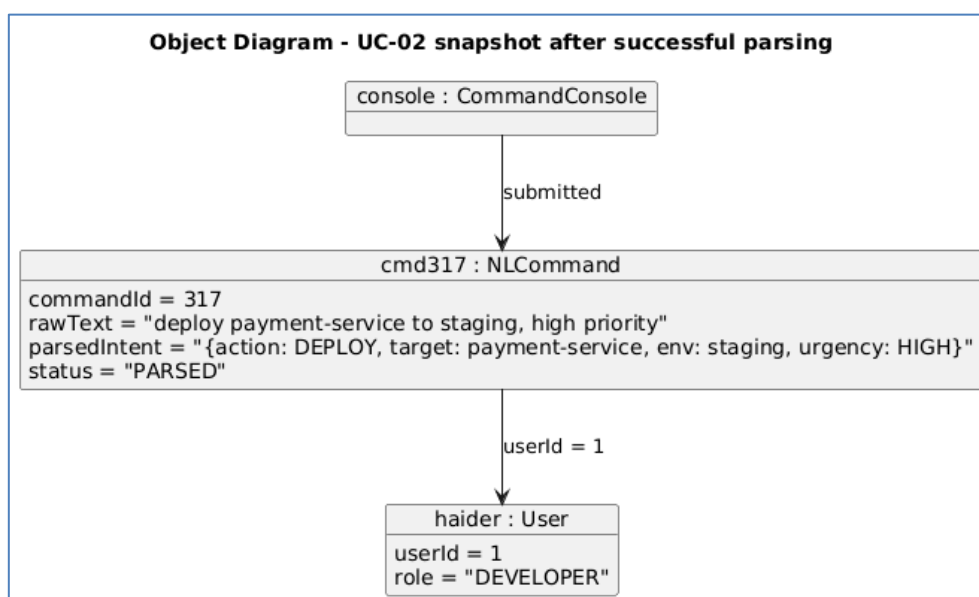


Figure 5.11: Object Diagram for UC-02 (parsed command snapshot)

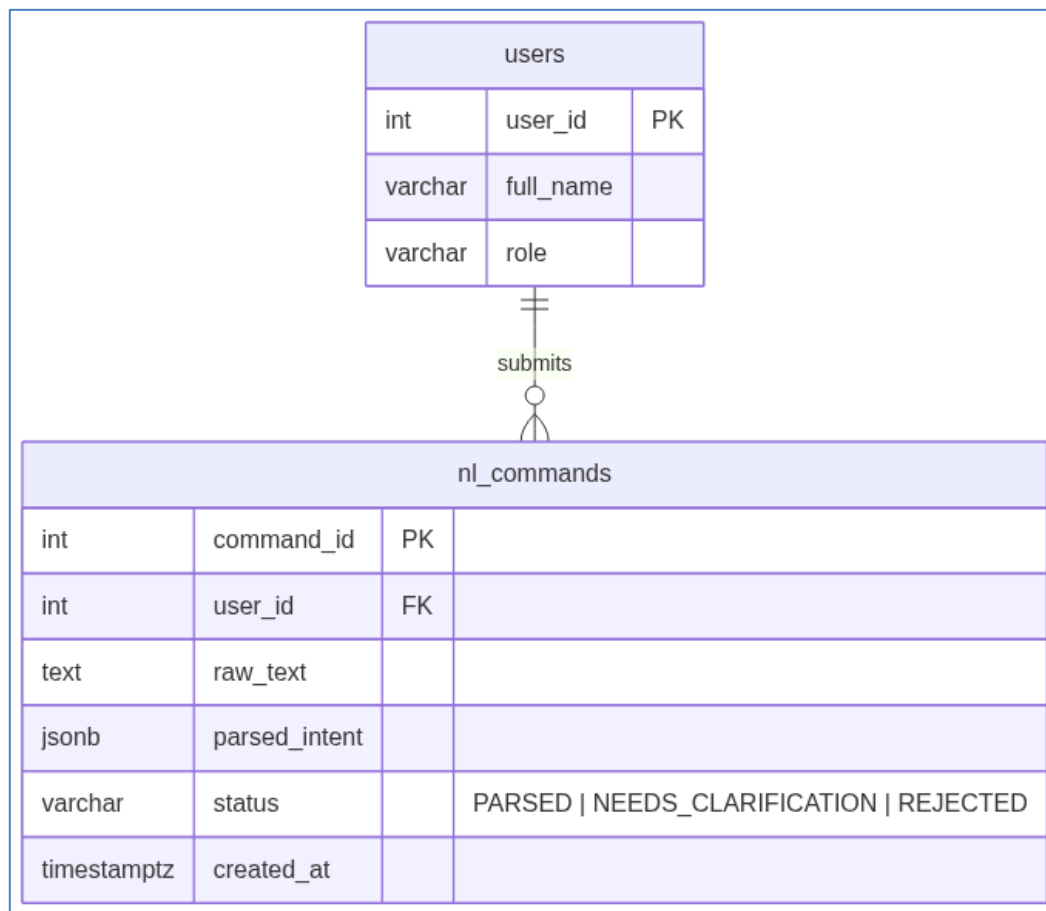


Figure 5.12: ERD fragment for UC-02 (users, nl_commands)

5.6 UC-03: Generate and Validate YAML Pipeline

This use case converts a parsed intent into a Jenkins pipeline definition and validates it before anything is executed. Validation matters because a syntactically wrong pipeline wastes queue capacity and confuses users, a risk that our literature review also identified for LLM generated configurations. Like UC-02, the full generation logic arrives with Milestone 3; the design is fixed now so that the database schema and class structure already reserve a place for it.

The presentation layer shows the generated pipeline to the user in the PipelinePreview component before approval. In the business layer, PipelineGenerationService renders the pipeline text from the parsed intent, and PipelineValidationService checks structure and required fields against the expected Jenkins schema. In the persistence layer, PipelineRepository stores each generated pipeline and its validation verdict in the generated_pipelines table, linked to the originating command.

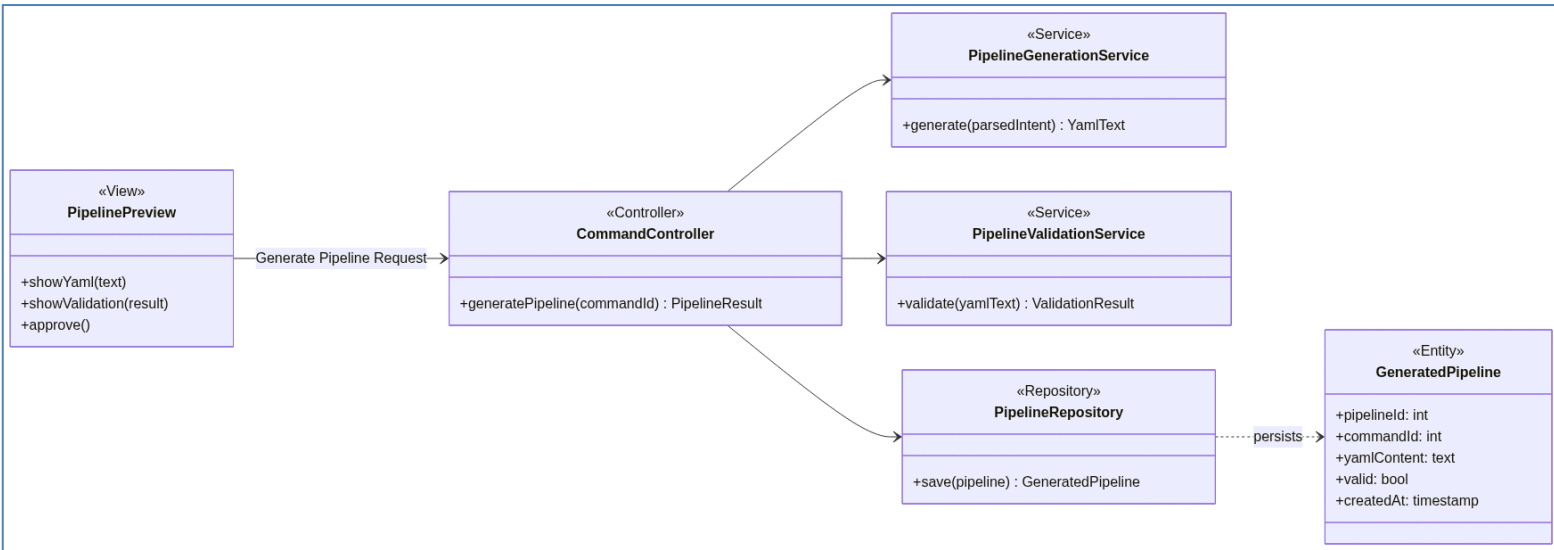


Figure 5.13: Class Diagram for UC-03 Generate and Validate YAML Pipeline



Figure 5.14: Interface design for UC-03 (pipeline preview with validation result)

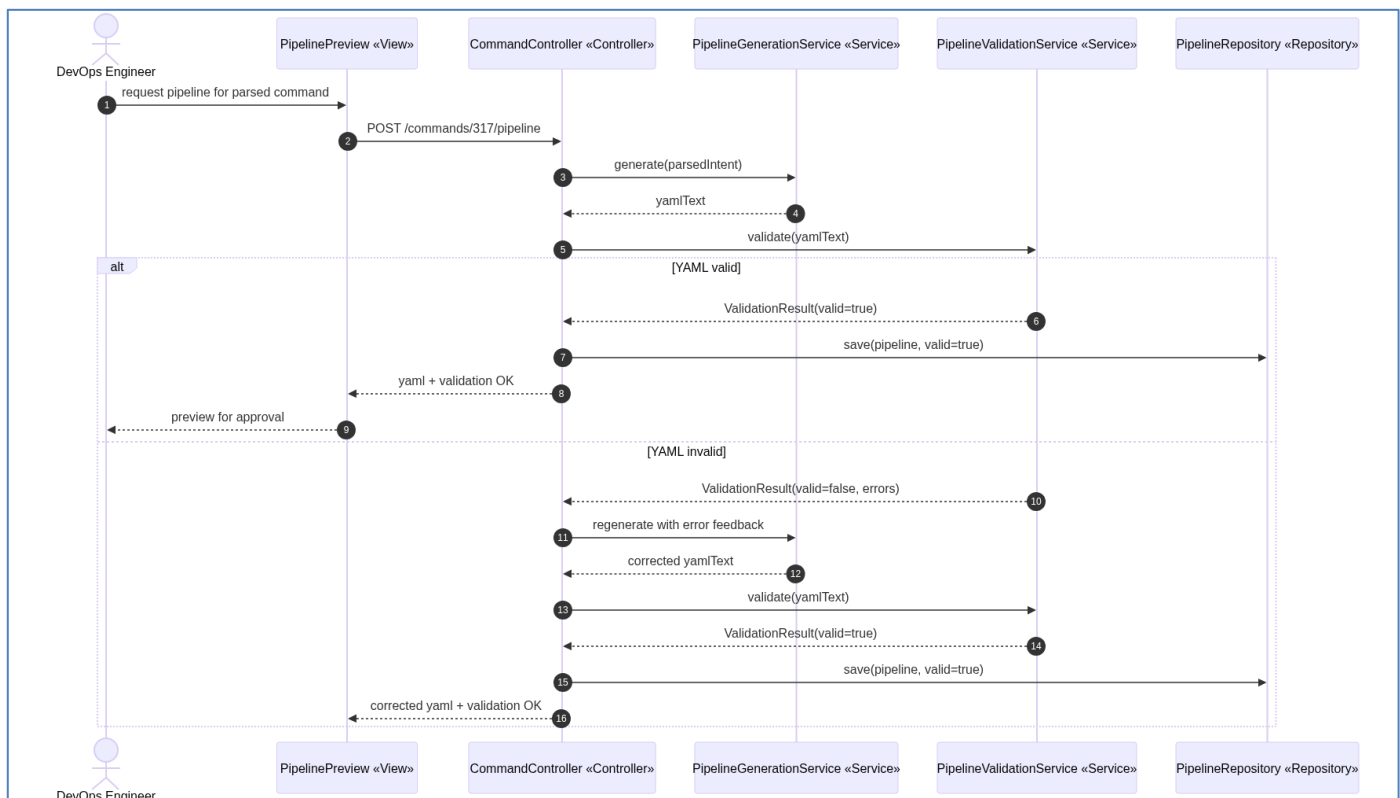


Figure 5.15: Sequence Diagram for UC-03 Generate and Validate YAML Pipeline

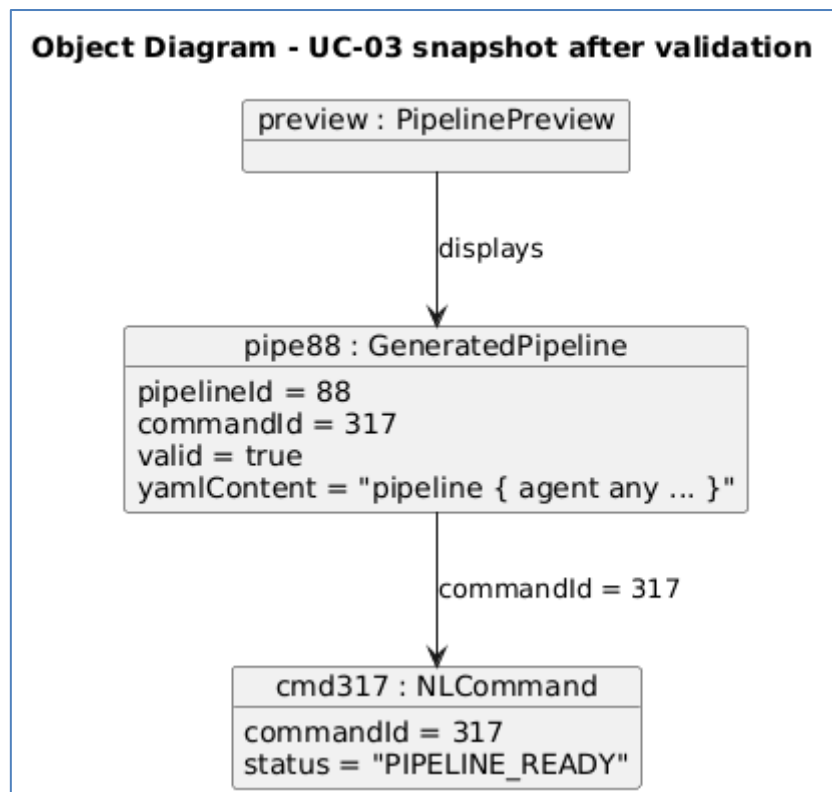


Figure 5.16: Object Diagram for UC-03 (validated pipeline snapshot)

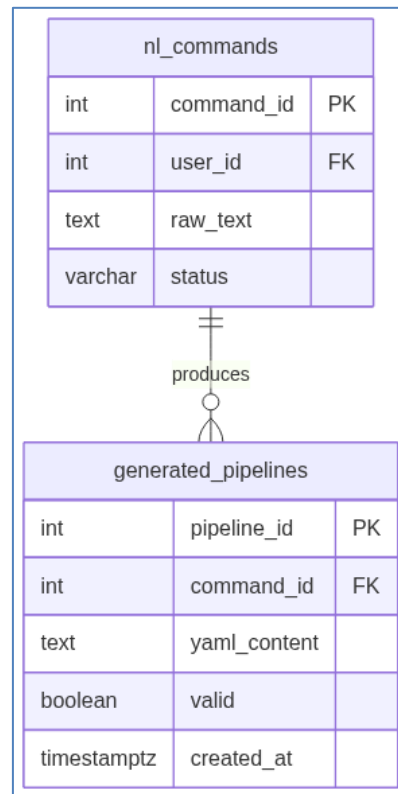


Figure 5.17: ERD fragment for UC-03 (nl_commands, generated_pipelines)

5.7 UC-04: Submit Prioritized Job

This use case is the entry point of our POC. A DevOps Engineer submits a Jenkins job and assigns it a priority level of High, Medium, or Low, and optionally declares that the job depends on other jobs. In Jenkins terms, the priority and the dependency list are job properties: a job property is extra configuration attached to a job definition, which Jenkins stores with the job and shows on the job configuration page. Our plugin contributes the `PriorityJobProperty` class for this purpose, so the user sets the priority directly in the familiar Jenkins job configuration screen, and our dashboard offers the same fields in a friendlier form.

In the presentation layer, the JobSubmissionForm React component and the Jenkins job configuration page both act as boundaries; PriorityJobProperty (a Jenkins JobProperty subclass) carries the user choices into the plugin world. In the business layer, no scoring happens yet at submission time; a lightweight JobIntakeService validates that declared dependencies exist and do not form a cycle. In the persistence layer, JobRepository and JobDependencyRepository write the jobs, and job_dependencies tables, so the dependency graph is known to both the plugin and the dashboard.

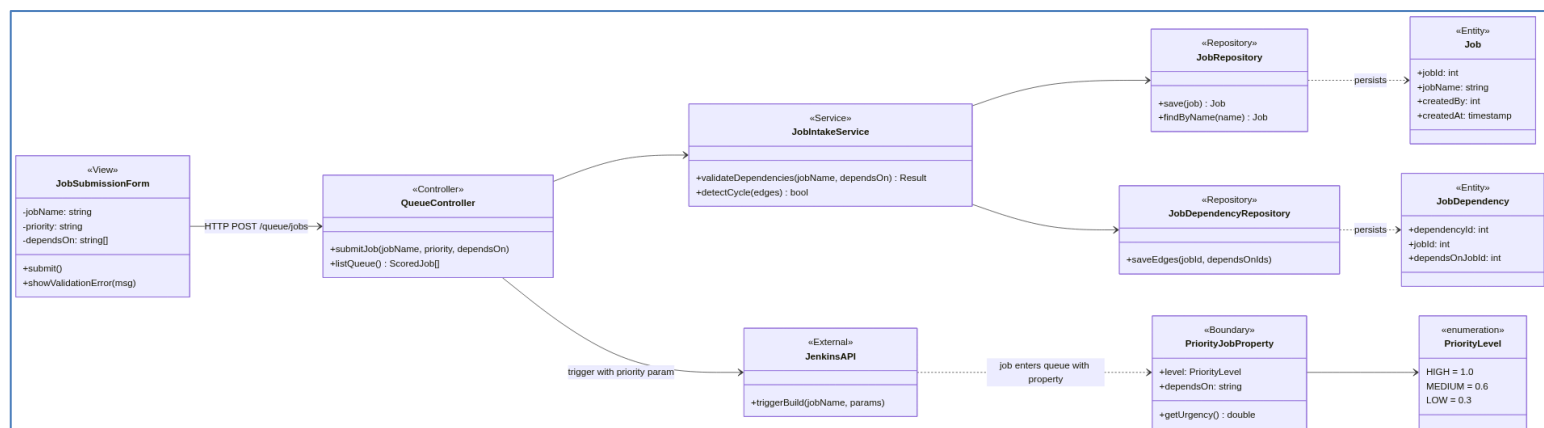


Figure 5.18: Class Diagram for UC-04 Submit Prioritized Job

Figure 5.19: Interface design for UC-04 (job submission form with priority and dependencies)

The sequence diagram shows both submission paths. When the user submits through our dashboard, JobSubmissionForm calls QueueController, which triggers the Jenkins job through the Jenkins REST API with the priority parameter set. When the user

submits directly in Jenkins, PriorityJobProperty already holds the configured values. In both cases the job enters the Jenkins queue, where UC-05 takes over.

Figure 5.20: Sequence Diagram for UC-04 Submit Prioritized Job

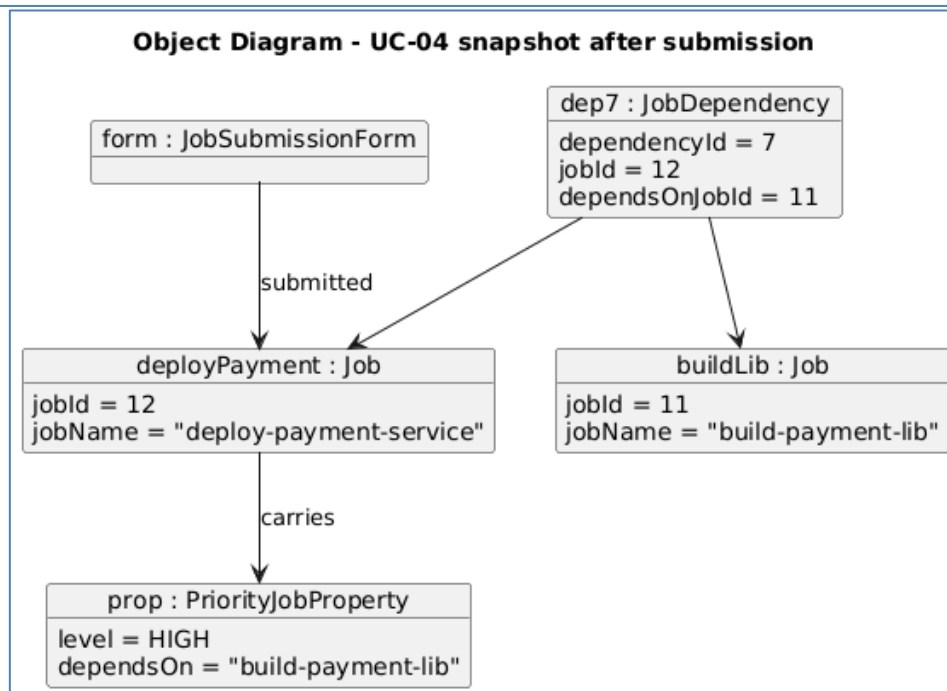
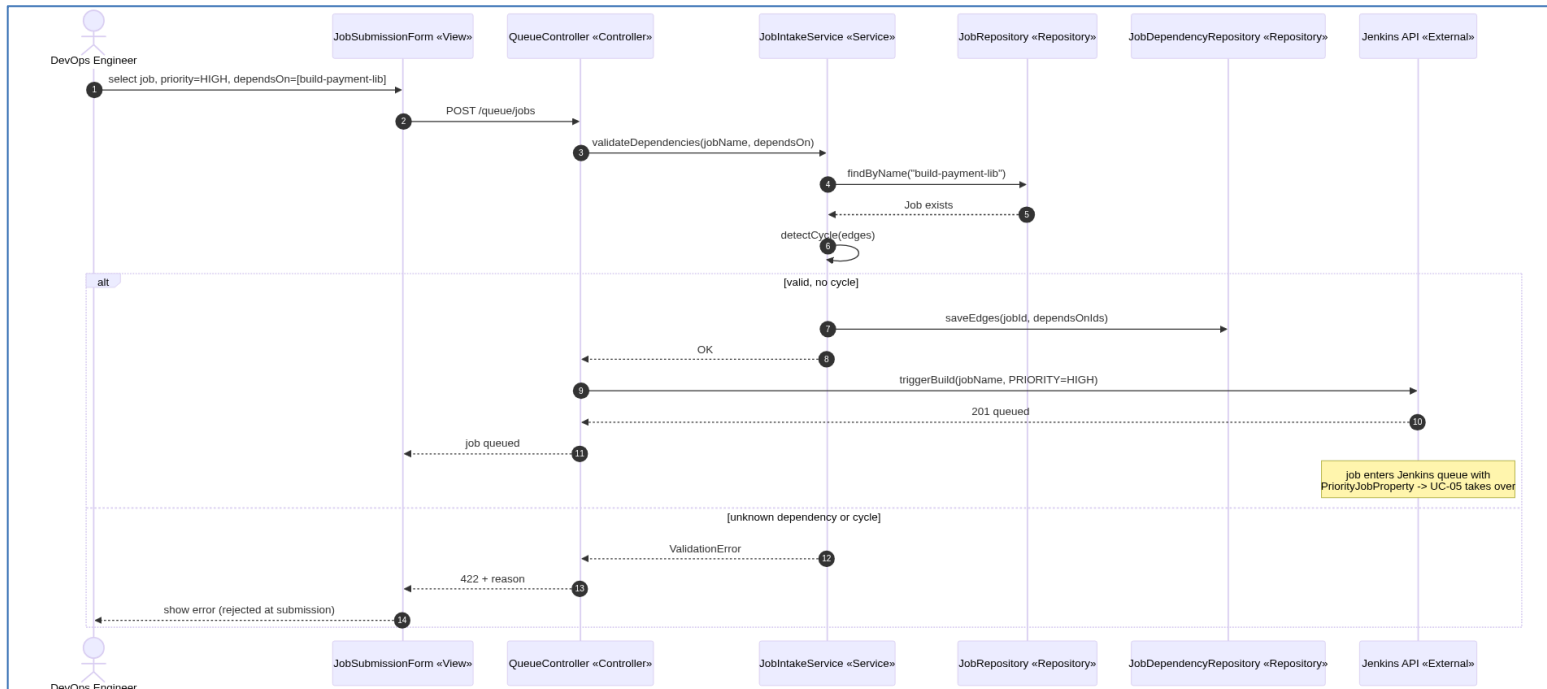


Figure 5.21: Object Diagram for UC-04 (a submitted job with priority High and one dependency)

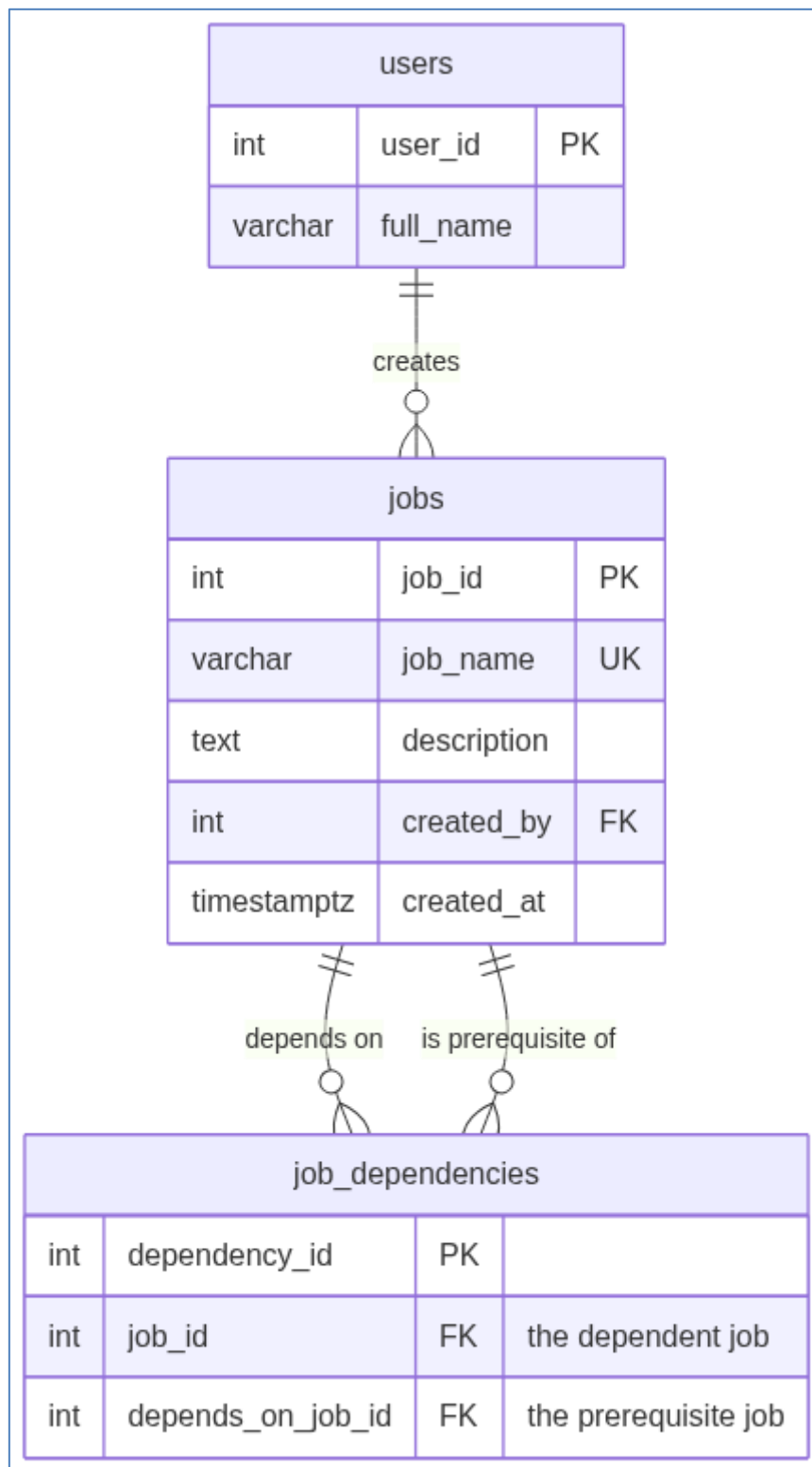


Figure 5.22: ERD fragment for UC-04 (jobs, job_dependencies)

5.8 UC-05: Optimize Dynamic Queue (POC Core)

This is the core use case of our Proof of Concept. It runs entirely inside our Jenkins plugin and has no direct human actor: it is triggered every time a job enters the Jenkins build queue. The use case includes three included sub-functions that we show as include relationships in the use case diagram: Group Dependent Jobs, Estimate Execution Time, and Compute Priority Score. Their algorithms are analyzed one by one in Chapter 6; here we present the structural design.

The flow is as follows. When a job becomes buildable in the Jenkins queue, our `QueueStateListener` (which Jenkins notifies through its `QueueListener` extension point) registers the job with the scheduling services. `DependencyGroupService` looks up the declared dependencies and places jobs that depend on each other into one group using a union-find structure, so a group is treated as a single logical unit. `ExecutionTimeEstimator` asks `BuildHistoryService` for past builds, compares the new job with previously executed jobs, and produces an estimated duration. `PriorityScoringService` then combines the urgency value derived from the user priority, the dependency factor of the group, and the execution time factor into one score using the weighted formula given in Chapter 6. Finally `HeapSchedulerService` inserts the scored job into a max-heap, which keeps the highest score on top. The actual gate that holds jobs back, `PriorityDispatcher`, belongs to UC-06.

In the layered view, `QueueStateListener` is the boundary that receives queue events from Jenkins; `DependencyGroupService`, `ExecutionTimeEstimator`, `PriorityScoringService`, and `HeapSchedulerService` form the business layer; `BuildHistoryService` is the persistence class that reads Jenkins build history through the Jenkins API, and `MetricsPublisher` later reports the scoring decisions to PostgreSQL through the FastAPI backend. `ScoredJob`, `JobGroup`, and `BuildRecord` are the entities passed between these classes, and `PriorityConfiguration` holds the weights so our supervisor can tune them without code changes.

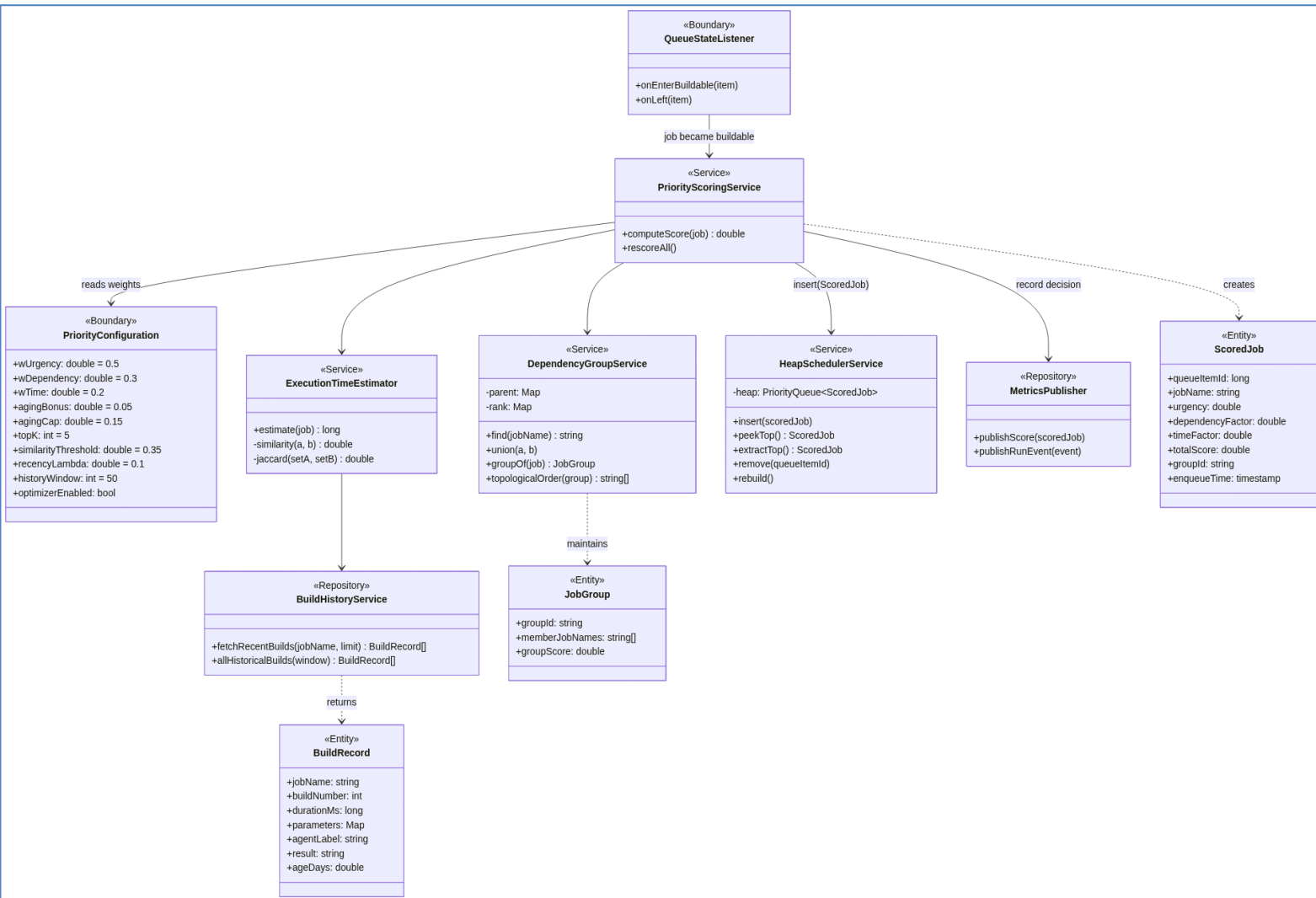


Figure 5.23: Class Diagram for UC-05 Optimize Dynamic Queue

The queue board is the interface of this use case. It shows the live queue ordered by score, and for each waiting job it displays the urgency, the dependency group it belongs to, the estimated duration with the historical builds that produced the estimate, and the resulting score, so the optimization is transparent rather than a black box.

INTELLIGENT
CI/CD ORCHESTRATOR

UIT University · FYP Batch 2023

[Home](#)
[Command Console](#)
[Submit Job](#)
[Queue](#)
[Executors](#)
[Pipelines](#)
[Analytics](#)
[Admin](#)

Live Queue — Optimize Dynamic Queue

mock screen · sample data

Optimized mode

haider@uit.edu.pk

H

#	JOB	PRIORITY	GROUP	EST. DURATION	U / D / T FACTORS	SCORE
1	deploy-payment-service enqueued 17:31:00	High	—	3.0 min 5 similar builds		0.667
2	build-api enqueued 17:31:05 · runs 1st in G1	Low	G1	2.0 min 5 similar builds		0.650 group score
3	integration-tests-api enqueued 17:31:06 · needs build-api	Medium	G1	7.0 min 4 similar builds		0.650 own 0.633 -- group
4	build-frontend enqueued 17:30:58	Medium	—	8.0 min 5 similar builds		0.300

Score = 0.5·U + 0.3·D + 0.2·T · aging +0.05 per 5 min waiting (cap 0.15) · bars show U, D, T from top to bottom · numbers match Appendix C worked example.

Figure 5.24: Interface design for UC-05 (live queue board with priority scores)

The sequence diagram below shows one job passing through the full scoring pipeline, including the call into build history and the heap insertion. Note that when any member of a dependency group is rescored, the group score is recomputed once and shared by all members, which is what makes the group behave as a single unit in the queue.

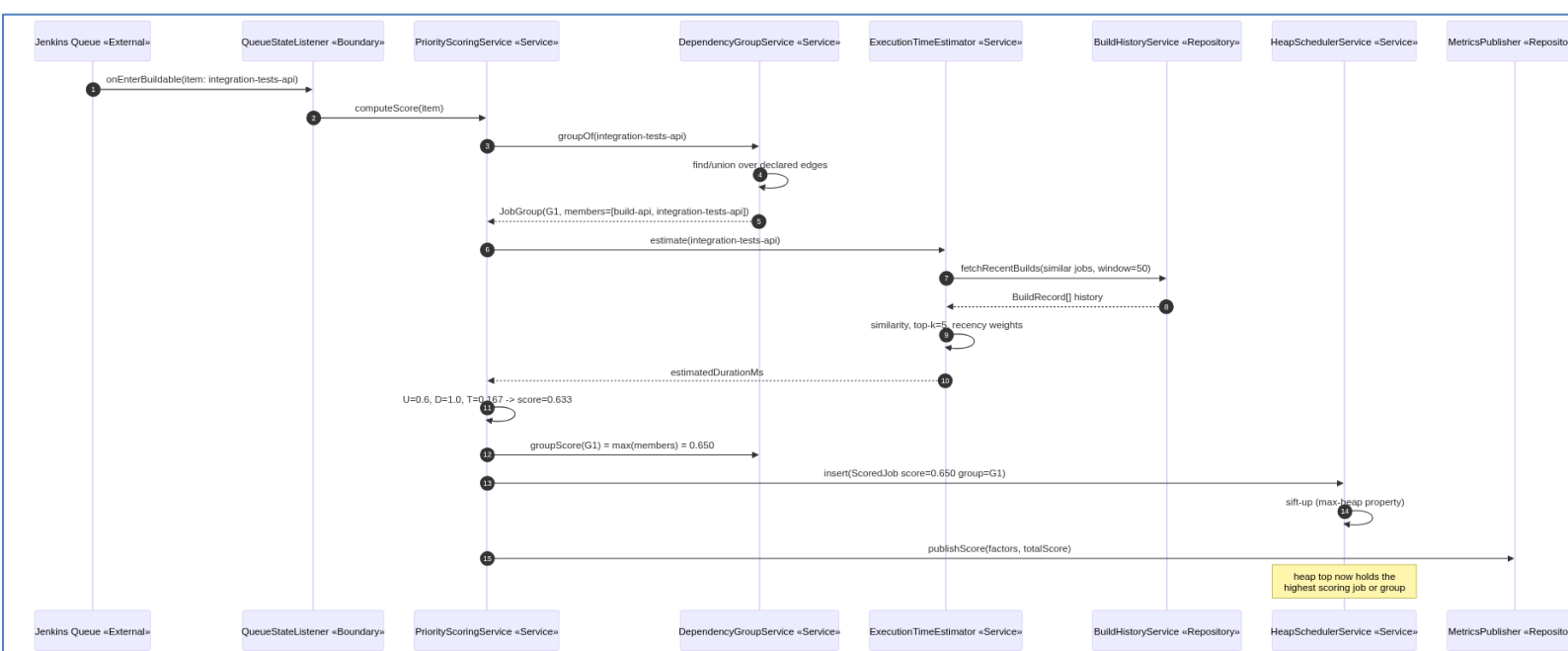


Figure 5.25: Sequence Diagram for UC-05 Optimize Dynamic Queue

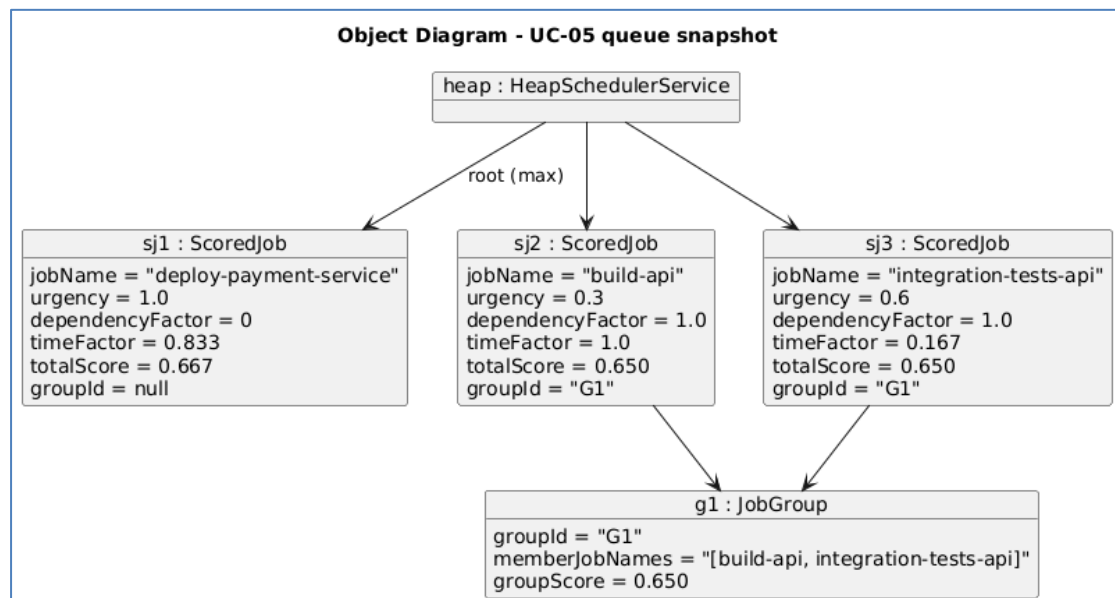


Figure 5.26: Object Diagram for UC-05 (three queued jobs, two grouped, with computed scores)

The ERD fragment for this use case covers the tables where scoring decisions are recorded for later analysis: `job_runs` stores one row per queued execution with its factors and final score, and `dependency_groups` stores the groups. We record the factors separately (`urgency_factor`, `dependency_factor`, `time_factor`) precisely so that the dashboard in UC-08 can explain every scheduling decision.

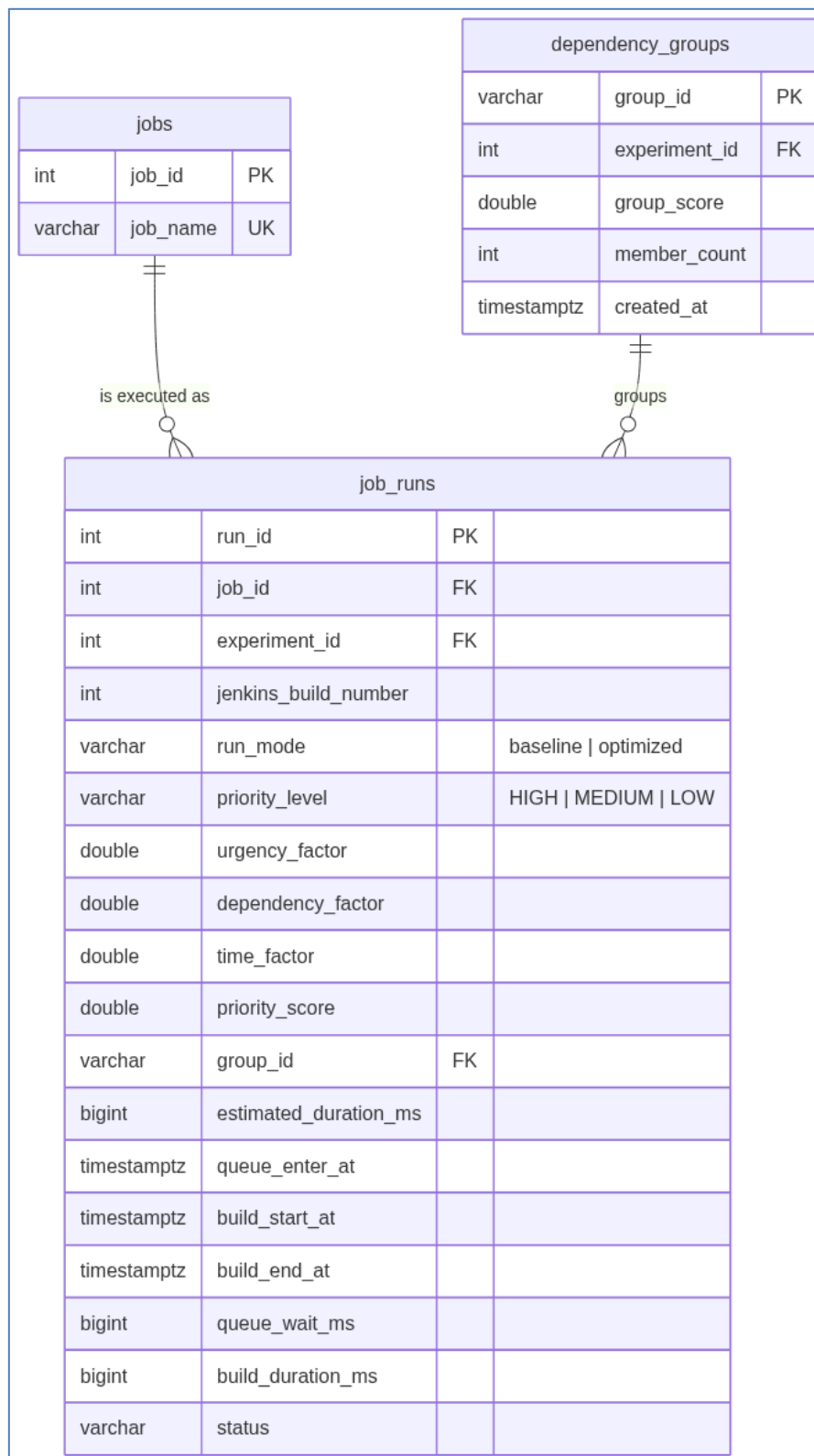


Figure 5.27: ERD fragment for UC-05 (jobs, job_runs, dependency_groups)

5.9 UC-06: Dispatch Job to Runner

This use case turns the heap order into actual Jenkins behavior. Jenkins decides which queued item may start by consulting every registered QueueTaskDispatcher, an extension point whose `canRun` method either returns null to allow execution or returns a cause of blockage to hold the item back [32]. Our PriorityDispatcher implements exactly this contract: when Jenkins asks about an item, the dispatcher allows it only if it is the current top of our max-heap (or a member of the top group) and an executor is free; every other item receives a blockage cause that names the higher priority job it is waiting for. The effect is that Jenkins itself still performs the execution, but the order in which items leave the queue is the order of our heap. When the running job finishes, the next heap top is released, and Jenkins re-evaluates blocked items.

In the layered view, PriorityDispatcher is a boundary class called by Jenkins; it contains no scheduling logic itself and only asks HeapSchedulerService for the current decision, which keeps the business rules testable outside Jenkins. BuildMetricsListener observes the moment the build actually starts and records the queue waiting time through MetricsPublisher.

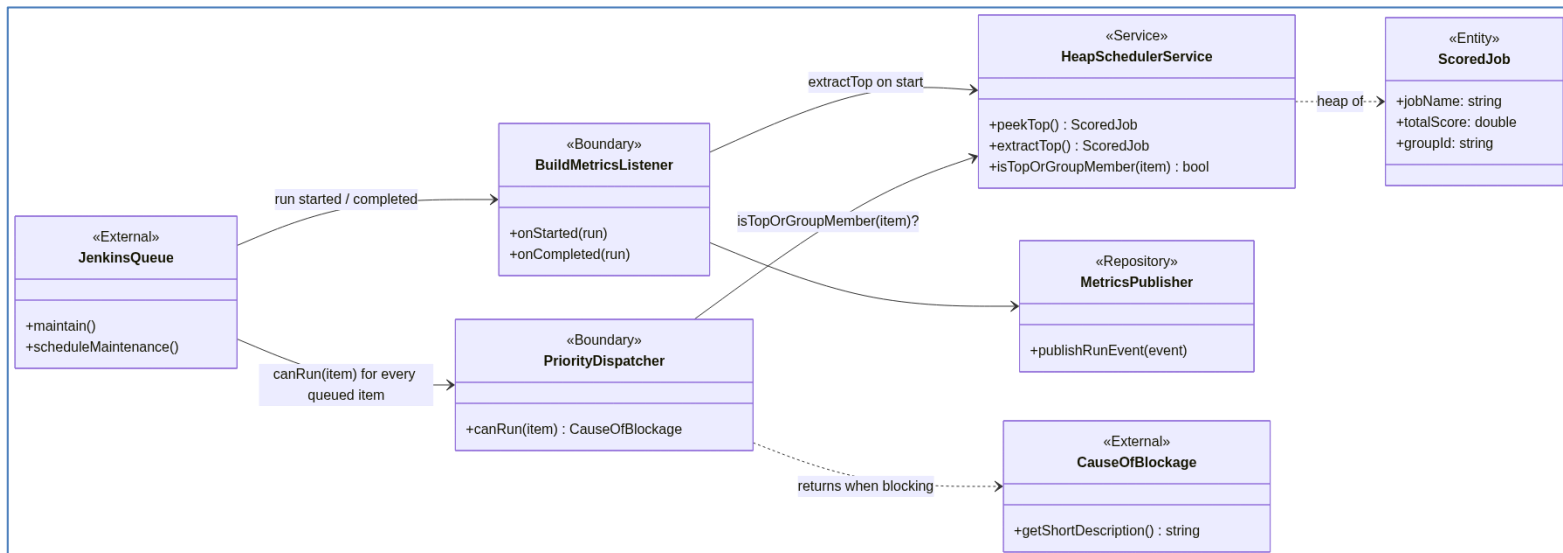


Figure 5.28: Class Diagram for UC-06 Dispatch Job to Runner

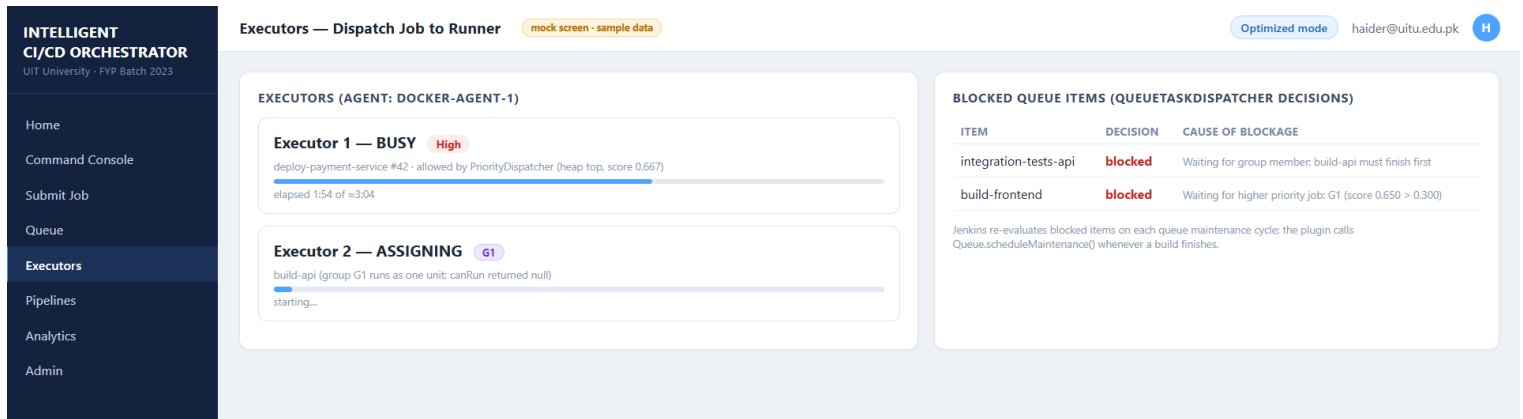


Figure 5.29: Interface design for UC-06 (executors panel showing the dispatch decision)

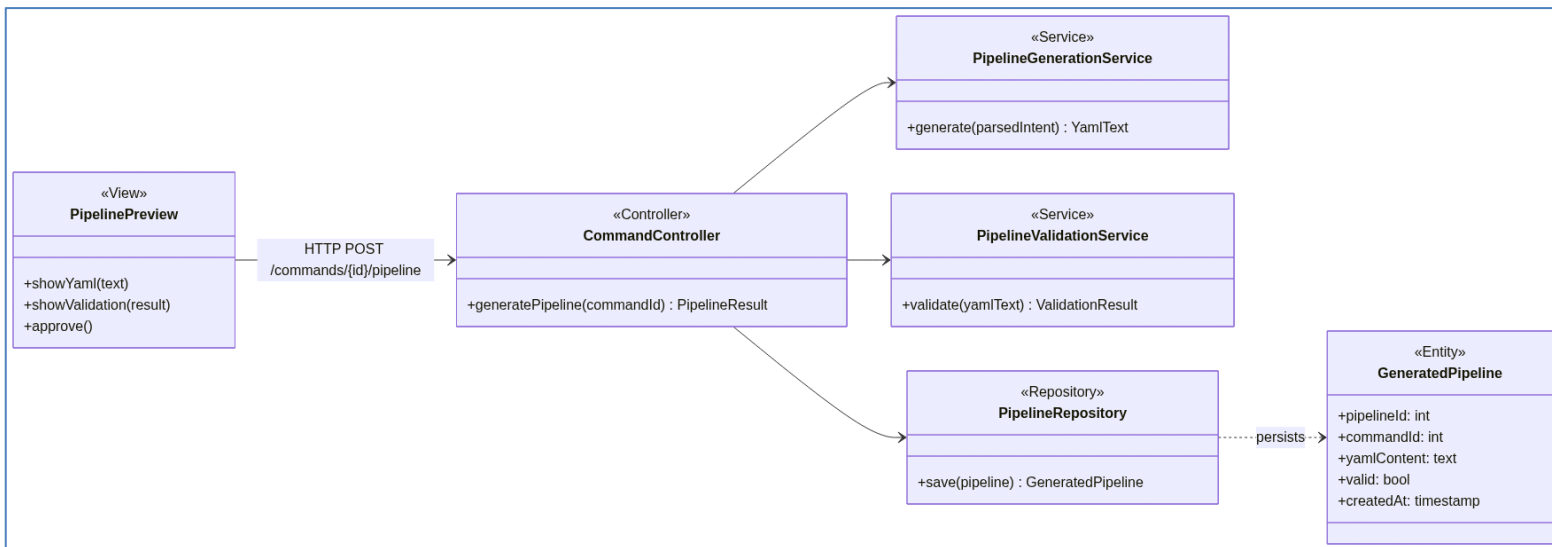


Figure 5.30: Sequence Diagram for UC-06 Dispatch Job to Runner

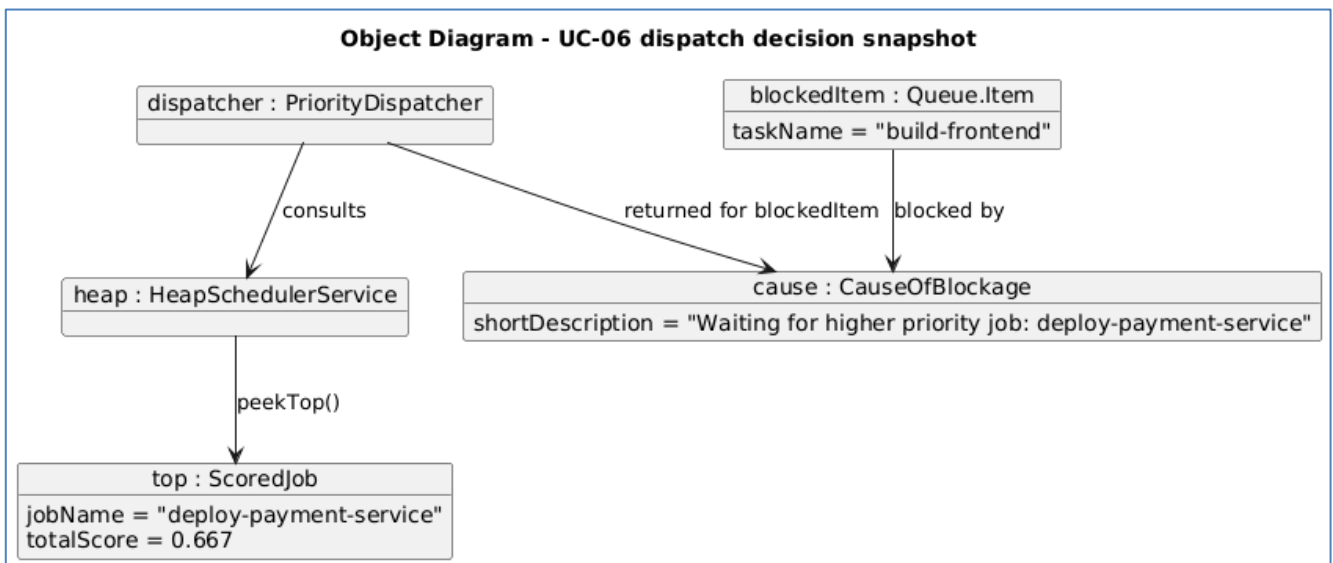


Figure 5.31: Object Diagram for UC-06 (dispatcher allowing the heap top, blocking the rest)

This use case updates the job_runs table with the dispatch timestamps (queue_enter_at, build_start_at), from which the queue waiting time is computed. The ERD is shared with UC-05, so we do not repeat it here.

5.10 UC-07: Execute Pipeline Stages

Once dispatched, the job runs on a Jenkins executor through the usual build, test, and deploy stages, fetching source code from the Git repository, which we model as the include relationship Fetch Source Code. Our plugin deliberately does not interfere with execution; this boundary keeps the POC safe, because a scheduling bug can delay a job but can never corrupt a build. The system observes execution through BuildMetricsListener, which Jenkins notifies when a run starts and completes, and which records the build duration and result. These records then serve two purposes: they feed the metrics tables for the dashboard, and they become the historical data that ExecutionTimeEstimator uses for future estimates, closing the loop of the POC.

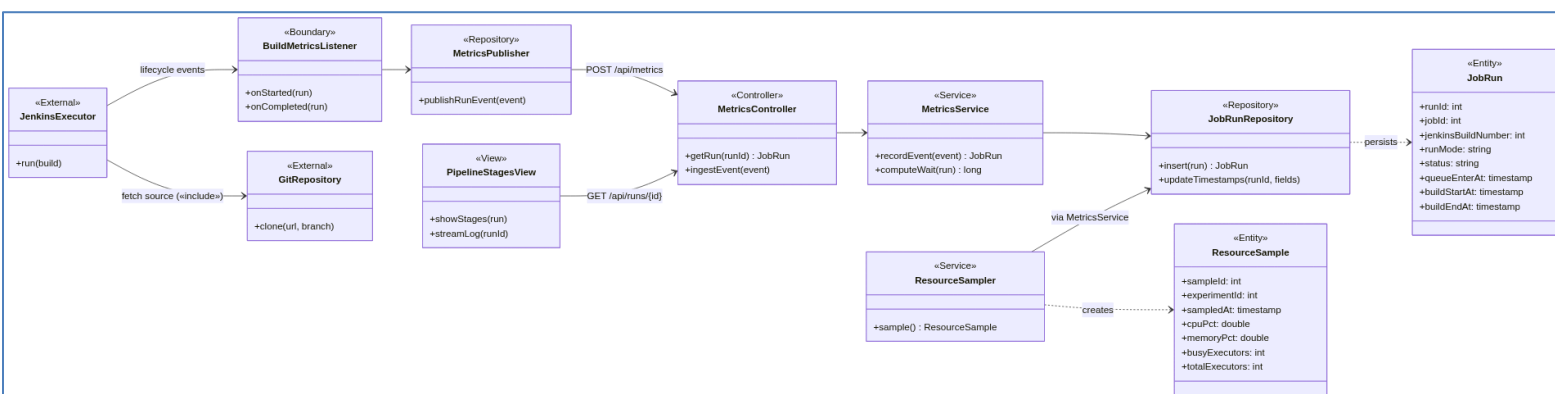


Figure 5.32: Class Diagram for UC-07 Execute Pipeline Stages

INTELLIGENT CI/CD ORCHESTRATOR



Figure 5.33: Interface design for UC-07 (pipeline stages view with live status)

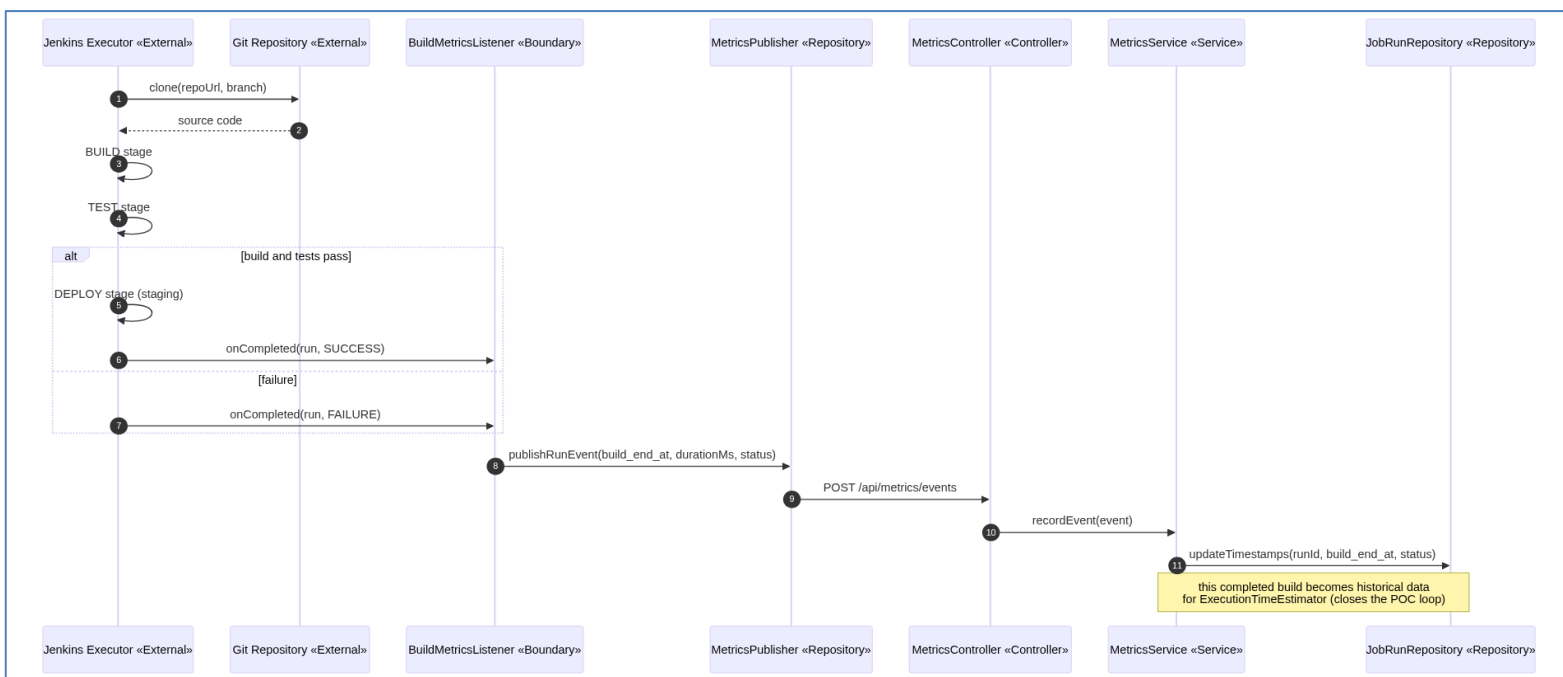


Figure 5.34: Sequence Diagram for UC-07 Execute Pipeline Stages

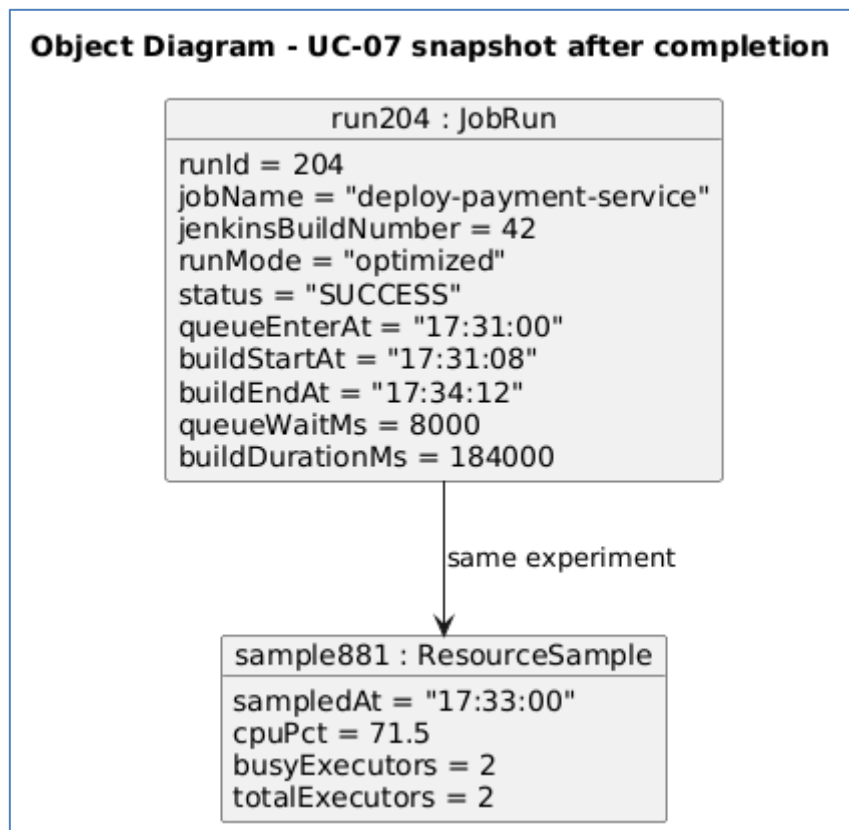


Figure 5.35: Object Diagram for UC-07 (a completed run with recorded duration)

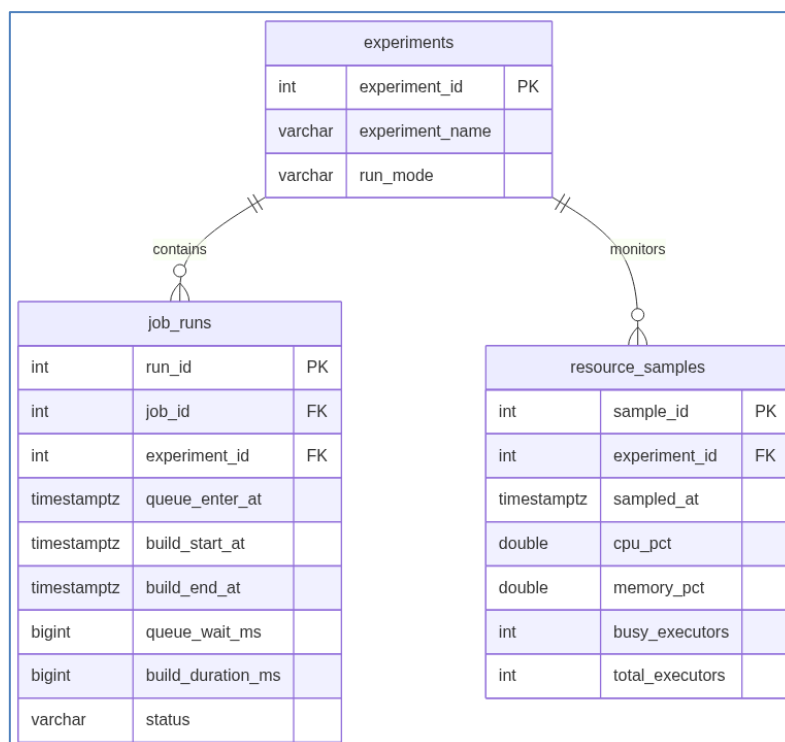


Figure 5.36: ERD fragment for UC-07 (job_runs, resource_samples)

5.11 UC-08: Monitor Dashboard and Analyze Performance

This use case presents the analytical dashboard that demonstrates whether our plugin actually optimizes the workflow, which is the validation requirement of the POC. The dashboard has two modes. The live mode shows the current queue, running builds, and resource usage. The comparison mode loads two experiments from the database, the baseline run with default Jenkins scheduling and the optimized run with our plugin enabled, and presents the key performance indicators side by side: average and maximum queue waiting time, average build completion time, throughput, executor utilization, and the average waiting time of high priority jobs. Status alerts, modeled as the extend relationship Trigger Status Alerts, notify users about completed or failed runs through the notification service.

In the presentation layer, the ComparisonDashboard and AlertsPanel React components render the views and MetricsController and ComparisonController expose the read APIs. In the business layer, MetricsService aggregates raw rows into KPIs and ComparisonService computes the baseline versus optimized deltas including percentage improvements. In the persistence layer, JobRunRepository, ExperimentRepository, and MetricsRepository read the tables written during UC-05, UC-06, and UC-07.

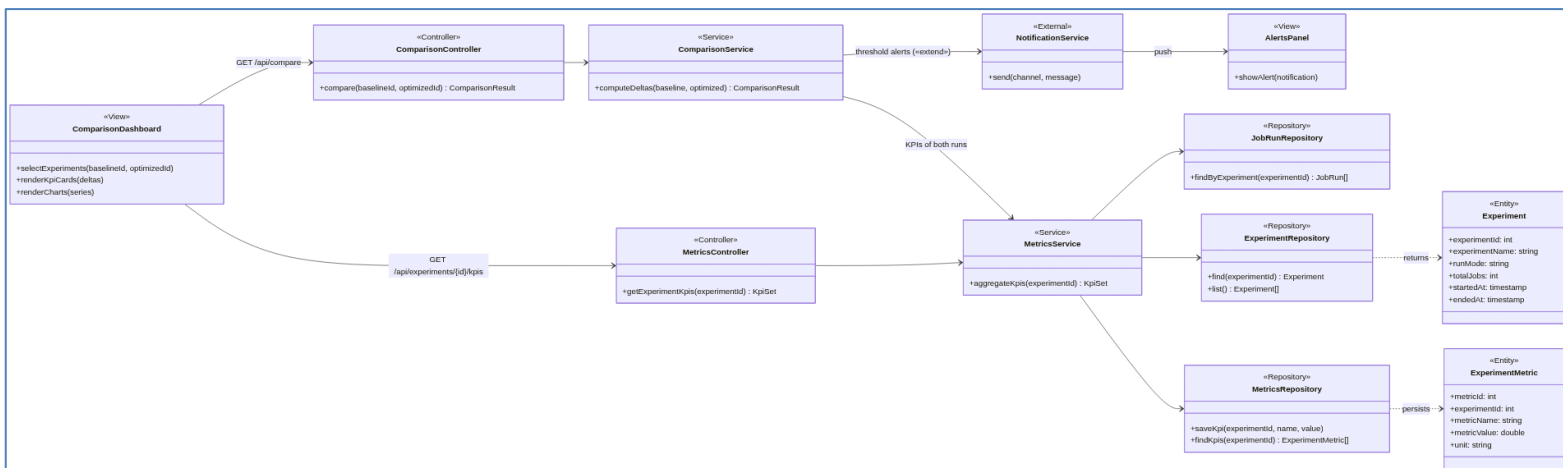


Figure 5.37: Class Diagram for UC-08 Monitor Dashboard and Analyze Performance

INTELLIGENT CI/CD ORCHESTRATOR

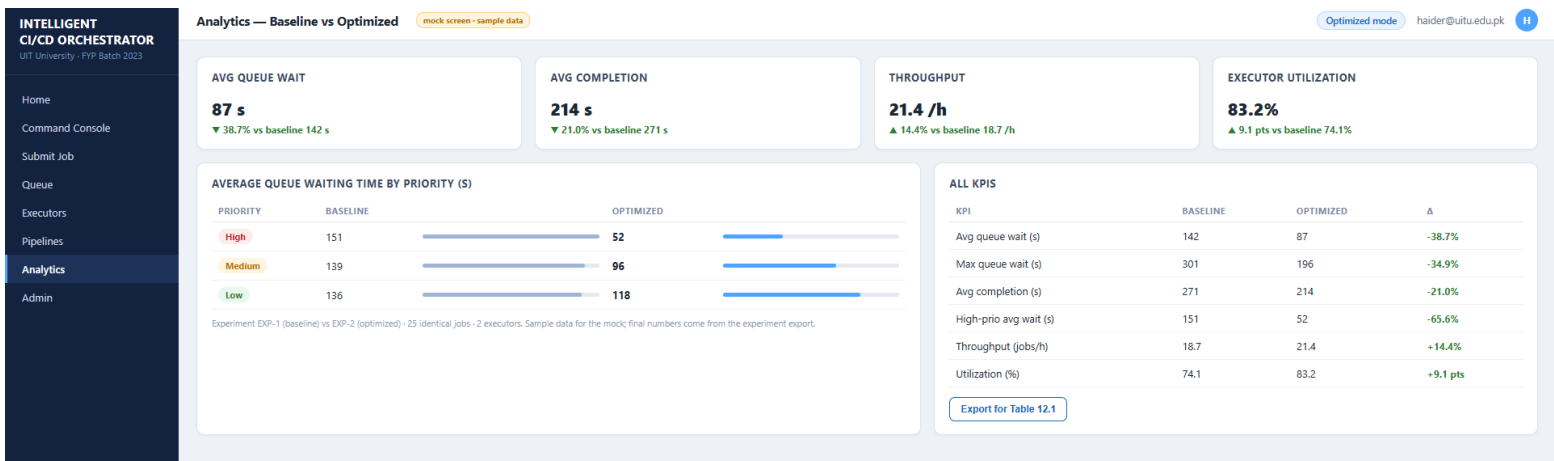


Figure 5.38: Interface design for UC-08 (baseline versus optimized comparison dashboard)

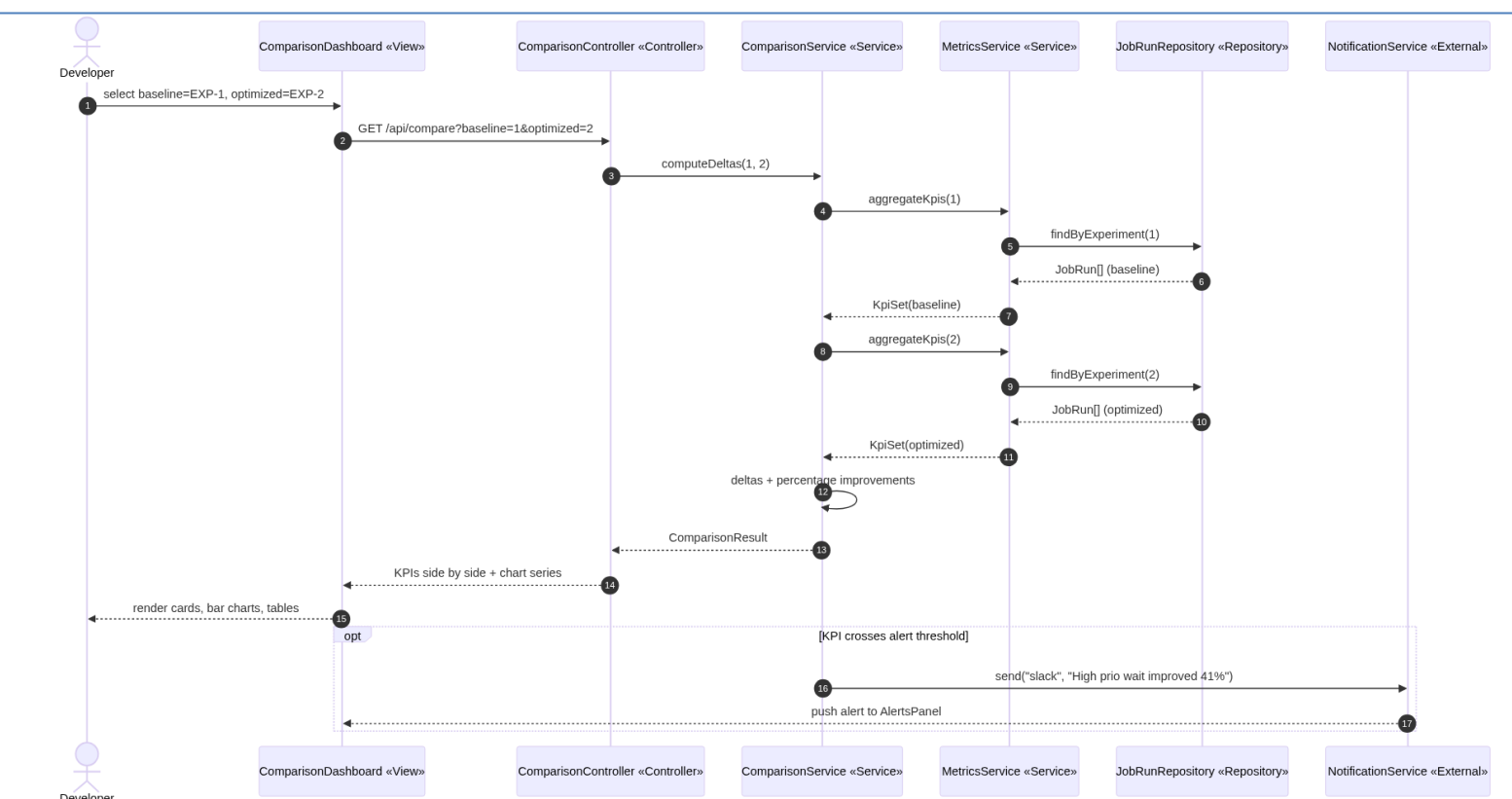


Figure 5.39: Sequence Diagram for UC-08 Monitor Dashboard and Analyze Performance

Object Diagram - UC-08 comparison snapshot

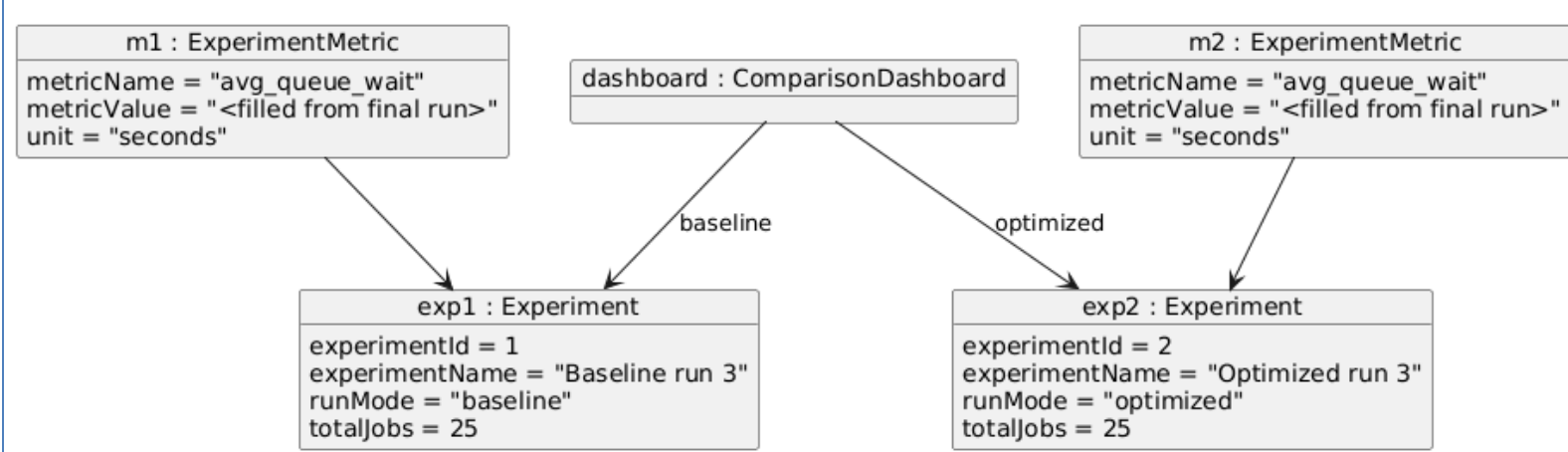


Figure 5.40: Object Diagram for UC-08 (comparison of two experiments)

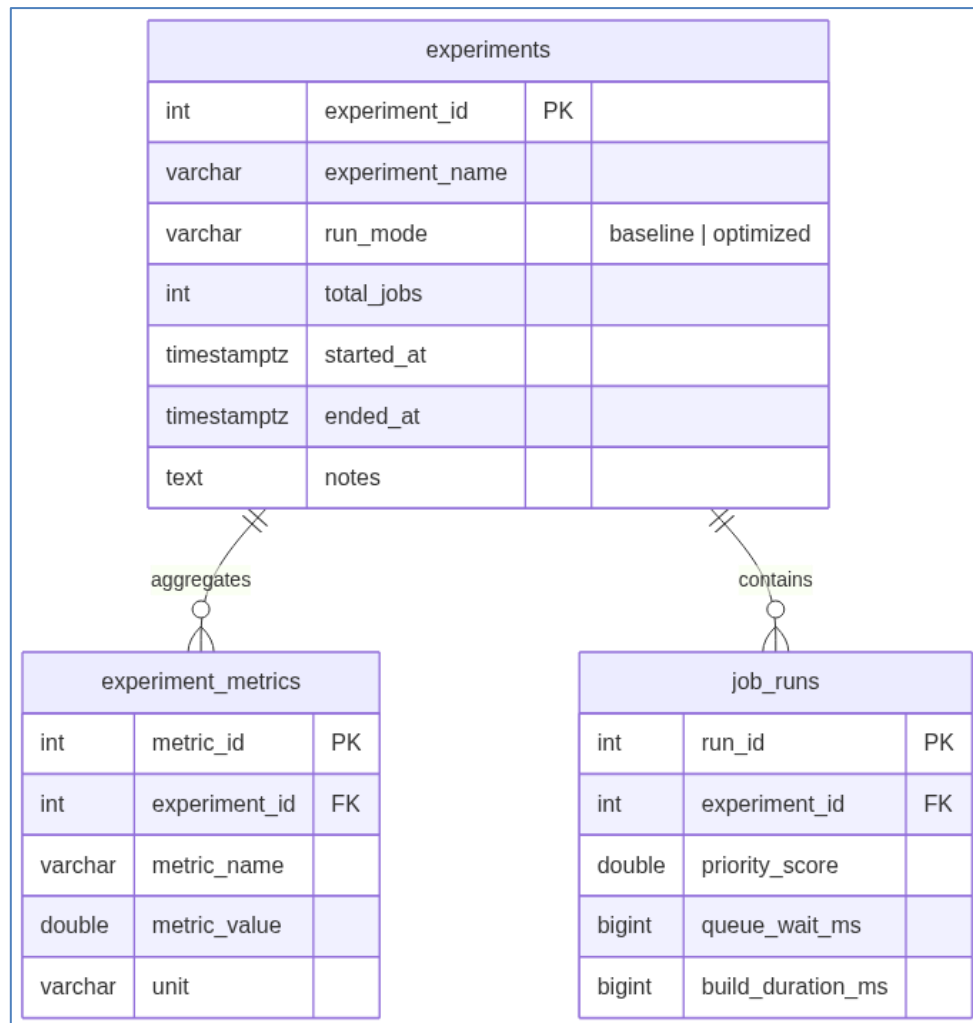


Figure 5.41: ERD fragment for UC-08 (experiments, experiment_metrics, job_runs)

5.12 Landing, Main and Home Interface

The landing interface of the platform is the orchestrator home dashboard, which a user reaches immediately after login. We designed it around the daily workflow of a DevOps engineer: the top bar shows the platform name, the signed-in user, and the global experiment mode (baseline or optimized); the left navigation lists the main areas (Home, Command Console, Submit Job, Queue, Pipelines, Analytics, Admin); and the central area is divided into four cards that answer the four questions a user asks first: what is queued right now (live queue with scores), what is running (active builds with stage progress), how is the system performing (waiting time and throughput sparklines), and what happened recently (last completed runs with status). From this home screen every use case interface of Sections 5.4 to 5.11 is one click away. The same screen serves as the main technical interface during our Milestone 2 demonstration, because it shows the plugin decisions live while the experiment jobs are running.

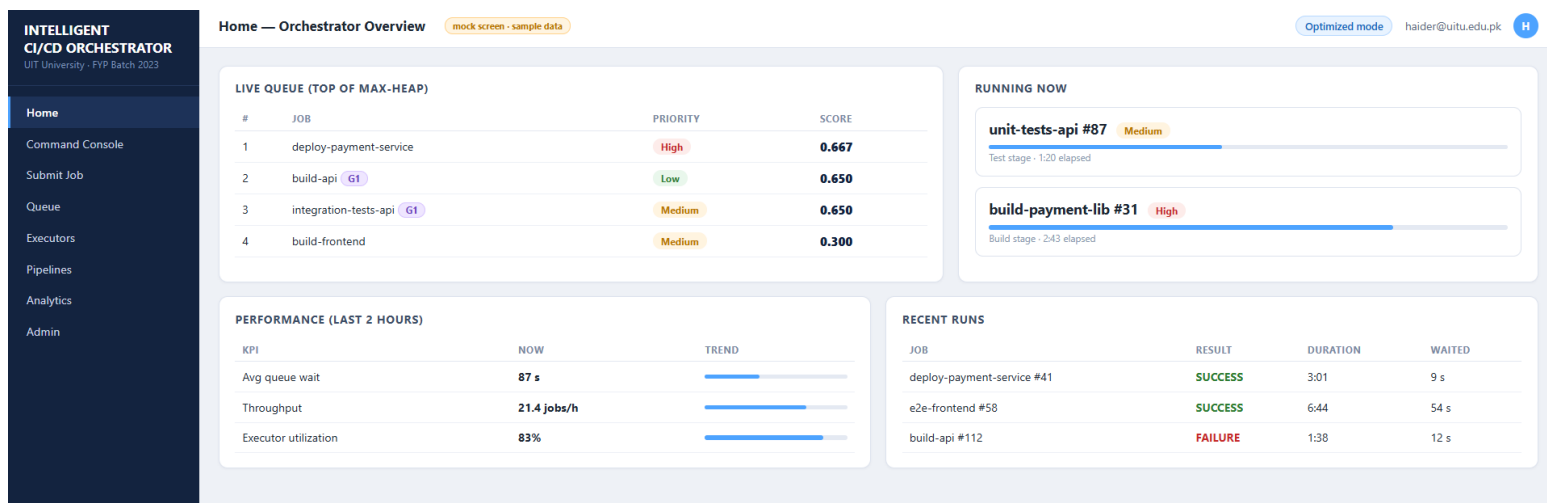


Figure 5.42: Landing / home interface of the Intelligent CI/CD Orchestrator

5.13 Complete System Class Diagram

The complete class diagram combines the per use case diagrams of the previous sections into one view, organized by the four layers of our architecture. The presentation layer contains the React views, the FastAPI controllers, and the plugin boundary classes (PriorityDispatcher, QueueStateListener, PriorityJobProperty, BuildMetricsListener, PriorityConfiguration). The business layer contains the plugin services (PriorityScoringService, DependencyGroupService, ExecutionTimeEstimator, HeapSchedulerService) and the backend services (AuthService, MetricsService, ComparisonService, IntentParsingService, PipelineGenerationService, PipelineValidationService). The persistence layer contains BuildHistoryService and MetricsPublisher on the plugin side and the repositories on the backend side. The entity layer contains the shared models. Because the diagram is large, we print it on a folded A3 sheet as recommended by the report template, and we keep the per use case diagrams as the readable views of the same model.

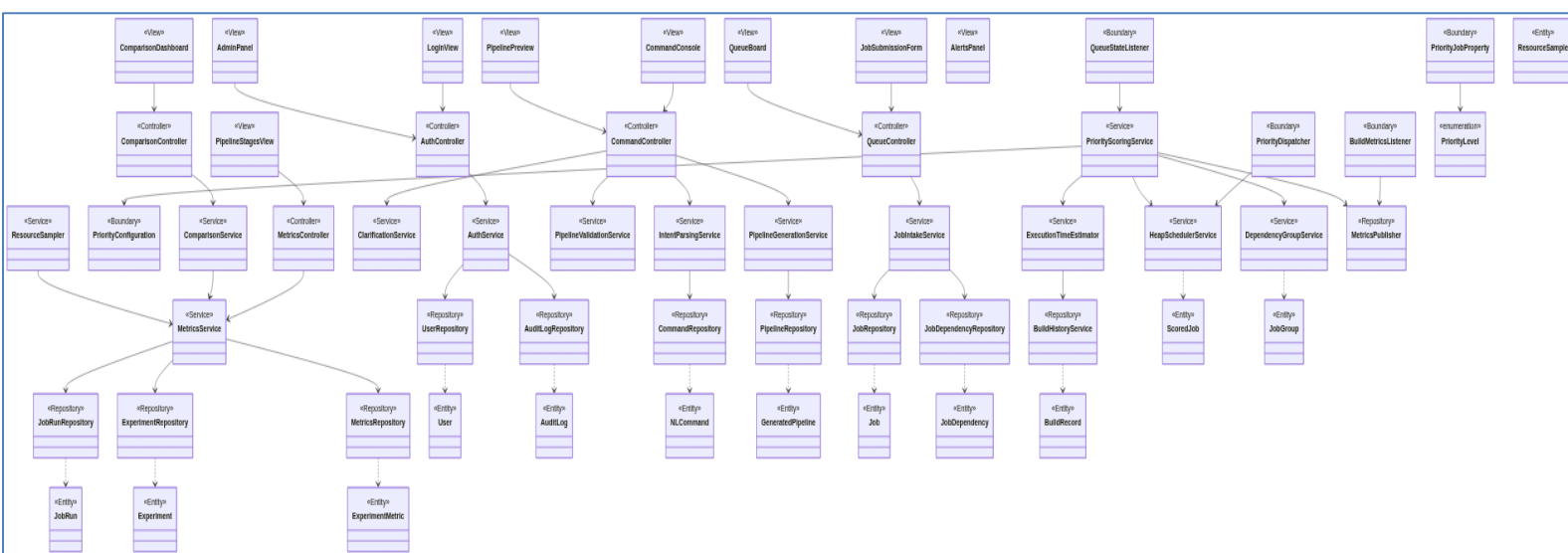


Figure 5.43: Complete class diagram of the Intelligent CI/CD Orchestrator (all layers)

5.14 Complete Database Schema

The complete PostgreSQL schema contains twelve tables. Eight tables belong to the POC: users, jobs, job_dependencies, dependency_groups, experiments, job_runs, experiment_metrics, and resource_samples. Two tables are reserved for the Milestone 3 natural language module (nl_commands, generated_pipelines), and two support cross-cutting concerns (notifications, audit_logs). The central table is job_runs: every

queued execution produces one row holding the run mode (baseline or optimized), the priority level chosen by the user, the three computed factors, the final score, the group reference, and the four timestamps from which all waiting and duration metrics are derived. We normalized the schema so that every metric shown on the dashboard can be recomputed from raw rows, which lets us defend our results during the demonstration instead of showing pre-aggregated numbers that cannot be traced.

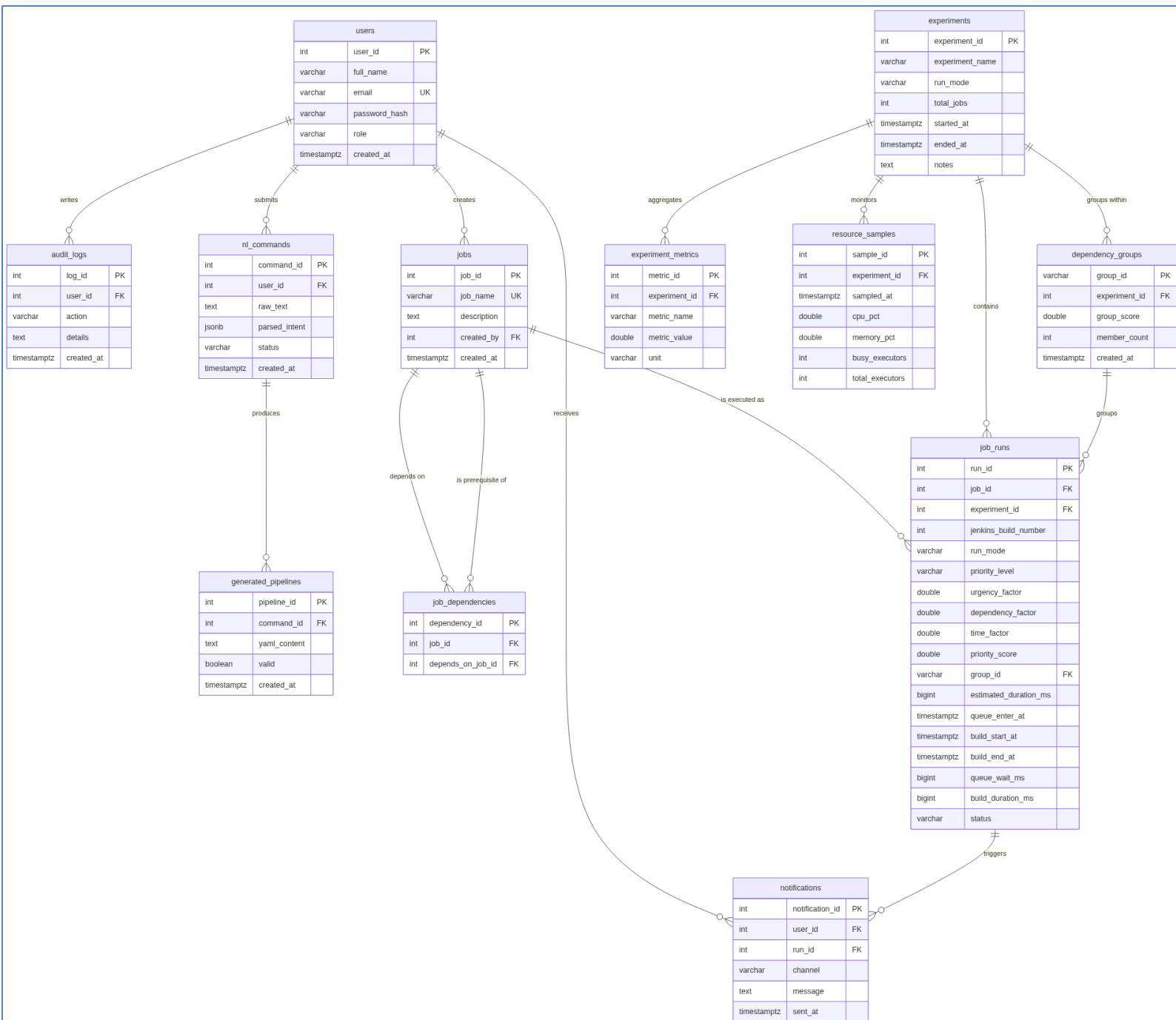


Figure 5.44: Complete database schema of the Intelligent CI/CD Orchestrator (all PostgreSQL tables)

5.15 Chapter Summary

In this chapter we fixed the architectural rules of our system using the Layered Architecture from Richards [22], revised the actor use case diagram to eight use cases that reflect the assigned POC, and designed each use case with its class diagram,

interface, sequence diagram, and database view. The design gives the Dynamic Queue Optimization plugin a precise structure: boundary classes receive Jenkins events, four business services implement grouping, estimation, scoring, and heap scheduling, and persistence classes record every decision so the dashboard can prove the optimization effect. Chapter 6 analyzes the four algorithms behind these services, and the POC implementation follows this design class for class.

6. Algorithm Analysis and Complexity

6.1 Introduction

In this chapter we analyze the four algorithms that our Dynamic Queue Optimization plugin uses: the weighted priority scoring algorithm, the max-heap based scheduling algorithm, the dependency grouping algorithm based on the union-find (disjoint set) structure, and the execution time estimation algorithm based on similarity matching over historical builds. For each algorithm we explain its purpose and significance in the system, give its pseudocode, compare it with alternative algorithms in terms of best, average, and worst case time and space complexity, and justify why we selected it. Throughout the chapter, n denotes the number of jobs waiting in the queue, m the number of declared dependency relations, h the number of historical builds examined for one estimate, f the number of features compared per historical build, and k the number of similar builds used in an estimate.

6.2 Algorithm 1: Weighted Priority Scoring

6.2.1 Significance

The scoring algorithm is the heart of the POC, because it converts three unrelated pieces of information into one number that the scheduler can sort by. Default Jenkins scheduling is first in, first out (FIFO): jobs leave the queue in arrival order regardless of importance, which our problem statement identified as the main cause of urgent deployments waiting behind routine builds. Dynamic priority scoring approaches in the literature address this by assigning each task a score from several weighted factors and keeping the score updated rather than fixed. Our formula combines the three factors required by our POC definition: the urgency that the user expressed (High, Medium, or Low), the dependency factor that raises grouped jobs so a group moves through the queue as one unit, and the execution time factor estimated from history, which gives shorter jobs a small advantage similar to shortest job first scheduling.

The score of a job j is computed as $\text{Score}(j) = 0.5 \cdot U(j) + 0.3 \cdot D(j) + 0.2 \cdot T(j)$, where all three factors are normalized to the range $[0, 1]$. $U(j)$ maps the user priority to 1.0 for High, 0.6 for Medium, and 0.3 for Low. $D(j)$ is the dependency factor $(\text{groupSize} - 1) / (\text{maxGroupSize} - 1)$, which is 0 for an independent job and 1 for the largest group in

the queue. $T(j) = 1 - (\text{est}(j) - \text{estMin}) / (\text{estMax} - \text{estMin})$ rewards shorter estimated duration, and a job with no usable history receives the neutral value 0.5. We chose the weights 0.5, 0.3, 0.2 deliberately: urgency dominates because it is the only factor the user states explicitly, dependency comes second because a blocked group wastes more capacity than a single slow job, and execution time acts as a tie breaker that improves average waiting time without ever letting a short Low job overtake a High job. With these weights the minimum possible score of a High job is 0.5, while the maximum possible score of a Low job is $0.3 \cdot 1 + 0.2 \cdot 1 + 0.5 \cdot 0.3 = 0.65$ only if it heads a maximal group; an independent Low job cannot exceed 0.5 + aging. To prevent starvation of Low jobs under a constant stream of High arrivals we add a small aging bonus of 0.05 for every five minutes a job waits, capped at 0.15, a standard remedy for starvation in priority scheduling.

6.2.2 Pseudocode

Algorithm 6.1: Weighted priority score computation

```

Input : job j with PriorityJobProperty (level, dependsOn),
        queue Q, configuration W = (wU=0.5, wD=0.3, wT=0.2)
Output: score(j) in [0, 1.15]

1  U <- map(level): HIGH -> 1.0 | MEDIUM -> 0.6 | LOW -> 0.3
2  g <- DependencyGroupService.groupOf(j)           // Algorithm 6.3
3  if size(g) <= 1 then D <- 0
4  else D <- (size(g) - 1) / (maxGroupSize(Q) - 1)
5  est <- ExecutionTimeEstimator.estimate(j)        // Algorithm 6.4
6  if est is UNKNOWN then T <- 0.5
7  else T <- 1 - (est - estMin(Q)) / (estMax(Q) - estMin(Q))
8  base <- wU * U + wD * D + wT * T
9  aging <- min(0.15, 0.05 * floor(waitMinutes(j) / 5))
10 score <- base + aging
11 if g is a group then score <- max(score, groupScore(g))
12 return score                                     // ties: earlier
enqueue first

```

6.2.3 Comparison with Alternative Algorithms

We compared our weighted scoring approach with three alternatives: plain FIFO (the Jenkins default), static multilevel priority queues in which a job's class never changes after submission, and a machine learning ranking model that learns scores from past scheduling outcomes. The comparison considers the cost of making one scheduling decision for one job and the memory needed beyond the queue itself.

Table 6.1: Comparison of priority assignment algorithms (per job decision)

Algorithm	Best time	Average time	Worst time	Space	Remarks
FIFO (Jenkins default)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	No prioritization at all; urgent jobs wait full queue length
Static multilevel priority	$O(1)$	$O(1)$	$O(1)$	$O(L)$ levels	Priorities never adapt; starvation of low levels without aging
Weighted scoring (ours)	$O(1)^*$	$O(1)^*$	$O(n)$	$O(1)$ per job	*after factors known; worst case is renormalizing T over n jobs
ML based ranking	$O(f)$	$O(f)$	$O(f)$	$O(\text{model})$	Needs training data and offline training; not explainable to examiners

The score itself is a constant time arithmetic expression once its three inputs are available, so the real costs live in Algorithms 6.3 and 6.4 and are analyzed there. The worst case $O(n)$ appears when a new minimum or maximum estimate arrives and the T factor of all n waiting jobs must be renormalized; with the 20 to 30 jobs of our experiment this is negligible. We rejected FIFO because it is the baseline we are trying to beat, static multilevel queues because they cannot express dependency grouping or execution time at all, and ML ranking because it requires training data that does not exist before the system runs and because a learned model cannot be justified factor by factor during a viva. A transparent weighted formula matches the dynamic priority scoring approach.

6.3 Algorithm 2: Max-Heap Based Scheduling

6.3.1 Significance

Scoring is useless without a structure that can always hand over the highest scored job quickly while jobs keep arriving and leaving. We keep all waiting jobs in a binary max-heap keyed by score, the classical priority queue structure introduced by Williams with the heapsort algorithm. The heap property guarantees that the root always holds the maximum score, so the dispatcher's question, which job should run next, is answered in constant time, and the expensive operations, insertion and removal, cost only logarithmic time. The heap is rebuilt or updated whenever scores change (for example

through aging), which the literature on heap based task ordering also recommends for dynamic scheduling.

6.3.2 Pseudocode

Algorithm 6.2: Max-heap insertion and extraction for job scheduling

```

Structure: array  $H[1..n]$ ,  $\text{parent}(i) = \text{floor}(i/2)$ , children  $2i, 2i+1$ 
Invariant:  $H[\text{parent}(i)].\text{score} \geq H[i].\text{score}$  (max-heap property)

INSERT( $H, \text{job}$ ):
1   $n \leftarrow n + 1$ ;  $H[n] \leftarrow \text{job}$ ;  $i \leftarrow n$ 
2  while  $i > 1$  and  $H[\text{parent}(i)].\text{score} < H[i].\text{score}$ :
3      swap  $H[i], H[\text{parent}(i)]$ ;  $i \leftarrow \text{parent}(i)$  // sift-up

PEEK-MAX( $H$ ): return  $H[1]$  //  $O(1)$ 

EXTRACT-MAX( $H$ ):
1   $\text{top} \leftarrow H[1]$ ;  $H[1] \leftarrow H[n]$ ;  $n \leftarrow n - 1$ ;  $i \leftarrow 1$ 
2  while true: // sift-down
3       $l \leftarrow 2i$ ;  $r \leftarrow 2i+1$ ;  $\text{largest} \leftarrow i$ 
4      if  $l \leq n$  and  $H[l].\text{score} > H[\text{largest}].\text{score}$ :  $\text{largest} \leftarrow l$ 
5      if  $r \leq n$  and  $H[r].\text{score} > H[\text{largest}].\text{score}$ :  $\text{largest} \leftarrow r$ 
6      if  $\text{largest} = i$ : break
7      swap  $H[i], H[\text{largest}]$ ;  $i \leftarrow \text{largest}$ 
8  return top

RESCORE( $H$ ): for each job: recompute score (Alg. 6.1),
              then BUILD-MAX-HEAP in  $O(n)$  [25]

```

6.3.3 Comparison with Alternative Algorithms

Table 6.2: Comparison of priority queue structures for n waiting jobs

Structure	Insert (best/avg/worst)	Peek max	Extract max (best/avg/worst)	Space	Remarks
Unsorted list	$O(1) / O(1) / O(1)$	$O(n)$	$O(n) / O(n) / O(n)$	$O(n)$	Every dispatch scans whole queue
Sorted array	$O(1) / O(n) / O(n)$	$O(1)$	$O(1) / O(1) / O(1)$	$O(n)$	Insertion shifts elements; bad under bursts
Red-black BST	$O(\log n)$ all cases	$O(\log n)$	$O(\log n)$ all cases	$O(n)$	Balanced but higher constants, complex code [25]
Binary max-heap (ours)	$O(1) / O(\log n) / O(\log n)$	$O(1)$	$O(\log n)$ all cases	$O(n)$	Array based, cache friendly, simple [25], [29]

Structure	Insert (best/avg/worst)	Peek max	Extract max (best/avg/worst)	Space	Remarks
Fibonacci heap	$O(1)$ amortized	$O(1)$	$O(\log n)$ amortized	$O(n)$	Wins only with decrease-key heavy workloads [25]

We selected the binary max-heap because our workload is dominated by exactly the operations a heap is best at: frequent insertions as jobs arrive, one peek per dispatcher query, and one extraction per dispatched job. The unsorted list makes every dispatch $O(n)$, which contradicts the goal of fast decisions. The sorted array degrades to $O(n)$ per insertion during burst submissions, which is precisely our stress scenario. A red-black tree matches the heap asymptotically but is considerably harder to implement correctly inside a Jenkins plugin and has larger constant factors, with no benefit since we never need ordered traversal of all jobs. The Fibonacci heap has better amortized insertion in theory but its advantage materializes only in algorithms with many decrease-key operations such as Dijkstra's algorithm, not in our pattern of full rescore, and its implementation complexity is far out of proportion for 20 to 30 jobs. With $n = 30$ the heap needs at most five comparisons per insertion, and Java's PriorityQueue gives us a production tested implementation that we adapt with a custom comparator.

6.4 Algorithm 3: Dependency Grouping with Union-Find

6.4.1 Significance

Our POC requires that two or more jobs that depend on each other are treated and executed as a single logical unit rather than scheduled independently. If job B needs the artifact of job A, scheduling B before A either fails B or blocks an executor while B waits. We therefore partition the waiting jobs into dependency groups: whenever a dependency A before B is declared, the groups of A and B are merged. The natural data structure for maintaining disjoint sets under repeated merging is the union-find (disjoint set) structure with union by rank and path compression, whose amortized cost per operation is the inverse Ackermann function $\alpha(n)$, effectively constant for any realistic input. Inside each group we order members topologically, so producers run before consumers, and the whole group inherits the score of its highest member (computed with Algorithm 6.1), which moves the unit through the queue together.

6.4.2 Pseudocode

Algorithm 6.3: Dependency grouping with union-find and group ordering

```

State: parent[x], rank[x] for every job x; edges E = declared pairs (a -
> b)

FIND(x):                                     // path compression
1   if parent[x] != x: parent[x] <- FIND(parent[x])
2   return parent[x]

UNION(a, b):                                // union by rank
1   ra <- FIND(a); rb <- FIND(b)
2   if ra = rb: return
3   if rank[ra] < rank[rb]: swap ra, rb
4   parent[rb] <- ra
5   if rank[ra] = rank[rb]: rank[ra] <- rank[ra] + 1

GROUP-JOBS(queue Q, edges E):
1   for each job x in Q: parent[x] <- x; rank[x] <- 0
2   for each (a -> b) in E: UNION(a, b)
3   groups <- partition of Q by FIND(x)
4   for each group g with |g| > 1:
5       order(g) <- KAHN-TOPOLOGICAL-SORT(g, E) // detects cycles [30]
6       if cycle detected: reject submission (UC-04 validation)
7       score(g) <- max over members of score (Alg. 6.1)
8   return groups

```

6.4.3 Comparison with Alternative Algorithms

Table 6.3: Comparison of grouping approaches for n jobs and m dependency edges

Approach	Best time	Average time	Worst time	Space	Remarks
Union-find, rank + path compression (ours)	$O(n + m \cdot \alpha(n))$	$O(n + m \cdot \alpha(n))$	$O(n + m \cdot \alpha(n))$	$O(n)$	$\alpha(n) \leq 4$ for all practical n [25], [27]
DFS/BFS connected components	$O(n + m)$	$O(n + m)$	$O(n + m)$	$O(n + m)$	Must re-run fully on every new edge or job arrival
Adjacency matrix closure	$O(n^2)$	$O(n^2)$	$O(n^3)$ (Warshall)	$O(n^2)$	Simple but quadratic memory, cubic closure
No grouping (naive)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Violates POC requirement; deadlocks possible

A single DFS pass is asymptotically equal for one static graph, and this is the honest comparison: if the queue never changed, DFS would be just as good. Our queue,

however, changes constantly. Jobs and dependency declarations arrive one by one, and union-find absorbs each new edge in amortized $\alpha(n)$ time without recomputing anything, while DFS based components must be recomputed from scratch (or maintained with much more complex incremental algorithms) on every arrival. The adjacency matrix needs $O(n^2)$ memory and a cubic closure computation, unjustifiable for our sizes and wasteful at any size. We accept one limitation consciously: union-find treats dependencies as undirected for the purpose of grouping, so it tells us which jobs belong together but not their order; Kahn's algorithm restores the order inside each group in $O(\text{size} + \text{edges})$ and simultaneously detects cyclic declarations, which we reject at submission time. The combination gives us constant time bookkeeping in the hot path of the scheduler and a clean validation story.

6.5 *Algorithm 4: Execution Time Estimation by Similarity Matching*

6.5.1 Significance

The third scoring factor requires an estimated execution time for a job that has not run yet, and our POC definition explicitly forbids asking the user for it. The only honest source is history: Jenkins keeps every past build with its duration, parameters, and result, and our BuildHistoryService reads this history through the Jenkins API. The estimation question is therefore: given a new job, which past builds are similar enough to predict its duration, a problem the scheduling literature also solves with historical data of similar tasks. Our algorithm is an instance based (nearest neighbor) method in the spirit of k-NN classification, we compute a similarity between the new job and each historical build from three features (Jaccard similarity of the tokenized job names, weight 0.5; Jaccard similarity of the parameter sets, weight 0.3; agent label match, weight 0.2), select the $k = 5$ most similar builds above a minimum similarity threshold, and return their recency weighted average duration. The weight of each selected build is its similarity multiplied by an exponential decay $e^{(-\lambda \cdot \text{age})}$ with $\lambda = 0.1$ per day, so a build from yesterday influences the estimate far more than an equally similar build from two months ago. If the identical job has run before, its own recent builds dominate automatically because their name similarity is 1.0. If nothing clears the threshold, the

estimator returns UNKNOWN and the scoring formula uses the neutral factor 0.5, so a brand new job is neither punished nor favored.

We chose a weighted average over k builds instead of simply copying the single most similar build for a statistical reason: a single historical duration is noisy (caches, network, test flakiness), and averaging k of them reduces the variance of the estimate while the similarity and recency weights keep it relevant. This is also why we preferred top- k averaging over an exponentially weighted moving average (EWMA) of the job's own series, the textbook method for predicting CPU burst lengths in shortest job first scheduling. EWMA works only for a job that already has its own history and ignores the cross job similarity that our SaaS scenario needs, where many pipelines are near copies of each other.

6.5.2 Pseudocode

Algorithm 6.4: Recency-weighted top- k similarity estimation

```
Input : new job  $j$ , history  $B = \{(\text{features}_i, \text{duration}_i, \text{age}_i)\}$  of  $h$ 
builds,
       $k = 5$ , threshold = 0.35, decay  $\lambda = 0.1$  per day
Output: estimated duration in ms, or UNKNOWN

1  C <- empty list                                // candidates
2  for each build  $b$  in  $B$ :                          //  $O(h * f)$ 
3      simName <- jaccard(tokens( $j$ .name), tokens( $b$ .name))
4      simParam <- jaccard(params( $j$ ), params( $b$ ))
5      simLabel <- 1 if  $j$ .agentLabel =  $b$ .agentLabel else 0
6      sim <-  $0.5 * \text{simName} + 0.3 * \text{simParam} + 0.2 * \text{simLabel}$ 
7      if sim  $\geq$  threshold: append ( $b$ , sim) to C
8  if C is empty: return UNKNOWN
9  K <-  $k$  elements of C with largest sim            // partial select
10 for each ( $b$ , sim) in K:
11      $w_b$  <- sim *  $\exp(-\lambda * b.\text{age\_days})$     // recency weight
12 return  $\text{sum}(w_b * b.\text{duration}) / \text{sum}(w_b)$     // weighted mean
```

6.5.3 Comparison with Alternative Algorithms

Table 6.4: Comparison of execution time estimation methods (h historical builds, f features)

Method	Best time	Average time	Worst time	Space	Remarks
Last build duration	$O(1)$	$O(1)$	$O(1)$	$O(1)$	One outlier build corrupts the estimate
Global mean of job history	$O(h)$	$O(h)$	$O(h)$	$O(1)$	Ignores parameters; stale builds weigh equally
EWMA (exponential averaging)	$O(1)$ update	$O(1)$	$O(1)$	$O(1)$	Needs the job's own history; no cross job transfer [26]
Recency-weighted top-k similarity (ours)	$O(h \cdot f)$	$O(h \cdot f + h \log k)$	$O(h \cdot f + h \log k)$	$O(h \cdot f)$	Transfers knowledge across similar jobs [28]
Regression / ML model	$O(h \cdot f)$ predict	$O(h \cdot f^2)$ train	$O(h \cdot f^2)$ train	$O(h \cdot f)$	Needs feature engineering and training set; opaque

Our method costs $O(h \cdot f)$ per estimate because every historical build is compared once, plus the selection of the top k . We bound h by fetching at most the last 50 builds per candidate job, so an estimate costs at most a few thousand cheap set operations, well under a millisecond, and estimates are cached per job and refreshed only when a new build of a similar job completes. The cheap alternatives lose exactly the property our POC is graded on: last build and global mean produce visibly wrong estimates after one anomalous build, and EWMA cannot say anything about a job's first run even when an almost identical job has run a hundred times. A trained regression model could in principle beat us, but it needs a labeled training corpus that does not exist on a fresh Jenkins installation, and our cold start path (UNKNOWN, neutral factor) degrades gracefully instead. The k -NN approach is the established baseline for exactly this transfer situation, and it keeps every estimate explainable: the dashboard can show the five historical builds and weights behind any number it displays.

6.6 End-to-End Complexity of the Scheduling Pipeline

Combining the four algorithms, the total cost of admitting one job into the optimized queue is $O(m \cdot \alpha(n))$ for group maintenance, $O(h \cdot f)$ for the estimate, $O(1)$ for the score, and $O(\log n)$ for the heap insertion, which is $O(h \cdot f + \log n)$ in practice since the union-find term is effectively constant. One dispatch decision costs $O(1)$ for the peek and $O(\log n)$ for the extraction, and a full periodic rescore (for aging) costs $O(n)$ heap rebuild after $O(n)$ score updates, run every 60 seconds. Space consumption is $O(n)$ for the heap and union-find plus $O(h \cdot f)$ for the per-estimate candidate set. For the experiment scale of this POC (n up to 30, h up to 50, $f = 3$) every operation completes in microseconds to a fraction of a millisecond, so the plugin adds no measurable overhead to Jenkins queue maintenance, which we verify experimentally in the testing chapter.

Table 6.5: Combined complexity summary of the optimized scheduler

Operation	Time complexity	Frequency
Job admission (group + estimate + score + insert)	$O(m \cdot \alpha(n) + h \cdot f + \log n)$	Once per submitted job
Dispatch decision (peek + extract on start)	$O(1)$ peek, $O(\log n)$ extract	On every queue maintenance cycle
Aging rescore + heap rebuild	$O(n)$	Every 60 seconds
Group order check (Kahn, per group)	$O(\text{size} + \text{edges})$	Once per group change

6.7 Chapter Summary

In this chapter we analyzed the four algorithms of our POC and justified each selection against named alternatives with their best, average, and worst case time and space complexities. The weighted scoring formula was chosen over FIFO, static priorities, and learned ranking for adaptivity and explainability; the binary max-heap over lists, balanced trees, and Fibonacci heaps for its fit to our operation mix; union-find with path compression over repeated graph traversals for its near constant incremental cost, with Kahn's algorithm providing order and cycle detection inside groups; and recency-weighted top-k similarity over naive history statistics for robustness and cross job transfer, with a safe neutral fallback for cold starts. The configured weights (0.5, 0.3, 0.2, aging 0.05 per 5 minutes capped at 0.15, $k = 5$, $\lambda = 0.1$) are stored in

PriorityConfiguration so the values can be tuned during the experiments without recompiling the plugin.

12. Achievements

12.1 Overview

This chapter records what our team has achieved up to the second milestone of the project. We present the achievements of Milestone 1 briefly for completeness and then describe the Milestone 2 achievements in detail, since they contain the assigned Proof of Concept, the Dynamic Queue Optimization plugin for Jenkins.

12.2 Achievements of Milestone 1

- We completed and submitted Chapters 1 to 4 of this report: the introduction, a literature review of 21 referenced sources covering LLM driven CI/CD automation and queue optimization, the aim and statement of the problem, and the hardware and software analysis with justified technology choices.
- We produced the initial system diagram, the activity diagram with swimlanes covering the full flow from natural language command to monitored execution, and the actor use case diagram with twelve use cases.
- We presented a non-working prototype of the platform and defended the project scope in front of the FYP committee, after which the Dynamic Queue Optimization module was assigned to us as the Proof of Concept for Milestone 2.

12.3 Achievements of Milestone 2

12.1.1 Design and Modeling

- We adopted a documented industry coding architecture, the Layered Architecture described by Richards , and restructured all of our class designs to follow its four class types, as presented in Chapter 5.
- We revised the actor use case diagram from twelve to eight use cases that reflect the assigned POC, and for every use case we produced the four required design artifacts: a class diagram conforming to the architecture, an interface design, a sequence diagram whose objects match the class diagram, and an object diagram with an ERD wherever the use case touches PostgreSQL.
- We completed the system level views: the complete class diagram across all four layers and the complete database schema with twelve tables.

12.1.2 Algorithms

- We designed, wrote pseudocode for, and analyzed the four algorithms of the POC (weighted priority scoring, max-heap scheduling, union-find dependency grouping with topological ordering, and recency-weighted top-k similarity estimation), and we justified each choice against named alternatives with best, average, and worst case time and space complexities in Chapter 6.

12.1.3 Proof of Concept Implementation

- We set up the Jenkins plugin development environment with the official Maven archetype and built a working custom plugin for the current Jenkins LTS line that intercepts the build queue through the `QueueTaskDispatcher` extension point.
- The plugin implements the full optimization workflow assigned to us: a job property through which the user selects High, Medium, or Low priority and declares dependencies; conversion of the priority into a numeric urgency value; dependency checking with grouping of dependent jobs into single logical units; automatic execution time estimation from historical Jenkins build data with no manual time input; computation of the weighted priority score; and dynamic ordering of the queue through a max-heap so that the highest scoring job (or group) is dispatched to the executor first.
- We built the metrics pipeline that records every scheduling decision and build outcome into PostgreSQL through a FastAPI backend, and the React analytical dashboard that visualizes the live queue with its scores and the side by side comparison of baseline and optimized experiment runs.
- We defined a repeatable experiment of 25 jobs with mixed priorities, durations, and three dependency chains (the protocol is given in Appendix B), which we execute once under default Jenkins scheduling and once with our plugin enabled to measure queue waiting time, build completion time, throughput, and resource utilization. Table 12.1 holds the measured results of the final experiment run; we fill it from the dashboard export before submission and present the same numbers during the demonstration.

Table 12.1: Baseline versus optimized results of the final experiment run (to be completed from the dashboard export before submission)

KPI	Baseline (default Jenkins)	Optimized (our plugin)	Change (%)
Average queue waiting time (s)			
Maximum queue waiting time (s)			
Average build completion time (s)			
Average waiting time of High priority jobs (s)			
Throughput (jobs per hour)			
Average executor utilization (%)			

12.1.4 Skills and Process

- Technically, we learned Jenkins plugin development in Java (extension points, job properties, global configuration, listeners), the Jenkins build and queue APIs, practical complexity analysis, and full stack integration between a Java plugin, a Python backend, and a React frontend.
- As a team, we worked with a shared Git repository using feature branches and reviewed merges, kept the report, diagrams, and code consistent through a single naming registry, and held weekly progress meetings with our supervisor, Engr. Laiba Mughal, whose feedback shaped the revised use case model.

13. Appendices

13.1 Appendix A: Glossary of Terms

Table 13.1: Glossary of technical terms used in this report

Term	Meaning in this project
CI/CD	Continuous Integration / Continuous Deployment, the automated building, testing, and releasing of software.
Jenkins controller	The central Jenkins server process that owns the queue, configuration, and build history.
Agent / executor	A machine (agent) connected to the controller, offering one or more execution slots (executors) that run builds.
Job / build	A job is the stored definition of a pipeline; a build (or run) is one execution of that job.
Queue item	A job waiting in the Jenkins build queue before an executor accepts it.
Extension point	A Java class Jenkins exposes so plugins can change its behavior by subclassing.
QueueTaskDispatcher (QTD)	The extension point consulted for every queued item; returning null allows execution, returning a cause of blockage holds the item [32].
JobProperty	Plugin supplied configuration attached to a job; our PriorityJobProperty stores priority and dependencies.
LTS	Long Term Support release line of Jenkins, patched for stability; our target platform.
Max-heap	A binary tree stored in an array where every parent outranks its children, so the maximum is always at the root.
Union-find	A disjoint set structure that merges groups and finds group membership in near constant amortized time.
Jaccard similarity	Size of the intersection of two sets divided by the size of their union; 1 means identical sets.
Baseline run	The experiment executed with default Jenkins FIFO scheduling, our plugin disabled.
Optimized run	The same experiment executed with our plugin enabled.
Makespan	Total wall clock time from the first job submission to the last job completion of an experiment.

13.2 Appendix B: Baseline versus Optimized Experiment Protocol

The purpose of the experiment is to prove whether the plugin actually optimizes the workflow. We run an identical workload twice and compare the metrics. The protocol below makes the two runs repeatable and fair.

1. Environment. One Jenkins controller (current LTS) with two executors on one agent, Docker based, with the FastAPI metrics backend and PostgreSQL running alongside. The same containers and resources are used for both runs.
2. Workload. 25 freestyle/pipeline jobs: 8 High, 9 Medium, 8 Low priority. Job bodies are scripted sleeps and small compile/test tasks with target durations between 30 seconds and 4 minutes. Three dependency chains are declared: build-api -> unit-tests-api -> integration-tests-api; build-frontend -> e2e-frontend; build-payment-lib -> deploy-payment-service. The remaining 18 jobs are independent.
3. History seeding. Before the measured runs, every job is executed three times so that the estimator has historical data; these warm-up builds are excluded from the results.
4. Submission. A script submits all 25 jobs through the Jenkins REST API in a fixed order within a five minute window, identical in both runs, so arrival patterns cannot bias the comparison.
5. Baseline run. The plugin is disabled (Jenkins default FIFO order). The metrics listener still records queue_enter_at, build_start_at, build_end_at, status, and resource samples for every run into PostgreSQL under experiment mode 'baseline'.
6. Optimized run. Jenkins build history is reset to the same seeded state, the plugin is enabled, and the identical submission script is executed under experiment mode 'optimized'.
7. Analysis. The dashboard computes the KPIs of Appendix F for both experiments and renders the side by side comparison; the export of that view fills Table 12.1.
8. Repetition. The pair of runs is executed three times on different days; we report the median pair to absorb machine noise.

13.3 Appendix C: Worked Example of the Priority Score

The following snapshot shows four queued jobs and every intermediate value of Algorithm 6.1, with weights 0.5, 0.3, 0.2. Estimates are in minutes; estMin = 2.0 and

estMax = 8.0 over the queue; the largest group has 2 members; no aging has accumulated yet.

Table 13.2: Worked scoring example for a queue of four jobs

Job	Priority	Group	Est (min)	U	D	T	Score = $0.5U + 0.3D + 0.2T$
deploy-payment-service	High	none	3.0	1.0	0	$1-(3-2)/6 = 0.833$	$0.500 + 0 + 0.167 = 0.667$
build-api	Low	G1	2.0	0.3	1.0	1.000	$0.150 + 0.300 + 0.200 = 0.650$
integration-tests-api (needs build-api)	Medium	G1	7.0	0.6	1.0	$1-(7-2)/6 = 0.167$	$0.300 + 0.300 + 0.033 = 0.633$
build-frontend	Medium	none	8.0	0.6	0	0.000	$0.300 + 0 + 0 = 0.300$

Group G1 takes the maximum score of its members, 0.650, and is ordered internally by Kahn's algorithm as build-api before integration-tests-api. The heap therefore dispatches: deploy-payment-service (0.667), then the unit G1 (build-api, integration-tests-api), then build-frontend (0.300). Under FIFO with arrival order build-frontend, build-api, integration-tests-api, deploy-payment-service, the High priority deployment would have waited behind an eight minute Medium build; with our plugin it runs first. The example also shows the safeguard discussed in Chapter 6: the Low job build-api overtakes only because it unblocks a group whose best member scores 0.633, not on its own merit.

13.4 Appendix D: Plugin Configuration Sample

The plugin contributes one job level configuration block and one global configuration page. On the job configuration page, the user sets the priority and dependencies; the equivalent pipeline syntax is shown below for a Jenkinsfile based job.

Algorithm 13.1: Job level configuration (Jenkinsfile properties step)

```
properties([
    dynamicQueuePriority(
        level: 'HIGH',           // HIGH | MEDIUM | LOW
        dependsOn: 'build-api'   // comma separated upstream job
    )
    names
])
```

Algorithm 13.2: Global configuration defaults (Manage Jenkins > Dynamic Queue Optimizer)

```

weights:      urgency = 0.5,  dependency = 0.3,  executionTime = 0.2
aging:        bonus = 0.05 per 5 minutes waiting, cap = 0.15
estimator:    k = 5,  similarityThreshold = 0.35,  recencyLambda =
0.1/day,
              historyWindow = 50 builds
metrics:      backendUrl = http://backend:8000/api/metrics,  enabled =
true
mode:         optimizerEnabled = true    // false during the baseline
run

```

13.5 Appendix E: Tools and Versions

Table 13.3: Tools and versions used for the Milestone 2 POC

Tool / technology	Version line	Role in the POC
Jenkins (LTS)	2.541.x LTS	CI/CD engine whose queue we optimize
Java (Temurin JDK)	21	Language of the Jenkins plugin
Apache Maven + plugin parent POM	3.9.x	Build and packaging of the plugin (.hpi) [33]
JUnit + Jenkins Test Harness	5.x / 2.x	Unit and integration tests of the plugin
Python + FastAPI	3.12 / 0.11x	Metrics backend (REST API)
PostgreSQL	16	Persistent storage of runs, experiments, metrics
Redis	7	Live queue state cache for the dashboard
React + Recharts	18 / 2.x	Analytical dashboard and comparison charts
Docker / Docker Compose	24.x	Reproducible environment for both experiment runs
Git + GitHub	-	Version control, feature branch workflow

13.6 Appendix F: KPI Definitions and Formulas

Table 13.4: Definitions of the key performance indicators compared on the dashboard

KPI	Formula (per experiment)	Interpretation
Queue waiting time	build_start_at - queue_enter_at, per run; report mean and max	How long jobs sit in the queue; the primary target of the POC
Build completion time	build_end_at - queue_enter_at, per run; report mean	Total turnaround a user experiences
Throughput	completed runs / makespan in hours	Jobs the system finishes per hour
Executor utilization	sum of busy executor seconds / (executors x makespan)	Share of available capacity doing useful work
High priority waiting time	mean queue waiting time over runs with priority = High	Whether urgency is actually honored

KPI	Formula (per experiment)	Interpretation
Scheduling efficiency	percentage change of each KPI: (baseline - optimized) / baseline x 100	Single comparable improvement figure per KPI

References

- [1] M. T. Ribeiro et al., “Integrating AI Planning with Natural Language Processing,” *arXiv preprint arXiv:2202.07138*, 2022.
- [2] S. S. et al., “IntentBased Networking Using Large Language Models,” *arXiv preprint arXiv:2403.02238*, 2024
- [3] A. Kumar et al., “L2CEval: Evaluating Language to Code Generation Capabilities,” *arXiv preprint arXiv:2309.17446*, 2023
- [4] J. Li et al., “JSONSchemaBench: Structured Output Generation for Large Language Models,” *arXiv preprint arXiv:2501.10868*, 2025.
- [5] M. Johnson et al., “Toward Automatically Completing GitHub Workflows,” *arXiv preprint arXiv:2308.16774*, 2023.
- [6] R. Patel et al., “CloudEvalYAML: A Benchmark for Cloud Configuration Generation,” *arXiv preprint arXiv:2401.06786*, 2024.
- [7] K. Smith et al., “Can LLMs Write CI? Evaluating CI/CD Pipeline Generation,” *arXiv preprint arXiv:2507.17165*, 2025.
- [8] T. Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools,” *arXiv preprint arXiv:2302.04761*, 2023.
- [9] S. Yao et al., “ReAct: Synergizing Reasoning & Acting in Language Models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [10] H. Zhang et al., “Action Engine: ToolAugmented LLMs for Workflow Generation,” *arXiv preprint arXiv:2411.19485*, 2024.
- [11] L. Wang et al., “AutoFlow: Automating Workflow Generation Using LLMs,” *arXiv preprint arXiv:2407.12821*, 2024.
- [12] M. Ali et al., “LLMBased Chatbot for CI/CD Knowledge Retrieval,” *arXiv preprint arXiv:2408.09277*, 2024.
- [13] A. Askholm, K. Johnsen, & J. Mauro, “Portable & Secure CI/CD for COBOL: Lessons from an Industrial Migration,” *arXiv preprint*, 2026.
- [14] G. Bournassenko, “Queueing Theory for CI/CD Pipeline Optimization,” *Medium*, 2023
- [15] IEEE, “A Novel ResourceAware Scheduling with MultiCriteria for Heterogeneous Systems,” *IEEE Xplore*, 2023.
- [16] R. Dalmia et al., “Dynamic Ready Queue Based Process Priority Scheduling Algorithm,” Jaypee University of Information Technology, Solan, India, 2022.
- [17] IEEE, “Queue Wait Time Prediction in High Performance Computing Systems,” *IEEE Xplore*, 2023.
- [18] A. Verma et al., “Dynamic Priority Scheduling Algorithms for Flexible Task Management in Cloud Computing,” 2023.

- [19] S. Kumar et al., “Task Scheduling in Cloud Computing: A Priority-based Heuristic Approach,” 2022.
- [20] N. Okafor, B. Lusch, and V. Vishwanath, “Queue Wait Time Prediction in High Performance Computing (HPC) Systems,” *Journal of Supercomputing*, 2026.
- [21] S. Lipsa, R. K. Dash, N. Ivković, and K. Cengiz, “Task Scheduling in Cloud Computing: A Priority-Based Heuristic Approach,” *IEEE Access*, vol. 11, pp. 27111–27126, 2023